

When Two Threads Are Better Than One: How Matching-Engine Transactions Cause Long Tails in Market-Data Handlers

Vincent Maciejewski*

September 2026

Abstract

The prevailing view in high-frequency-trading (HFT) system design is that the entire hot path — market-data decode, order-book application, strategy evaluation, order construction — should run on a single-threaded event loop, because each additional thread hop adds wall-clock latency. This paper puts two questions to that view, on more than a year of CME market data for the NQ front-month contract. First, what produces the queueing tail behind a single-threaded receiver: the packet rate, the clustering of arrivals, or the number of messages per packet? Second, is a servicing chain cut into stages on separate threads better than one thread, and when?

On the first question: the tail is produced by the timing of the matching engine's transactions and, at the service times production receivers run at, by the number of messages each packet carries. It is not produced by the rate: utilisation is tiny at HFT service times, and the busiest windows have the same tail, relative to the service time, as the quietest. Nor is it produced by how the exchange packs transactions into packets: almost every transaction fits in one packet, and the short gaps that make a receiver queue lie between different transactions. Putting transactions at random moments removes most of the tail; changing their sizes or their packing does not. The exchange's own timestamps show where the short gaps come from. The matching engine often processes consecutive transactions within a fraction of a microsecond of each other; the exchange's market-data publisher is a queue that sends about one packet every $7.5\mu\text{s}$, so a burst at the engine reaches the receiver as a train of packets at that spacing. A receiver is therefore the second stage of a tandem that starts at the exchange, and its tail threshold is the publisher period. Why orders reach the engine within a microsecond of each other, which fits many participants racing on the same signal, is a question about the market rather than the receiving software, and is left open.

On the second question: yes above a threshold and no below it, and the threshold is the publisher period. Below it there is no queue to shorten and a cut only adds a hop to every message; above it, one balanced cut removes most of the tail for one hop on the median, and only the slowest stage matters, so a cut that does not shorten the largest stage gains nothing. For a stage without state, such as decode, a pool of cores that each take whole packets does as well as a cut and better once the pool is large enough, with every cost charged; the order book carries state and can only be cut. A production receiver runs just under the publisher period, at about $7\mu\text{s}$, and should not be split. There, most of the tail comes from long packets, whose messages are decoded one after another while the packets behind them queue, and from service times that vary from packet to packet; the levers are the cost per message and the spread of service, not the number of threads. We give the analytic framework — a burst-limit throughput identity and an exact pathwise reduction of the deterministic-service tandem to a single bottleneck server —

*M2 Tech (16425640 Canada Inc.), Montreal. majeski@gmail.com. KASPAR-HFT is open source at <https://github.com/vincent212/kaspar-hft>.

test it on the corpus, and check the arrival statistics against a live production receiver on other instruments; they agree, and the production decode floor coincides with the service time at which the simulated tail appears. Code is in the KASPAR-HFT repository; the packet captures are not redistributable.

Contents

1	Introduction	6
1.1	The single-thread event loop against the multi-actor tandem	6
1.2	Prior recommendations for single-thread hot paths	6
1.3	Plan of the paper	8
1.4	Contribution and prior-art positioning	10
1.5	Note on the use of AI in this paper	10
1.6	Notation	11
I	Theory	13
2	Analytical framework	13
2.1	Burst-limit throughput theorem	13
2.2	Exact reduction of the tandem to one server, and the scaling bound	15
2.3	Informal statement of the two results	18
II	Empirical experiment	21
3	Experiment setup	21
3.1	Data corpus	22
3.2	Hawkes clustering of packet arrivals (empirical prerequisite)	23
3.3	Simulator	24
3.4	How the simulator models concurrent stages	24
3.5	Sweep design	25
3.6	Null streams and the ingredient each removes	26
4	Empirical results	28
4.1	Single-thread and tandem tails on the real stream and on the Poisson null	28
4.2	Gap-shuffle and 1 s-binned nulls	30
4.3	Grouping packets into matching-engine transactions	32
4.4	Tandem, sharding and dispatch on the same N cores	39
4.5	Median cost of an N -stage tandem	42
4.6	The design equation: hop cost against tail excess	43
4.7	Per-window results	46
4.8	Location of the tail onset	47
4.9	Dependence of the tail on packet rate and branching ratio	48
4.10	Sensitivity to the hop cost h	52
5	The exchange side: matching engine, publisher and bursts	53
5.1	Three timestamps	53
5.2	Transactions and events	53
5.3	Gaps between transactions on the engine and publisher clocks	54
5.4	The publisher as a queue	57
5.5	Packet size under publisher backlog	58
5.6	The receiver as the second stage of a tandem starting at the exchange	60
5.7	Bursts at the matching engine	61

5.8	Conjecture: the publisher’s capacity as an information-side speed limit	63
5.9	Summary of the findings on the exchange side	63
6	The analytical framework checked against the simulations	64
7	Cross-validation against a live production receiver	67
7.1	The recorded corpus and the live measurement compared	67
7.2	Clustering statistics in the corpus and on the live receiver	68
7.3	The production decode floor and the tail threshold	69
7.4	Statistics that differ between the corpus and the live receiver	71
7.5	Per-packet service time, message count and coalesced transactions	72
7.6	The sweep rerun with span-dependent service	75
7.7	The production handler’s tail	79
8	Summary of Part II: findings and what they mean for HFT system design	83
III	Best practices: implications for HFT systems design	85
9	The reference instance: KASPAR-HFT and its fast_send pipeline	85
10	Implementation on KASPAR-HFT: setting the send mode of each boundary	87
10.1	Receiver transparency of fast_send and send	88
10.2	End-to-end measurement of the boundary choice	88
10.3	The measure-and-flip procedure for setting each boundary	88
10.4	Thread mapping and send mode as deployment decisions	89
10.5	Frameworks in which the send mode is part of the receiver’s type	89
11	Pre-spinning receivers and pre-warming on self-exciting feeds	90
12	Recommendations for HFT system designers	91
12.1	Recommendations left unchanged	91
12.2	Recommendations the paper argues against	92
12.3	Recommended design flow	92
IV	Conclusions	94
13	Conclusion	94
13.1	Summary: what causes the receiver’s tail and how to reduce it	94
13.2	What causes the tail: what the paper establishes	95
13.3	What causes the tail: what the paper does not establish	96
13.4	How to reduce the tail	98
13.5	Revision of a prior design context	99
13.6	Caveats	99
14	Future work	100
14.1	Open questions this paper raises	100
15	Reproducibility	102

Appendices	107
A The Borel batch-size tail at fixed branching ratio	107
B Numerical check of Theorem 1	108
C Numerical check of the unequal-stage claims	109
D A synthetic feed on which the tail at $8\mu s$ depends on intensity and branching ratio	109

1 Introduction

1.1 The single-thread event loop against the multi-actor tandem

The design of the ingress-to-order pipeline in a high-frequency trading system is a choice between two archetypes: the single-threaded event loop that processes every step of every message on one core (SBE decode, book application, strategy evaluation, order construction, all under one call chain, taking a total service time we denote T), and the multi-actor pipeline — a *tandem* of N servers in series — in which those steps are split across N separate actors, each on its own thread with its own mailbox, performing its part of the work and handing the message to the next across one of $N - 1$ explicit hops whose per-hop wall-clock cost we denote h . The low-latency HFT community is predominantly in the single-thread camp, and single-threaded is generally regarded as the safe choice. The argument cites the cost of a mailbox hop (a mailbox enqueue, a cache-line transfer between the sender’s core and the receiver’s core, and the cost of scheduling or spinning the receiver) and holds that $(N - 1)h$ adds to every message’s end-to-end latency, so that each additional stage is a fixed cost that no throughput or tail benefit recovers.

That argument holds when the single-server throughput $1/T$ exceeds the peak arrival rate. On CME MDP3 the arrival process is strongly self-exciting: packets arrive in Hawkes-clustered bursts [24, 27, 28]. The exchange publishes each matching-engine event — a transaction — as, almost always, one packet, so the packet stream relays the timing of transactions without adding clustering of its own (Section 4.3). During such bursts the instantaneous arrival rate exceeds the single-thread event loop’s service rate $1/T$, and cluster-induced queueing produces a tail one to two orders of magnitude larger than h . The paper’s central empirical observation, on over a year of NQ front-month packet arrivals, is that this cluster tail dominates end-to-end latency: the 99th-percentile latency (p_{99}) of a constant-service queue driven by real packet arrivals is about twice the service time for a $16\mu\text{s}$ task and between four and five times for a $32\mu\text{s}$ task, and more in busy windows. A Poisson stream with the same number of arrivals has no tail at $16\mu\text{s}$ and a small one at $32\mu\text{s}$; over this band the tail is almost entirely a clustering effect.

The question is then whether splitting the pipeline into $N > 1$ stages, each an independent actor on its own core with per-stage service T/N , reduces the tail, and whether the additional median cost $(N - 1)h$ is small relative to the tail reduction. Neither architecture is better unconditionally. Both conditions hold above a service time that depends on the feed’s arrival statistics and fail below it; the paper measures that threshold on one feed, and its recommendation is that designers measure it on theirs.

1.2 Prior recommendations for single-thread hot paths

The single-thread rule has a lineage in the low-latency-systems community, summarised here in reverse chronological order. It has three layers: the HFT-specific codification of the last decade, the general low-latency-systems and event-driven-server culture from which HFT inherited it, and one prominent published counter-argument that the field discarded.

Modern HFT-community codification (2010–present). The founding statement of the current HFT-community formulation is Thompson’s Single Writer Principle [38] — “a single-writer design consistently outperforms a ‘properly’ lock-free or wait-free multi-writer design on most realistic workloads” — motivated by and applied in the LMAX Exchange architecture [39, 40]. Rigtorp’s widely cited low-latency tuning guide [41] formalises the operational rule (“run a single thread in `SCHED_OTHER` per core and use busy waiting/polling”), and Chronicle Software’s public engineering

materials [42] adopt the same architecture for market-data connectors (“within a single event loop ... busy-spinning and affinity locks”), pinning Chronicle Queue’s design to a single writer per queue. The Aeron messaging library [43] inherits the same lineage. The academic side is thinner, but Kwan et al. [44] codify the pattern as the standard low-latency-HFT design pattern, and recent practitioner posts [45] restate it explicitly (“all critical trading logic runs in a single-threaded event loop, eliminating context switches and lock contention entirely”). Inside HFT specifically, the same recommendation has been reinforced by two decades of private prop-trading arms-race practice among the Chicago desks (Getco, Jump, Optiver, DRW; late 2000s), reported second-hand in industry retrospectives but without an open-literature reference.

Older event-driven-server lineage (1995–2010). HFT did not invent this rule; it inherited it from the event-driven-server culture. The single-threaded event loop as an architectural pattern dates at least to Schmidt’s Reactor pattern [46] in the mid-1990s. It was sharpened by Kegel’s C10K essay [47], which named the OS thread-per-connection model as the bottleneck of scalable network servers and argued for event-driven designs. The pattern went mainstream in production with Sysoev’s nginx web server [48], whose single-threaded worker process is explicitly justified as context-switch avoidance, and later with Dahl’s Node.js runtime [49], which pushed the single-threaded event loop into ordinary application code.

Pro-pipelining minority — outside HFT. A legitimate multi-decade pro-pipelining tradition exists in the datacenter and networking-systems literature; it has been the field’s non-single-thread minority throughout the period covered by the previous two paragraphs. The earliest prominent published argument is the Staged Event-Driven Architecture (SEDA) of Welsh et al. [50], which advocated exactly the multi-stage pipeline of independently scheduled event handlers that the present paper’s tandem design revives. Welsh’s 2010 retrospective [51] is often cited as a walk-back to single-thread, but it is actually a hedged pro-pipelining recommendation: it advocates decoupling stages from thread pools, grouping multiple stages within a single thread-pool domain where latency is critical, and putting a separate thread pool and queue in front of any stage with long or nondeterministic runtime — effectively the present paper’s design rule in different vocabulary. Beyond SEDA the pro-pipelining tradition includes the Click Modular Router [52] for staged packet processing; Naiad [53] for low-latency dataflow; Snap [54] for Google’s multi-engine network stack; and the datacenter microsecond-tail-latency scheduler line — Shinjuku [55], Shenango [56], Caladan [57], Perséphone [58] — which argues for centralised multi-core dispatch specifically to control the microsecond-scale tail. Perséphone in particular is targeted at “wide service-time distributions,” the closest existing framing to our arrival-law claim; but the datacenter line frames the problem as heavy-tailed *service*, not clustered *arrivals*. DPDK explicitly supports both run-to-completion and pipeline modes and its documentation recommends pipeline mode “if some set of packets require longer to process” [59]. Inside HFT specifically the pro-pipelining position is rare in the open literature; a Chinese patent [60] describing a pipelined FAST market-data decoder is one commercial instance.

None of the pro-pipelining sources above ties the design recommendation to Hawkes / self-exciting / near-critical arrival dynamics. The closest theoretical hooks are the MMPP-input tandem work of Kim and Chydzinski [16], discussed further in Section 1.4, and the batch-arrival tandem phase-transition analysis of Vinnikov [61]. The present paper’s mechanism, that the recommendation reverses when arrivals are near-critically self-exciting, is one we have not found in prior work.

The present author’s earlier work belongs on this list with a qualification.¹ An earlier paper by

¹The tail compression reported here had been observed in production before its mechanism was understood:

the present author [36] did not argue for the single-thread principle in its own right. That paper addresses a different objection: the actor model had been considered unsuitable for HFT because of asynchronous-dispatch overhead, and **fast_send** is presented there as the mechanism that removes that overhead for co-located actors, making the actor model usable inside a servicing chain that the prevailing design placed on one thread. Its reference architecture (reader to buffer by asynchronous **send**; decode, book and signal chained by **fast_send** on one thread; signal to order manager by asynchronous **send**) takes the single-thread servicing chain as given and does not compare it against more asynchronous topologies. The present paper revises that design context, not the **fast_send** mechanism: on Hawkes-clustered feeds the servicing chain should be split into asynchronous stages once its service time exceeds the threshold established in Part II. **fast_send** remains the mechanism for chaining actors placed on one thread; what changes is where the thread boundaries inside the chain should be. The design rule is stated in Section 13.

1.3 Plan of the paper

The paper asks two questions and answers each in a named place.

1. *What produces the queueing tail behind a single-threaded receiver?* Three candidates are tested. The packet rate: no, at HFT service times (Section 4.9). The clustering of arrivals: yes (Section 4.2), and Section 4.3 identifies the clustered process as the stream of matching-engine transactions, acting through consecutive transactions sent back to back at the publisher period at short service times and through runs of transactions, the self-exciting component, at long ones. Why the matching engine processes transactions within a microsecond of each other, which is what backs up the publisher, is a question about the market, outside the scope of this paper; Section 13.3 states what can and cannot be claimed about it. The number of messages per packet: at the service time a production receiver runs at, about $7\mu\text{s}$, it carries most of the tail, through the decode of a long packet message by message and the queue that packet opens behind it; from $16\mu\text{s}$ up it adds a few percent (Sections 7.6 and 7.7). Service times that vary from packet to packet, which the simulation does not model, add a tail of their own on the live receiver.
2. *Are two queues better than one?* Yes above a service-time threshold set by the feed and no below it (Sections 4.1 and 4.6); only the slowest stage matters (Theorem 1); with equal cores and every cost charged, a dispatch pool is level with a two-stage cut and better than a cut into four or more for a stateless stage, and the stateful order-book stage cannot be dispatched (Section 4.4); at the production operating point the answer is no (Section 9).

Section 5 looks one stage upstream, at the exchange’s matching engine and market-data publisher through the timestamps the feed carries, and shows that the publisher is a queue whose output spacing, the publisher period, is the shortest gap the receiver sees.

The paper proceeds in four parts.

- **Theory.** Two closed-form results (Section 2) predict the pipeline behaviour. Proposition 1 shows that under any arrival process an N -stage tandem has an N -fold throughput speedup on isolated bursts. Theorem 1 shows that a deterministic-service tandem reduces exactly to a single server with the largest stage’s service, and that the end-to-end wait is bounded pathwise

splitting a long task into shorter asynchronously connected stages was known to reduce the tail on a per-strategy basis, and the working assumption at the time was that single-thread implementations were simply faster in the inner loop, not that the recommendation depended on the arrival law. The present paper is the explanation of that observation.

by $1/N$ of the single-stage wait plus $(N - 1)h$, at every quantile and under every arrival sequence. Corollary 2 shows that under a rate-matched Poisson null the single-stage wait at the q -quantile is already zero whenever the utilisation satisfies $\rho < 1 - q$ — which at this feed’s rate covers p_{99} for every task up to $16\mu s$ — so over that band there is nothing to compress and splitting costs exactly $(N - 1)h$. The Hawkes–Oakes cluster representation explains why the single-stage wait on a real feed is one to two orders of magnitude above h . A self-exciting process is equivalent to a Poisson stream of immigrant arrivals, each of which generates a cluster of offspring. Cluster size is governed by the branching ratio n , the mean number of direct offspring per arrival: the mean cluster holds $1/(1 - n)$ arrivals and the largest clusters reach a scale of $(1 - n)^{-2}$ arrivals (the cluster-size law is the Borel distribution; Appendix A). At the fitted $n \approx 0.8$ the mean is about five and the model-implied scale about twenty-five transactions. The last member of a run of K arrivals closer together than the service time waits behind $K - 1$ predecessors on a single server (Proposition 1), so the single-stage wait grows with run length. At $8\text{--}16\mu s$ the run is typically two consecutive packets sent back to back at the publisher period; from $64\mu s$ many members of one cluster fall within one service time. This is the wait the tandem reduces.

- **Simulation.** We build an N -stage tandem Lindley recursion on real CME MDP3 packet-arrival streams at a hop cost $h = 1.7\mu s$ (the one-way async mailbox cost derived by halving the round-trip figure in Table 4 of Maciejewski [36]) and a service time swept across $T \in \{2, 4, 8, 16, 32, 64, 128\}\mu s$, from tasks below the hop cost to tasks far above it. We sweep $N \in \{1, 2, 4, 8\}$ per scenario on each of the 3512 thirty-minute RTH windows of the corpus. Each window is also simulated under a uniform Poisson null P that redraws the same number of arrivals uniformly over the same support (Section 3.5); the null isolates the contribution of clustering, and is not a model of what any designer assumes. Section 3.6 adds seven further streams that remove the ingredients of the clustering one at a time, at the packet level and at the level of matching-engine transactions.
- **Design equation for practitioners.** From the empirical result we extract design rules for a latency-critical servicing chain (Sections 9 and 10): split a stage only when its service time is above the publisher period s_{pub} , about $7.5\mu s$ on this corpus, where a queueing tail first appears, and in practice from about $10\mu s$, where the tail excess first exceeds the hop cost on nearly every window; choose the partition with the smallest achievable largest stage s_{max} , since the tail depends only on s_{max} and a cut that does not lower it adds a hop and nothing else; place the largest stage first, at ingress; keep sub-tasks below the bottleneck together on one thread via `fast_send`, using an asynchronous `send` only at boundaries that lower s_{max} ; and settle each boundary by end-to-end measurement against a representative feed rather than by local reasoning at a single call site.
- **Live cross-validation.** The corpus statistics are compared with an independent measurement on a production receiver taken seven months after the corpus ends, on three instruments (Section 7); the clustering statistics agree, the measured decode floor coincides with the service time at which the simulated tail appears, and the sweep is rerun with the measured per-message decode cost (Section 7.6). The live ZN stream’s far tail is traced to the message count of a few large packets (Section 7.7).

1.4 Contribution and prior-art positioning

A literature search covering foundational tandem theory [6–10], tandem theory under bursty arrivals [13–17], Hawkes single-queue and adjacent work [5, 18–22], and empirical and applied HFT and RPC-pipeline work [31–35] together with the datacenter tail-latency literature found no direct prior art on any of the paper’s three claims. The nearest neighbour on the exchange side is Yoon [30], who reports that a matching engine’s tail latency under load is set by queueing at the serialised match loop during bursts; the present paper is the receiver-side counterpart, with the arrival process measured rather than assumed. The nearest on the arrival-process side is the Hawkes literature on order flow [24–28], which models order-book events for microstructure and forecasting and does not connect them to a receiver’s service queue or to the packet process that carries them. The first claim is theoretical: that the choice between a single-queue (single-thread) architecture and an N -stage tandem (multi-actor pipeline) is not fixed but *reverses with the arrival law*. At the service times an HFT hot path occupies, single-digit to low-tens of microseconds, Poisson arrivals make the single queue the better choice, because at that utilisation there is no wait to compress and each additional stage adds hop latency only; Hawkes-clustered arrivals at the same mean rate make the tandem better on the tail, because the cluster wait it removes exceeds the hop cost it adds. The reversal is between arrival laws at fixed rate and service time. A Poisson queue also develops a tail at sufficiently long service times, but later and more mildly. We have found no prior work stating that the architectural recommendation reverses between these two arrival laws. The second is empirical: a tandem-queue latency sweep over service times and stage counts, driven by CME MDP3 packet timestamps, controlled against a rate-matched Poisson null, with a hop cost calibrated from the low-load round trip measured on a production HFT actor framework. We have found no prior study that combines a real exchange-calibrated tandem sweep with both that Poisson counterfactual and a measured framework hop cost. The third is the identification of the clustered process. Grouping 3.55×10^9 messages by matching-engine transaction shows that the packet stream is the transaction stream relayed one packet per event, that the transaction stream is self-exciting with the same branching ratio as the packets, and — through seven null streams that remove one ingredient of the clustering at a time (Section 3.6) — that from $16\mu\text{s}$ up the receiver’s tail is produced by the timing of transactions and not by their size, their division into packets, the packet rate, or second-scale load, acting through transactions sent back to back at the publisher’s spacing at short service times and through runs of them at long ones; at the decode floor of a production receiver the decode of long packets message by message carries most of it. We have found no published measurement of the packet process of an exchange feed at this level, and no prior test of whether an exchange’s packetisation or the market’s self-excitation is what a receiver queues on. The full reference list appears in Section 15.

1.5 Note on the use of AI in this paper

This paper was produced with Anthropic’s Claude (via Claude Code) under the author’s direction for code generation (the analysis pipeline in the KASPAR-HFT repository), exploratory analysis, mathematical drafting, prior-art search, prose editing, and figure preparation. The analytical results of Part I, including the proof of Theorem 1, were drafted with the model and then independently checked by the author. The author independently reviewed all mathematical arguments, experimental methodology, numerical results, and citations, and takes sole responsibility for the contents of the paper.

1.6 Notation

The following symbols are used throughout.

Symbol	Meaning
T	total service time per message on a single-stage pipeline, in μs
N	number of stages in the tandem pipeline (one actor per stage)
T/N	per-stage deterministic service time in the N -stage tandem
h	per-hop wall-clock cost of the async mailbox handoff between adjacent stages
k	size of an arrival burst (number of packets in one cluster)
$\tau_k^{(N)}$	exit time of the last message of a k -burst under N stages
$s_i, s_{\max}$	per-stage services of an unequal tandem, $\sum_i s_i = T$, and their maximum
$w_j(s)$	Lindley waiting time of message j at a single deterministic server with service s
$W_j^{(N)}, W_q^{(N)}$	end-to-end wait of message j (latency minus T , hops included) under N stages, and its q -quantile
p_q	q -th percentile of the end-to-end latency distribution (p_{50} , p_{99} , etc.)
$\Delta(N)$	clustering tail excess $p_{99}(N) - p_{50}(N)$ (§4.6)
$\Delta_{\text{single}}(s)$	single-server clustering tail excess at deterministic service s ; $\Delta(N) = \Delta_{\text{single}}(T/N)$ (§4.6)
γ	fitted exponent in $\Delta(N) \approx \Delta(1)N^{-\gamma}$
κ	operator's tail-versus-median weight in the design equation
N^*	stage count minimising the design objective $(N-1)h + \kappa \Delta(N)$ (§4.6)
μ, α, β	Hawkes background rate, jump size, and decay rate (§3.2)
$n = \alpha/\beta$	Hawkes branching ratio
K	Borel-distributed offspring cluster size (§2.2)
$\bar{\lambda}_{\text{pkt}}$	mean packet-arrival intensity per 30-min window, packets per second
n_{pkt}	branching ratio of the packet-arrival Hawkes fit per window
n_{tx}	branching ratio of the transaction-start Hawkes fit per window (§4.3)
$n_{\text{pkt}}^{\#}$	per-window packet count
ρ	utilisation, $\rho = \bar{\lambda}_{\text{pkt}} \cdot T$
τ	bin width of the Fano factor, in seconds (distinct from the service time T)
$F(\tau)$	Fano factor at bin width τ : $\text{Var}[C_\tau]/\text{E}[C_\tau]$
H	Hurst exponent, from log-log slope of $F(\tau)$
H, P, G, B	as stream labels: the real stream and the three packet-level null streams (§3.6); the stream label H is distinct from the Hurst exponent
$\sigma_i, \bar{\sigma}$	span of packet i (SBE messages per datagram) and its window mean (§7.6)
S_i	span-dependent service time of packet i (§7.6)
s_{pub}	publisher period: the minimum period between consecutive packets from the exchange's market-data publisher, i.e. the publisher's service time per packet, the inverse of its maximum send rate; about $7.5 \mu\text{s}$ on NQ, measured as the 1st-percentile gap between packets, $7.43\text{--}7.53 \mu\text{s}$ (§5.4). A receiver stage slower than s_{pub} queues on a burst; a faster one does not.
no-queue latency	the latency of a message that finds every queue empty, $T + (N-1)h$
decode floor	the production decoder's time for a one-message packet with nothing queued ahead of it, $6.8\text{--}7.2 \mu\text{s}$ (§7.3)

The per-window packet count and the packet-Hawkes branching ratio would both naturally be written n_{pkt} ; the paper reserves n_{pkt} for the branching ratio and writes $n_{\text{pkt}}^{\#}$ for the count.

Part I

Theory

2 Analytical framework

Readers not interested in the formal presentation may skip to Section 2.3, which restates the results in practical terms.

Throughout, the *tandem* is the paper’s model of the multi-actor pipeline: the total per-message service time T is split into N stages placed in series, each stage an independent deterministic-service server running on its own thread that performs its share of the work and passes the message to the next stage across a mailbox hop of cost h . The single-thread event loop is the $N = 1$ case, in which one server performs all of T ; adding stages shortens each server’s service but inserts a hop between neighbours.

A proposition, a theorem and a corollary underlie the simulation. Proposition 1 is the isolated-burst identity. A message’s end-to-end *latency* is its *service time* T plus its *queueing wait*, the time it spends behind predecessors before service begins. Running the N stages concurrently raises throughput from $1/T$ to N/T , so an N -stage tandem clears a back-to-back burst N times faster: the service time is unchanged at T , and the last message’s queueing wait falls by a factor of N . This is the classical pipeline speedup, stated without proof.

For example, split a service- T server into two equal $T/2$ stages and feed it a burst of k back-to-back messages. All of the queueing now happens at the first stage: its last message waits behind $k - 1$ predecessors that each occupy it for only $T/2$, a wait of $(k - 1)T/2$ against $(k - 1)T$ on the single server. The second stage never queues: stage-one departures arrive spaced $T/2$ apart, equal to its service time, so each message passes through in $T/2 + h$. The wait is not spread across two queues; it remains in one bottleneck stage that is now twice as fast. The halving requires an equal split: cutting $T = 10$ as $7 + 3$ leaves a bottleneck of 7, so the wait falls only to $7/10$. In general the tail scales with the largest stage $s_{\max}$ rather than with the number of stages; Theorem 1 makes this exact. Theorem 1 is the general statement: the tandem’s end-to-end wait equals the wait at a single server with the largest stage’s service plus $(N - 1)h$, exactly and on every sample path, and that single-server wait is at most $1/N$ of the wait at service T . Both are elementary and independent of the arrival law; Theorem 1 rests on a reduction theorem of Friedman [11] and on the monotonicity of the Lindley recursion. Corollary 2 treats the Poisson null: the Poisson wait at quantile q is zero until $\rho = 1 - q$, which at the utilisation of a CME packet feed covers p_{99} for every task up to $16\mu\text{s}$; over that band splitting costs $(N - 1)h$ and removes nothing. The arrival law enters only through the size of the single-stage wait to which the factor $1/N$ is applied, and the Hawkes–Oakes cluster representation explains why that wait is large on a real feed. Part II measures the slack in the bound on real arrivals.

2.1 Burst-limit throughput theorem

Proposition 1 (Burst-limit N -factor speedup). *Consider an isolated cluster of k arrivals whose j -th member arrives no later than $(j - 1)T/N$ after the first, so that no server idles during the burst (an instantaneous batch is the canonical case). Let $\tau_k^{(1)}$ denote the exit time of the last arrival under a single-stage deterministic-service queue with service T , and let $\tau_k^{(N)}$ denote the exit time under an N -stage tandem with per-stage deterministic service T/N and per-stage hop delay h . Then*

$$\tau_k^{(1)} = kT, \quad \tau_k^{(N)} = T + (k - 1)\frac{T}{N} + (N - 1)h, \quad (1)$$

and in particular

$$\lim_{k \rightarrow \infty} \frac{\tau_k^{(1)}}{\tau_k^{(N)}} = N. \quad (2)$$

Remark 2.1. *Proposition 1 is the classical pipelined-tandem throughput result [8, 9]; we state it without proof and use it only to fix the baseline against which the arrival-dependent tail reduction of Theorem 1 is measured. What is paper-specific is the numerical calibration of T and h on a production HFT actor framework and the observation that under real CME MDP3 arrivals the ratio $\tau_k^{(1)}/\tau_k^{(N)}$ is what governs the observed tail-latency compression.*

Reading Equation (1). In the paper’s HFT setting we take $T = 10\mu\text{s}$ as a typical HFT task on a single-thread event loop, and k is the size of a Hawkes-cluster burst that hits the pipeline back-to-back. The single-stage formula $\tau_k^{(1)} = kT$ says that in an event-loop design the last message of a k -burst waits behind $k - 1$ predecessors, each of which spends the full T on the one server; a 10-packet cluster on a $10\mu\text{s}$ server therefore has a last-message latency of $100\mu\text{s}$. The tandem formula $\tau_k^{(N)} = T + (k - 1)(T/N) + (N - 1)h$ decomposes the last-message latency into three physical pieces: (i) T , the total service work every message must traverse regardless of N ; (ii) $(k - 1)(T/N)$, the pipelined wait behind $k - 1$ predecessors at throughput N/T rather than $1/T$; and (iii) $(N - 1)h$, the one-time hop-cost tax paid to traverse $N - 1$ mailbox boundaries on the way through. The single message ($k = 1$) pays only (i) and (iii). Under Hawkes arrivals term (ii) is large on a single stage and falls by a factor of N under a tandem, while term (iii) is an additive cost of $(N - 1)h$.

Reading Equation (2). The ratio $\tau_k^{(1)}/\tau_k^{(N)}$ tends to N as the cluster size k grows, meaning that in the burst-saturation limit a tandem of N actors clears the burst N times faster than an event loop. This is the classical pipeline throughput speedup restated for HFT: a 4-actor pipeline drains a large burst in one-quarter of the wall-clock time that a single-thread event loop would take, and a 12-actor pipeline in one-twelfth. The speedup is obtained with the same total service work per message: the pipeline does not reduce the work per event, it overlaps the work across cores so that the queue behind a cluster clears in parallel. The arrival law does not enter Equation (2); what the arrival law controls is *how often* the burst regime is entered, and Theorem 1 shows the same factor governs every message on every sample path.

On what T is, and on the natural stage boundaries. The paper adopts $T = 10\mu\text{s}$ as a ballpark typical HFT single-thread event loop. Any real HFT pipeline of that duration is already a sequential chain of internally atomic sub-tasks (SBE decode, order-book application, strategy touch, order construction on CME MDP3, per Section 10), so the N stages of the tandem are placed at the existing boundaries between those sub-tasks; the paper is not subdividing any single sub-task, and in particular is not subdividing the SBE decode, which is a linear pass over the wire bytes of a single message and cannot be split. Ten microseconds is chosen for exposition; none of the paper’s qualitative results depend on it. The relevant knob for the design equation is the ratio T/h : doubling T at fixed h doubles the cluster-wait terms of Equation (1) and leaves the hop tax unchanged. The service time below which splitting cannot pay (Section 4.6) turns out to be set by the feed rather than by h : it is the publisher period, and on this corpus it binds well above $2h$. The paper writes “per-stage service T/N ” as a simplifying modelling convention that assumes equal-sized stages; in reality the per-stage services s_i sum to T but are unequal, and the achievable pipeline throughput is $1/\max_i s_i$ rather than N/T . Theorem 1 handles the unequal case exactly: the factor becomes

$s_{\max}/T$ in place of $1/N$, the tail still contracts under Hawkes, and the median still grows linearly at slope h per hop.

The mechanism applies to any deterministic-service tandem regardless of the arrival law; the arrival law determines how often large- k clusters occur. The next subsection makes the per-burst factor a pathwise bound.

2.2 Exact reduction of the tandem to one server, and the scaling bound

Proposition 1 treats one isolated burst. The result that carries the paper's design claim is that the same N -factor holds pathwise for every message under every arrival sequence, at the level of the end-to-end wait. Fix an arrival sequence $a_1 \leq a_2 \leq \dots$ (any point process, clustered or otherwise, with batches allowed as coincident times) and write $\delta_j = a_{j+1} - a_j \geq 0$. For a single FIFO server with deterministic service s fed by this sequence, the waiting time of message j obeys the Lindley recursion [12]

$$w_1(s) = 0, \quad w_{j+1}(s) = (w_j(s) + s - \delta_j)^+. \quad (3)$$

Define the end-to-end wait of message j in the N -stage tandem as $W_j^{(N)} = (\text{exit time}) - a_j - T$, i.e. latency in excess of the total service work; under $N = 1$ this is $W_j^{(1)} = w_j(T)$.

Theorem 1 (Reduction and scaling bound). *Consider an N -stage tandem of FIFO servers with deterministic per-stage services $s_1, \dots, s_N$ summing to T , deterministic hop delay h between consecutive stages, unbounded buffers, and an arbitrary arrival sequence. Let $s_{\max} = \max_i s_i$. Then for every message j and every sample path,*

$$W_j^{(N)} = w_j(s_{\max}) + (N - 1)h, \quad (4)$$

$$w_j(s') \leq \frac{s'}{s} w_j(s) \quad \text{for all } 0 < s' \leq s, \quad (5)$$

and consequently

$$W_j^{(N)} \leq \frac{s_{\max}}{T} W_j^{(1)} + (N - 1)h = \frac{W_j^{(1)}}{N} + (N - 1)h \quad \text{under equal stages } s_i = T/N. \quad (6)$$

Since Equation (6) holds pathwise, it holds for every quantile of the stationary distribution whenever one exists: $W_q^{(N)} \leq W_q^{(1)}/N + (N - 1)h$ for all q . The same inequality holds for the empirical (sample) quantiles computed over any finite trace, directly from the pathwise bound and without invoking stationarity; the sample-quantile form is what Part II measures, and it should be read as a statement about the observed windows rather than about a stationary-distribution quantile or a finite-sample maximum.

Proof. Equation (4), equal stages. Departures from a FIFO server with deterministic service s are spaced at least s apart, because each departure requires s of uninterrupted service after the previous one. A FIFO server with deterministic service s whose arrivals are spaced at least s apart never queues: by induction, message j departs at its arrival time plus s , which is at or before the arrival of message $j + 1$. Stage 1 is a single server with service T/N fed by the original sequence, so message j leaves it at $a_j + w_j(T/N) + T/N$. The arrival sequence at stage 2 is the stage-1 departure sequence shifted by h , spaced at least T/N apart, so stage 2 adds exactly $h + T/N$ and its departures are again spaced at least T/N apart; the same holds at each subsequent stage. Summing, the exit time is $a_j + w_j(T/N) + N \cdot T/N + (N - 1)h$, and subtracting $a_j + T$ gives Equation (4) with $s_{\max} = T/N$.

Equation (4), unequal stages. By the order-invariance theorem of Friedman [11], for a tandem of FIFO stages with constant service times, unlimited queues, and any arrival sequence, each customer's time in system is independent of the order of the stages. Deterministic hop delays preserve this: a hop is a constant shift of one stage's departure sequence into the next stage's arrival sequence, and constant shifts commute with the Lindley recursion, so reordering stages together with their hops leaves every exit time unchanged. Reorder so that the stage with service $s_{\max}$ comes first. Its departures are spaced at least $s_{\max}$ apart, and every subsequent stage has service $s_i \leq s_{\max}$, so by the no-queueing argument above each of them adds exactly $s_i + h$ to every message. The exit time is $a_j + w_j(s_{\max}) + \sum_i s_i + (N - 1)h$, and subtracting $a_j + T$ gives Equation (4).

Equation (5). By induction on j with $w_1(s') = 0 = (s'/s)w_1(s)$. The map $x \mapsto (x + s' - \delta_j)^+$ is nondecreasing, so

$$w_{j+1}(s') = (w_j(s') + s' - \delta_j)^+ \leq \left(\frac{s'}{s}w_j(s) + s' - \delta_j\right)^+ \leq \left(\frac{s'}{s}w_j(s) + s' - \frac{s'}{s}\delta_j\right)^+ = \frac{s'}{s}w_{j+1}(s),$$

where the second inequality uses $\delta_j \geq (s'/s)\delta_j$, which holds because $\delta_j \geq 0$ and $s' \leq s$. Combining Equations (4) and (5) with $s' = s_{\max}$, $s = T$ gives Equation (6). $\square$

Remark 2.2 (What the theorem says and what it does not). *Equation (4) says all queueing in an equal-stage deterministic tandem happens at stage 1; stages $2, \dots, N$ contribute exactly $T/N + h$ each to every message. Proposition 1 is the isolated-burst special case, where Equation (6) holds with equality. Equation (6) is a bound rather than an identity because the tandem runs its bottleneck stage at utilisation ρ/N while the single stage runs at ρ : the induction step in the proof of Equation (5) gives away $(1 - 1/N)\delta_j$ at every message j that the single server is still serving when message $j + 1$ arrives, so the tandem clears each inter-arrival gap N times more effectively than the scaled bound credits it for. The paper's empirical γ (Section 4.6), defined by $W_q^{(N)} - (N - 1)h \approx N^{-\gamma} W_q^{(1)}$, therefore satisfies $\gamma \geq 1$ in the model, and the excess $\gamma - 1$ measures that slack. Nothing in the theorem depends on the arrival law; what the arrival law determines is the size of $W_q^{(1)}$ that the factor $1/N$ is applied to, which is the subject of the next remark. The hop cost h enters every quantile as exactly $(N - 1)h$, in the median and in the tail alike.*

Remark 2.3 (Unequal stages: only the largest stage matters). *Real cut points do not give equal stages. Equation (4) states the consequence: the end-to-end wait of any deterministic tandem equals the wait at a single server with service $s_{\max}$, plus h per hop. Three consequences follow.*

1. *The tail factor is $s_{\max}/T$, and the stage count enters only through the hop tax. Splitting a $10 \mu\text{s}$ task as $7 + 3$ compresses the tail excess to at most 70% of single-stage, against 50% for $5 + 5$, and both pay one h .*
2. *The order of the stages is irrelevant to end-to-end latency: $7 + 3$ and $3 + 7$ give identical latencies on every sample path. Order determines only where the queue physically sits (below).*
3. *A cut that does not lower $s_{\max}$ adds a hop and nothing else. Refining $7 + 3$ into $7 + 2 + 1$ leaves every quantile of the wait unchanged and raises end-to-end latency by h . Conversely, sub-tasks can be merged onto one thread (via **fast_send**) without any tail cost as long as the merged service does not exceed $s_{\max}$. The design problem is therefore to choose the partition with the smallest $s_{\max}$ reachable at the natural cut points, and then the fewest hops that realise it.*

Appendix C checks all three numerically.

Remark 2.4 (Where the burst goes: the bottleneck absorbs the clustering). *The proof of Equation (4) rests on one fact: departures from a deterministic server with service s are spaced at least s apart. Applied to a Hawkes input, that fact says what the bottleneck stage does to the clustering. Upstream of the bottleneck the stream is the raw feed, with intra-cluster inter-arrival times far below $s_{\max}$ and instantaneous intensity far above $1/s_{\max}$. Downstream of the bottleneck, every cluster still leaves as the same group of K messages, so the count-level branching structure is unchanged (the Borel cluster sizes, and the Fano factor at time scales beyond $Ks_{\max}$, are exactly those of the input), but the intra-cluster spacing has been regularised to $s_{\max}$ and the instantaneous intensity is capped at $1/s_{\max}$. The clustering is not removed; it is regularised to a spacing that any consumer at least as fast as the bottleneck can absorb, which is why stages with service $s_i \leq s_{\max}$ never queue. If the bottleneck is stage 1, all of the waiting in the pipeline sits in its mailbox and every downstream mailbox holds at most one message; if the bottleneck is stage $b > 1$, the stages before it queue partially (the raw burst is still bursty relative to their smaller services) and stage b absorbs the remainder, with the same total by Equation (4).*

Four practical consequences follow. First, place the largest stage first, at ingress: end-to-end latency is unchanged by the order, but this confines all queueing to one mailbox, keeps the downstream mailboxes at depth one, and places the burst-absorbing queue where its receiver is continuously busy during a cluster (the regime in which the load-dependent hop cost of Section 4.10 is smallest). Second, any burst-aware logic, such as load shedding, coalescing of superseded book updates, or a cancel-on-burst risk rule, can only observe the raw burst upstream of the bottleneck; downstream it sees a rate- $1/s_{\max}$ stream and cannot tell a cluster from steady flow except by counting. Third, a packet-Hawkes fit run on the bottleneck's departure stream would report a lower intensity peak and a longer effective kernel while recovering the same branching ratio in counts; operators instrumenting a pipeline should fit on the ingress stream. Fourth, deterministic hop delay is what keeps downstream stages queue-free; hop jitter of amplitude ϵ lets a downstream stage queue by at most ϵ per message, so the reduction holds up to that jitter in practice.

Fact (Hawkes–Oakes 1974 cluster representation [2]). *A stationary Hawkes process with background rate $\mu > 0$ and branching ratio $n \in [0, 1)$ is stochastically equivalent to a Poisson-cluster process: a rate- μ Poisson process of immigrant arrivals, each of which independently spawns an offspring cluster whose total size K (including the immigrant) follows the Borel distribution*

$$\Pr(K = k) = \frac{(nk)^{k-1} e^{-nk}}{k!}, \quad k = 1, 2, 3, \dots \quad (7)$$

with mean $E[K] = 1/(1 - n)$ and variance $\text{Var}[K] = n/(1 - n)^3$ [3].

Remark 2.5 (Where Hawkes enters: the size of the single-stage wait). *Under Poisson arrivals at the utilisation of a CME packet feed ($\rho = \bar{\lambda}_{\text{pkt}}T \ll 1$), the single-stage wait is zero for almost every message, so $W_q^{(1)} = 0$ for every quantile q with $\rho < 1 - q$ and Equation (6) has nothing to compress; that is Corollary 2, and at this corpus's rate it holds at p_{99} for all $T \leq 16 \mu\text{s}$. Under Hawkes arrivals the cluster representation puts K near-coincident messages behind one immigrant, and in the instantaneous-batch picture the last of them waits $(K - 1)T$ on a single stage. Stirling's formula applied to Equation (7) gives, for large k ,*

$$\Pr(K = k) \approx \frac{1}{n\sqrt{2\pi}} k^{-3/2} (ne^{1-n})^k, \quad (8)$$

and $ne^{1-n} < 1$ for every $n < 1$, with $-\log(ne^{1-n}) \approx (1 - n)^2/2$ near criticality. The Borel batch size is therefore light-tailed at every fixed $n < 1$: its survival function follows the critical $k^{-1/2}$ power

law only in the window $k \ll k^* = (1 - n)^{-2}$ and decays geometrically beyond it (Appendix A). The consequences for the paper are, first, that the largest clusters, and hence the single-stage cluster wait they produce, have a characteristic scale of order $Tk^* = T(1 - n)^{-2}$, which at the corpus-median $n \approx 0.8$ (Section 3.2) is $\approx 25T$ and at $n = 0.9$ would be $100T$, one to two orders of magnitude above the service time and the hop cost; second, that heavy-tail results built on subexponential batch sizes, such as the single-big-batch principle for $M^{[K]}/G/1$ [4] and the monotone-separable tail asymptotics of Baccelli et al. [14], do not apply to this batch distribution at any fixed $n < 1$, so no closed-form power-law exponent for the tail probability should be expected; and third, that the design claim does not depend on any of this, because Theorem 1 applies the factor $1/N$ to whatever $W_q^{(1)}$ the feed produces. Part II measures $W_q^{(1)}$ on real arrivals.

Corollary 2 (Poisson null). *Under homogeneous Poisson arrivals at rate $\bar{\lambda}$ with $\rho = \bar{\lambda}T < 1 - q$, the q -quantile of the single-stage wait is $W_q^{(1)} = 0$, because $\Pr(W^{(1)} > 0) = \rho$ for $M/D/1$ by the PASTA property (Poisson arrivals see time averages). By Theorem 1, $W_q^{(N)} \leq (N - 1)h$, and by Equation (4) $W_q^{(N)} \geq (N - 1)h$, so*

$$W_q^{(N)} = (N - 1)h \quad \text{for all } N \text{ and all } q \leq 1 - \rho. \quad (9)$$

At the utilisation of a CME packet feed, well under 1% at $T = 8 \mu\text{s}$, this covers p_{99} at every $T \leq 16 \mu\text{s}$ in the paper's sweep: end-to-end latency is $T + (N - 1)h$ at p_{50} and at p_{99} alike, and splitting costs $(N - 1)h$ with nothing to compress. Where ρ/N exceeds $1 - q$, Equation (6) still holds and the tandem reduces the Poisson wait as well; splitting then helps under Poisson for the ordinary utilisation reason rather than for clustering. The sweep contains six such p_{99} cells, all at the long-service end, and Section 4.1 finds the wait in those cells and in no others.

2.3 Informal statement of the two results

This subsection restates Proposition 1, Theorem 1 and Corollary 2 informally. The formal statements above are the reference.

Components of latency. A message's end-to-end latency is the sum of three terms: the work done on it (T in total, however many stages that work is spread over), the hop tax (h for every mailbox boundary it crosses, so $(N - 1)h$ in an N -stage pipeline), and the time it spends sitting in queues waiting for a busy actor. The third term is the wait, W . The work is fixed by the code and the hop cost by the framework; the wait is the only term the architecture changes, and on a bursty feed it dominates the tail.

Proposition 1: one burst. Suppose k messages land at once. On a single thread the last of them waits for the $k - 1$ ahead of it, each costing T : it waits $(k - 1)T$. On an N -stage pipeline the first stage takes only T/N per message, so the last message waits $(k - 1)T/N$ at stage 1 and then flows through the remaining stages without waiting again, because each of them is fed at exactly the rate it can serve. The pipeline clears a burst N times faster for the same total work, because N cores process N different messages at the same instant.

Theorem 1: every message, any arrival sequence. Two facts.

1. In a pipeline where each stage does a fixed amount of work per message, only the slowest stage ever builds a queue. Every other stage receives messages no faster than it can process them,

because the slowest stage upstream (or, if the slowest stage is downstream, the stages before it) meters them out. The pipeline therefore behaves as one thread whose per-message work is $s_{\max}$, plus the hop cost. With equal stages, $s_{\max} = T/N$.

2. Take any recording of arrival times, real or synthetic, and replay it through two single-threaded servers: one doing T of work per message and one doing T/N . The second server's queue wait for message number j is never more than $1/N$ of the first server's wait for the same message. The reason is that in the faster server every backlog is N times smaller and every idle gap between arrivals clears at least as much of it.

Combined: for every message in every replay, the pipeline's wait is at most the single-thread wait divided by N , plus $(N - 1)h$. Because this holds message by message, it holds for any summary statistic: median, p_{99} , $p_{99.9}$, or maximum. The bound is met with equality on an isolated instantaneous burst and is strict whenever arrivals are spread out, because the pipeline's first stage is idle more often than the single thread and drains more of its backlog in each idle gap. The measured compression is written $N^{-\gamma}$; the theorem guarantees $\gamma \geq 1$, and Section 4 measures $\gamma = 3.3$ and 2.7 at $T = 64$ and $128 \mu\text{s}$. At shorter service times the tandem reduces the excess to zero and no exponent is defined.

Role of the arrival law. The arrival law does not enter either theorem. It enters only through the size of the single-thread wait to which the factor $1/N$ is applied. Under Poisson traffic at the load of a CME packet feed, the single thread is idle when almost every message arrives, so for tasks up to $16 \mu\text{s}$ its p_{99} wait is zero; there is nothing to divide by N , and the pipeline is simply $(N - 1)h$ slower (Corollary 2). Under the real feed, transactions arrive in self-exciting clusters and the packets relay them, the single thread's p_{99} is two to about forty times T for tasks of 16 to $128 \mu\text{s}$, and the pipeline divides the excess by N or better. The Hawkes cluster representation gives a model-implied cluster size: Borel-distributed, with large clusters of order $(1 - n)^{-2}$ transactions, about 25 at the corpus's $n \approx 0.8$. What the receiver waits behind depends on the service time: at $8\text{--}16 \mu\text{s}$ the wait is usually behind one predecessor sent $7.5\text{--}16 \mu\text{s}$ earlier, and only from about $64 \mu\text{s}$ does a run of cluster members fall within one service time (Sections 4.2 and 6).

Design rules.

- The tail improvement is set by the slowest stage, $s_{\max}/T$, and by nothing else about the decomposition. Balanced stages are the best case for a given number of hops.
- A cut that does not reduce the slowest stage costs h at every quantile and improves nothing. Sub-tasks can be merged onto one thread until their combined work reaches $s_{\max}$ at no cost to the tail.
- The order of the stages does not change any message's latency. It changes only where the queue physically sits; putting the slowest stage first keeps all queueing in one mailbox.
- Every hop costs h on every message, at the median and in the tail.
- For a stream that must be processed in order, whether to split reduces to one comparison: the single-thread wait at the quantile of interest, times $(1 - s_{\max}/T)$, against h per hop added. On a Poisson feed at HFT service times the left side is zero; on a clustered feed at the same rate it is tens to thousands of microseconds. With the same cores, a stateless stage on a stream that tolerates reordering can instead dispatch whole packets to N servers; Section 4.4 compares the two without charging dispatch its ingress, hop or resequencing cost.

The guarantees above are exact within a specific model. Its assumptions follow.

Fixed work per message. Each stage is assumed to spend the same amount of time on every message. That is what makes only the slowest stage queue: a stage that sometimes takes longer than its upstream neighbour would occasionally fall behind and build its own queue, and then the pipeline’s wait would exceed the single-server wait at $s_{\max}$. The simulator of Part II uses constant per-packet service, so the theorems describe it exactly; Section 7.6 relaxes this for the message count per packet. Real actors also have work that varies with book state, and for them the theorems are a model to be checked by measurement (Section 10).

No drops, no bounds, in-order. Mailboxes are unbounded, messages are never discarded or coalesced, and each stage processes in arrival order. A framework that drops or merges messages under load changes the arithmetic (in the pipeline’s favour, at the cost of lost messages); a priority queue changes which message pays the wait but not the total.

Fixed hop delay. The hop is modelled as a constant h . A jittery hop lets a downstream stage queue by at most the jitter amplitude per message, so the reduction holds up to that amplitude. The paper’s calibration uses the low-load, wake-from-sleep value of h , which Section 4.10 argues is an upper bound inside a burst.

Equal stages, no cross-core cost beyond h . The N stages are taken to split the work evenly and to exchange nothing but the message: moving book state or cache lines between the cores that run adjacent stages is charged nothing beyond h . Unequal stages are handled by Remark 2.3; a cross-core cost would add to h and is a matter for measurement (Section 4.10).

Proposition 1 needs a dense burst. The one-burst identity assumes the k messages arrive faster than the first stage can drain them, i.e. faster than one per T/N . For a burst that trickles in, the ratio in Equation (2) is $kN/(k + N - 1 + N(N - 1)h/T)$, which climbs toward N as the burst grows. Theorem 1 does not need this assumption; it covers every arrival pattern.

Limits of the theory. It does not say how large the single-thread wait is on a given feed. That number depends on the feed’s clustering and utilisation and, as the remark after the Borel fact explains, has no clean closed form at a fixed branching ratio: the Borel cluster size is light-tailed for every $n < 1$, so the heavy-tail formulas that would give a power-law exponent do not apply. The paper measures the single-thread wait on real CME arrivals instead (Part II). Nor does the theory predict how far below the $1/N$ bound the pipeline lands; that slack is also measured. What the theory guarantees is that the pipeline’s wait is at most $1/N$ of the single-thread wait plus the hop cost, on any arrival sequence.

Part II

Empirical experiment

3 Experiment setup

This section describes the empirical experiment that tests the analytical predictions of Section 2: the data corpus, the confirmation that packet arrivals are Hawkes-clustered (the reason the single-stage wait that Theorem 1 divides by N is large at all), the tandem-Lindley simulator, and the sweep design that produces the numbers reported in Section 4. Figure 1 shows the end-to-end data flow of a single 30-minute window; Algorithm 1 states the full per-window procedure in pseudocode.

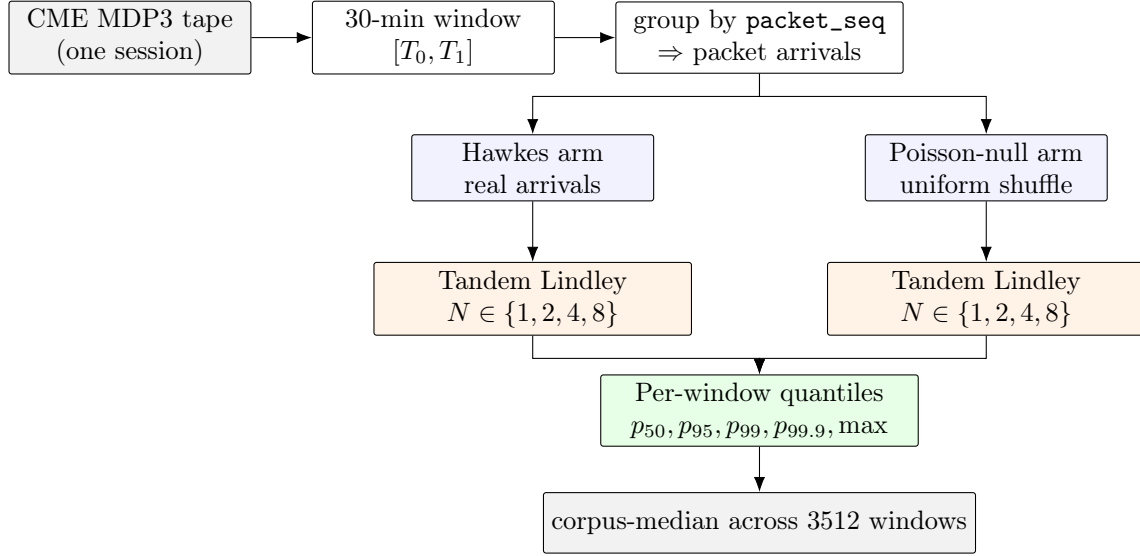


Figure 1. Per-window data flow of the empirical experiment. A single session tape is chopped into 30-minute RTH windows. Each window’s messages are grouped by `packet_seq` into a real packet-arrival stream (Hawkes arm). A rate-matched Poisson-null arm is generated by uniformly shuffling the same arrival count over the same window. Both arms are run through the N -stage tandem Lindley simulator at $N \in \{1, 2, 4, 8\}$. Per-window quantiles are recorded and aggregated by corpus median across all 3512 windows in the 276-session corpus.

Algorithm 1 Per-window procedure of the empirical experiment.

```
1: input: session tape  $S$ , service  $T$ , hop  $h$ , stage counts  $\mathcal{N}$ 
2: for each 30-min RTH window  $W$  in  $S$  with  $[T_0, T_1]$  do
3:    $\{m_i\} \leftarrow$  messages in  $W$ 
4:    $\{(\text{seq}, t)\} \leftarrow \{(\text{packet\_seq}(m_i), \text{sendingTime}(m_i))\}$ 
5:    $\text{arr}_H \leftarrow \text{sort}(\{\min_i t_i : \text{seq}_i = s\}_s)$   $\triangleright$  Hawkes arm (real arrivals)
6:    $n_{\text{pkt}}^\# \leftarrow |\text{arr}_H|$ 
7:    $\text{arr}_P \leftarrow \text{sort}(\text{Uniform}(\min \text{arr}_H, \max \text{arr}_H)^{n_{\text{pkt}}^\#})$   $\triangleright$  Poisson-null arm (seeded per window)
8:   for regime  $\in \{H, P\}$ ,  $N \in \mathcal{N}$  do
9:      $\text{lat} \leftarrow \text{TANDEMLINDLEY}(\text{arr}_{\text{regime}}, N, T, h)$ 
10:    record  $(S, W, \text{regime}, N, \{p_{50}, p_{95}, p_{99}, p_{99.9}, \max\}(\text{lat}))$ 
11:   end for
12: end for
13: return corpus-median of each recorded quantile across all  $(S, W)$  pairs
14: function TANDEMLINDLEY( $\text{arr}, N, T, h$ )
15:    $\text{dep} \leftarrow \text{arr}$ 
16:   for stage  $k = 1, \dots, N$  do
17:     if  $k > 1$  then  $\text{dep} \leftarrow \text{dep} + h$ 
18:   end if
19:    $\text{dep} \leftarrow \text{dep} + \text{LINDLEY}(\text{dep}, T/N)$ 
20: end for
21:   return  $\text{dep} - \text{arr}$ 
22: end function
```

3.1 Data corpus

The corpus is 281 sessions of CME MDP3 channel 318 (NQ) front-month pcap-derived tapes covering 2025-01 through 2026-02, RTH only, holidays and half-days excluded. Each MBO record carries a `packet_seq` (the UDP packet sequence number) and two nanosecond timestamps: `transactTime`, the CME matching-engine event time carried in the MDP3 message body, and `sendingTime`, the publisher’s per-packet header timestamp. Messages are grouped into packets by `packet_seq`, and a packet’s arrival time is its `sendingTime`.

`transactTime` is not a suitable arrival clock. It is the matching engine’s event time, not the time at which anything reaches the receiver: the 1.7% of transactions that the publisher splits across two or more packets (Section 4.3) carry one `transactTime` on packets that leave about $15\mu\text{s}$ apart, so keying arrivals on it collapses distinct packets onto one instant. `sendingTime` is emitted once per packet at nanosecond resolution and is the receiving pipeline’s input clock; ordered by `packet_seq` it produces no ties and no sequence inversions on the corpus. All results below are keyed on `sendingTime`. Section 4.3 shows that merging each transaction into one packet leaves the tail within $\pm 15\%$, so the choice of clock does not affect the results.

Each session tape is chopped into non-overlapping 30-minute RTH windows; for each window we compute the packet-arrival stream and the per-window quantities listed in Section 1.6. The corpus yields 3512 per-window records from the 276 sessions that contain at least one complete RTH window, roughly 13 windows per session; the remaining five of the 281 tapes are holiday or half-day sessions with none.

3.2 Hawkes clustering of packet arrivals (empirical prerequisite)

The paper’s simulator input is the packet-arrival stream, so the first empirical question is whether that stream is clustered or Poisson. Three model-free diagnostics and one model-based fit, on every window of the corpus, reject Poisson; Table 1 gives the values. Nearly three quarters of the gaps between packets are shorter than a tenth of the mean gap, where a Poisson stream of any rate has under a tenth, and no window comes close to the Poisson value. The Fano factor of packet counts, $F(\tau) = \text{Var}[C_\tau]/\text{E}[C_\tau]$ in bins of width τ , is 1 at every bin width for Poisson; here it grows from about 50 at a tenth of a second to about a thousand at a minute, and the Hurst exponent from its log-log slope is about 0.75 against 0.5. These are within-window values; pooling a whole session instead raises the Fano level without changing the shuffle ratio, for reasons taken up in Section 7.4. The same estimators applied to the paper’s own Poisson null return the Poisson values at every bin width, so the figures are not an artefact of binning or of window length.

statistic	real stream	Poisson	real stream, gaps shuffled
gaps shorter than a tenth of the mean gap	73%	9.5%	73%
Fano factor, 0.1 s bins	52	1	16–18
Fano factor, 1 s bins	108	1	17.5
Fano factor, 5 s bins	260	1	17.6
Fano factor, 60 s bins	1027	1	16–18
Hurst exponent	0.74	0.50	0.50
Hawkes branching ratio n	0.798	0	—
Hawkes kernel decay time $1/\beta$	167 μs	—	—

Table 1. Clustering diagnostics of the packet stream, corpus median over windows, against the Poisson values and against the same stream with its gaps shuffled within each window. The kernel decay time is from 254 windows in 20 sessions. What the table shows: every diagnostic is far from Poisson. Shuffling the gaps keeps the share of short gaps, by construction, and removes all the growth of the Fano factor with bin width, leaving a flat value of about 17. What follows: the long-range dependence lives in the order of the gaps, which is what distinguishes a self-exciting process from a renewal one, while the flat residual comes from the gap lengths themselves, which are heavy-tailed.

Fitting the standard exponential-kernel Hawkes process [1] $\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}$ to each window by maximum likelihood [23] (Ogata log-likelihood with the standard recursive intensity, closed-form compensator, Nelder–Mead over (μ, α, β)) gives a branching ratio $n = \alpha/\beta$ of about 0.8, varying little between windows, with every fit converged; a reparameterised L-BFGS-B fit finds the same optimum on spot-checked windows. The kernel decays over about 160 μs , so the fitted process triggers at lags of order a hundred microseconds and places only a few percent of its triggering within the publisher period. This is the near-critical regime that Filimonov and Sornette [24] and Hardiman et al. [27] document on ES, at a somewhat lower level than their $n \approx 0.9$; they fit trade arrivals rather than packet arrivals, and on this corpus the fit on matching-engine transactions gives the same value as on packets (Section 4.3). Filimonov and Sornette [25] show that an exponential-kernel fit with a constant background over a fixed window overstates n when the background intensity varies within the window, and that ES estimates fall well below criticality once that variation is modelled; the n reported here is descriptive and subject to the same caveat, and none of the paper’s conclusions rests on its value. The model-free diagnostics hold whatever the fitted n , and the nonparametric estimates of Achab et al. [26] on EUREX order books give the same near-critical picture without a kernel assumption.

A shuffle of the gaps within each window keeps every gap and destroys only their order. It removes most but not all of the excess over Poisson: all of the growth of the Fano factor with bin

width goes, leaving a flat over-dispersion of about 17 and a Hurst exponent of 0.5 (Table 1). All of the long-range dependence therefore lives in the ordering, which is what distinguishes a self-exciting process from a renewal process, and the flat residual is contributed by the gap lengths, which are heavy-tailed. The corpus is neither a renewal process nor a pure Hawkes process with light-tailed gaps, but a self-exciting process with heavy-tailed gaps. This bounds what the Hawkes fit describes: the fitted n characterises the ordering component, and the simulator is driven by the recorded arrivals, not by the fitted process. The simulator input is therefore an arrival process far from Poisson, and the Poisson null at the same rate is a strict counterfactual. The tandem reduction of Section 2 does not depend on the Hawkes assumption, since any arrival process with the same clustering drives the same single-server queue. The model-free diagnostics, not the fit, establish the empirical premise.

Findings.

- *Packet arrivals are far from Poisson.* Nearly three quarters of gaps are shorter than a tenth of the mean gap, against under a tenth for Poisson.
- *Clustering grows with time scale:* the Fano factor rises from about 50 at a tenth of a second to about a thousand at a minute (Poisson: 1).
- *A Hawkes fit describes it as near-critical,* with branching ratio about 0.8 and a kernel acting over about $160\ \mu\text{s}$, mostly beyond the publisher period.
- *Two ingredients:* shuffling the gaps removes the growth with time scale (the ordering) and leaves a flat over-dispersion from heavy-tailed gaps.

3.3 Simulator

Packet arrivals $t_1 < t_2 < \dots$ enter stage 1. Each stage k is a single-server FIFO queue with deterministic service T/N (own actor, own thread), and departures from stage k are delayed by h before arriving at stage $k + 1$. Standard Lindley recursion per stage:

$$W_{i+1}^{(k)} = \max\left(0, W_i^{(k)} + \frac{T}{N} - (a_{i+1}^{(k)} - a_i^{(k)})\right), \quad (10)$$

with $a_i^{(k+1)} = a_i^{(k)} + W_i^{(k)} + T/N + h$.

Service is the same for every packet. Section 7.6 makes it grow with the number of messages in the packet; nothing in the paper makes it random. A real receiver’s service time varies from packet to packet for reasons other than message count (Section 7.7), and for a single server, more variable service at the same mean lengthens the mean wait (the Pollaczek–Khinchine formula for Poisson arrivals). The simulated tails are therefore what the arrival timing and the message count produce on their own, not everything a real receiver pays.

3.4 How the simulator models concurrent stages

Each stage’s actor runs on its own thread, so stage 2 can be processing message $k - 1$ while stage 1 is processing message k . The simulator captures this because Lindley is applied separately per stage, each with its own busy-idle timeline. What the split buys is not the overlap as such but the shorter queue in front of each stage: a burst that one server of service T holds in a queue of length k is held by the first stage at T/N and drained N times faster.

Worked example. Three packets arriving at $t = 0, 1, 2 \mu\text{s}$; single-stage total service $T = 8 \mu\text{s}$ (a service time in the sweep of Table 3); two-stage tandem with per-stage service $T/2 = 4 \mu\text{s}$ and hop $h = 1.7 \mu\text{s}$:

	msg 1	msg 2	msg 3
single-stage arrival	0	1	2
single-stage latency (μs)	8.0	15.0	22.0
tandem arrive stage 1	0.0	1.0	2.0
tandem wait stage 1	0.0	3.0	6.0
tandem leave stage 1	4.0	8.0	12.0
tandem arrive stage 2 ($+h$)	5.7	9.7	13.7
tandem wait stage 2	0.0	0.0	0.0
tandem leave stage 2	9.7	13.7	17.7
tandem latency (μs)	9.7	12.7	15.7

At wall-clock $t \approx 7$, stage 1 is processing message 2 while stage 2 is processing message 1. Message 1 pays the $1.7 \mu\text{s}$ hop cost (latency $8.0 \rightarrow 9.7$); message 3, at the end of the burst, falls from 22.0 to $15.7 \mu\text{s}$. The median rises and the tail falls, at the same message count, with throughput $2/T$ rather than $1/T$. This is the mechanism of Proposition 1.

A chain of synchronous `fast_sends` on one thread shortens no queue and pays $(N - 1)h$; only stages on their own threads do (Section 10).

Calibration and cost. The paper reports numerical results at $h = 1.7 \mu\text{s}$ — the one-way async mailbox cost obtained by halving the round-trip figure in Table 4 of Maciejewski [36] — swept across $T \in \{2, 4, 8, 16, 32, 64, 128\} \mu\text{s}$ to cover the sub-hop regime through the slow-servicing regime. The simulator is deterministic given the tape and (T, h, N) ; one $O(n)$ pass per stage per window makes its wall-clock cost negligible next to the tape read.

3.5 Sweep design

The service times are a power-of-two grid; $T = 8 \mu\text{s}$ is the sweep point just above the publisher period $s_{\text{pub}} \approx 7.5 \mu\text{s}$ (Section 5.4), and “the sweep point just above the publisher period” means that cell throughput. The design asks whether an N -stage tandem reduces p_{99} latency under real Hawkes packet arrivals and whether the same architecture achieves the same compression under a rate-matched Poisson null. For each 30-minute window we run the tandem Lindley of Section 3.3 at $N \in \{1, 2, 4, 8\}$ under two arrival regimes: the real packet-arrival timestamps from the tape (Hawkes arm, stream H) and a uniform redraw of the same $n_{\text{pkt}}^{\#}$ arrivals over the same support (the uniform Poisson null P); the further null streams that separate the ingredients of the clustering are defined in Section 3.6. The support is the observed packet span $[\min \text{arr}_H, \max \text{arr}_H]$ rather than the nominal window $[T_0, T_1]$, so the null is rate-matched to the arrivals it replaces rather than to wall-clock; the two coincide on busy windows and the former is the stricter counterfactual on sparse ones. The null is therefore a *conditional homogeneous-Poisson* process: a homogeneous Poisson process at rate $n_{\text{pkt}}^{\#}/(\max \text{arr}_H - \min \text{arr}_H)$ conditioned on the observed count over the observed support. Per (window, N , regime) we record $p_{50}, p_{95}, p_{99}, p_{99.9}$, max of end-to-end latency. Aggregation is corpus-median across all windows; the two headline curves per scenario are $p_{99}(N)$ and $p_{50}(N)$ under Hawkes and Poisson overlaid.

The paper’s empirical calibration is $h = 1.7 \mu\text{s}$ (one-way async cross-thread mailbox cost, per Table 4 of Maciejewski [36], halved for one-way) swept over $T \in \{2, 4, 8, 16, 32, 64, 128\} \mu\text{s}$ so the scenarios span from the sub-hop regime through the slow-servicing regime. Theorem 1 predicts $p_{99}^H(N) - (N - 1)h - T \leq (p_{99}^H(1) - T)/N$ on every window, i.e. a fitted exponent $\gamma \geq 1$ in $\Delta(N) \approx \Delta(1)N^{-\gamma}$; Corollary 2 predicts $p_{99}^P(N) = p_{50}^P(N) = T + (N - 1)h$ exactly in every cell whose per-stage utilisation satisfies $\rho/N < 0.01$, and a strictly positive Poisson wait in every cell where it does not; the prediction is two-sided, so a Poisson p_{99} sitting at the no-queue latency at $\rho/N > 0.01$ would falsify it just as a Poisson p_{99} above the no-queue latency at $\rho/N < 0.01$ would. Any of (i) a window on which the Hawkes bound is violated, (ii) a Poisson $p_{99}(N)$ on the wrong side of the $\rho/N = 0.01$ boundary, or (iii) Hawkes $p_{50}(N)$ growing at slope $\gg h$ per stage falsifies the paper’s model of the simulator.

3.6 Null streams and the ingredient each removes

The claim under test is causal: the clustering of arrivals produces the queueing tail. A tail in the Lindley recursion can be built by four distinct properties of an arrival stream, and the real stream has all four at once:

1. the *rate*, the number of packets per second;
2. the *gap distribution*: how often two consecutive packets land closer together than the service time T , regardless of the order in which those tight gaps occur;
3. the *ordering*: whether the tight gaps come in runs (a burst of ten packets $5 \mu\text{s}$ apart) or lie singly among long gaps;
4. *slow rate variation*: busy and quiet stretches at the scale of seconds within a window.

A null stream is a synthetic stream that keeps some of these and destroys the rest. Running the identical simulator on the real stream and on the null, and comparing the tails, measures the share of the tail that the destroyed property was carrying. Each null is a controlled subtraction of one ingredient, and the design is factorial in the sense that the four properties are removed in different combinations by different nulls. The reference quantity throughout is the single-stage p_{99} excess, $p_{99} - T$, and the statistic reported is the fraction of the real stream’s excess that survives on the null, corpus-median over the 3512 windows. All nulls are built per window from that window’s own real stream, with seeds derived from the session and window identifiers so that every run is reproducible. The 3512 windows are nested in 276 sessions, so the corpus medians reported here are point estimates and no interval is attached to them.

Packet-level nulls. The uniform Poisson null P of Section 3.5 redraws every packet time uniformly over the window: it keeps (1) and destroys (2), (3) and (4) together, so a tail that vanishes under it is attributable to clustering in some form but not to any particular form. The *gap shuffle* G takes the window’s sequence of interarrival gaps, permutes it at random (Fisher–Yates, the renewal null already used for the Fano diagnostic of Section 3.2), and re-accumulates arrival times: the same packet count, the same multiset of gaps and therefore the same tight-gap fraction and the same shortest gap, s_{pub} , with the ordering destroyed. G keeps (1) and (2) and removes (3) and (4). A tail that survives G is set by how many tight gaps the stream contains; a tail that G removes needs the runs. The *1s-binned Poisson null* B cuts the window into one-second bins, keeps each bin’s real packet count, and redraws the arrivals uniformly inside the bin: it keeps (1) and (4) and removes (2) and (3). A tail that survives B is a busy-second load effect, not a microsecond one. Section 4.2 reports all three.

stream	kept	destroyed	question answered
H	the real stream	—	reference
P	packet count, support	gap distribution, ordering, slow rate variation	is the tail due to clustering in any form?
G	count; the multiset of gaps, hence the tight-gap fraction and the shortest gap	ordering of gaps into runs; slow rate variation	is the tail set by how many tight gaps there are, or by their arrangement?
B	count per one-second bin	microsecond gap structure and ordering	is the tail busy-second load?
TG	every transaction intact, in its original order	the idle gaps between transactions, permuted	does the clustering of transactions into runs build the tail?
TP	every transaction intact	transaction start times, redrawn uniformly	does transaction timing matter at all?
TS	real start times	which transaction shape sits at which start (shapes permuted across starts)	do large transactions arriving at particular moments matter?
TM	real start times	the split of a transaction into packets (each transaction becomes one packet)	does the exchange’s packetisation matter?
TW	real start times, the split into packets	the tight spacing inside a split transaction (packets spread by the median idle gap)	does back-to-back spacing inside a transaction matter?

Table 2. The nine arrival streams. P , G and B act on packet times with constant service and are reported in Table 4; TG–TW act on transaction blocks with span-dependent service and are reported in Table 8. Reading the results together: $P \approx 0$ and $TP \approx 0$ say the tail is produced by the timing of the transaction process; TS, TM, TW ≈ 1 say sizes, packetisation and intra-transaction spacing are not the mechanism; G and TG, which remove the same thing at two levels of description, give the same profile across T and split the clustering into the two channels named in Section 4.3.

Transaction-level nulls. The packet nulls cannot distinguish two accounts of the tight-gap surplus: that the exchange splits one matching-engine event across back-to-back packets, or that the events themselves arrive clustered and the packets relay them. Section 4.3 therefore groups every message by its `transactTime`, the identifier of one matching-engine event (a *transaction*), and builds five further streams in which each transaction is a block — its packets, their message counts and their internal spacing — and exactly one ingredient is removed. Service in these runs is charged per packet at its measured cost, $S_i = T(1 + r(\sigma_i - 1))$ with σ_i the message count and r the live decoder’s ratio of per-message slope to decode floor, so that a large packet adds work rather than redistributing it; if the size of a transaction matters to the tail, these runs can show it. Table 2 lists all nine streams.

The logic of the two families is the same and they are read together. If P and TP remove the tail, the tail is produced by the timing of the transaction process. If TS, TM and TW leave it, the size of transactions, their division into packets, and the spacing of those packets are not the mechanism, and the exchange is relaying events rather than creating bursts. G and TG remove the same ingredient — the arrangement of events into runs — at the packet and at the transaction level respectively; agreement between them identifies the packet-level ordering channel with the clustering

of transactions. What survives G is the surplus of tight gaps over a rate-matched Poisson stream, which Table 6 measures directly. That surplus shows the gap distribution is far from exponential; it does not show what produces it, and G itself is a stream with the surplus and no self-excitation. What G removes, the runs, is the self-exciting component.

4 Empirical results

4.1 Single-thread and tandem tails on the real stream and on the Poisson null

This section presents the main result. It runs the simulated pipeline over the whole corpus and reports, for each task length and each stage count, how long a typical message takes and how long the slowest one in a hundred takes. Each case is run twice: once on the real packet arrivals, which come in bursts, and once on a control in which the same number of packets is scattered uniformly at random over the window. The comparison isolates the effect of bursts. The findings are these. On randomly scattered arrivals there is almost no slow tail at the task lengths a trading hot path uses, so splitting a task across threads gains nothing and costs a hop per stage. On the real, bursty arrivals a slow tail appears once a task is longer than about seven microseconds, and it grows quickly with task length, until the slowest messages take tens of times longer than the task itself. There, splitting the task into two or more stages cuts the tail sharply for a small, fixed cost on the typical message, and the longer the task, the more stages pay off. In short, it is the bursts, not the average load, that create the tail, and that is why splitting helps on real data and not on the random control.

Table 3 gives the corpus-median latency for every service time T and stage count N at $h = 1.7 \mu\text{s}$, and Figure 2 plots it. The Poisson columns are the uniform null P of Section 3.5, with the same packet count per window.

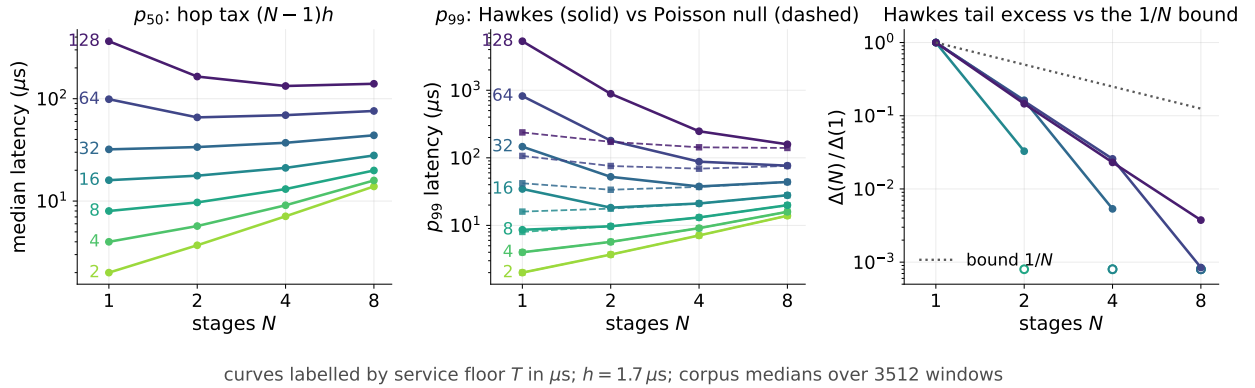


Figure 2. Corpus-median latency versus stage count N for each service time T (one shade per T , each curve labelled at its $N = 1$ end). Left: median. Centre: p_{99} , Hawkes solid and Poisson null dashed. Right: the Hawkes tail excess ratio $\Delta(N)/\Delta(1)$ against the Theorem 1 bound $1/N$ (dotted); open markers are cells where the tandem drives $\Delta(N)$ to exactly zero, drawn at the bottom of the panel because zero cannot be placed on a log axis. The median rises by exactly h per hop wherever the queue is empty at the median, which is every cell except the three longest-service Hawkes cells. The Hawkes p_{99} falls with N only at $T \geq 16 \mu\text{s}$; at $T \leq 8 \mu\text{s}$ there is no tail to compress and it rises with the hop count, tracking the median. The Poisson p_{99} likewise tracks the median up to $T = 16 \mu\text{s}$ and separates from it at $T \geq 32 \mu\text{s}$, where utilisation queueing begins.

On the Poisson stream almost every message finds the queue empty, so every quantile equals the no-queue latency up to $T = 16 \mu\text{s}$. A wait appears first at $32 \mu\text{s}$, where utilisation passes the 1% at which Corollary 2 predicts it at p_{99} ; from there on the Poisson stream behaves as an ordinary queue,

T	N	Hawkes (real arrivals)			Poisson null		
		p_{50}	p_{99}	$p_{99.9}$	p_{50}	p_{99}	$p_{99.9}$
2	1	2.00	2.00	2.00	2.00	2.00	2.00
2	2	3.70	3.70	3.70	3.70	3.70	3.70
2	4	7.10	7.10	7.10	7.10	7.10	7.10
2	8	13.90	13.90	13.90	13.90	13.90	13.90
4	1	4.00	4.00	4.00	4.00	4.00	5.78
4	2	5.70	5.70	5.70	5.70	5.70	5.70
4	4	9.10	9.10	9.10	9.10	9.10	9.10
4	8	15.90	15.90	15.90	15.90	15.90	15.90
8	1	8.00	8.61	10.14	8.00	8.00	13.79
8	2	9.70	9.70	9.70	9.70	9.70	11.48
8	4	13.10	13.10	13.10	13.10	13.10	13.10
8	8	19.90	19.90	19.90	19.90	19.90	19.90
16	1	16.00	34.59	53.97	16.00	16.00	29.84
16	2	17.70	18.31	19.84	17.70	17.70	23.49
16	4	21.10	21.10	21.10	21.10	21.10	22.88
16	8	27.90	27.90	27.90	27.90	27.90	27.90
32	1	32.00	146.14	259.11	32.00	42.02	62.02
32	2	33.70	52.29	71.67	33.70	33.70	47.54
32	4	37.10	37.71	39.24	37.10	37.10	42.89
32	8	43.90	43.90	43.90	43.90	43.90	45.68
64	1	98.88	820.70	1548.68	64.00	106.73	126.72
64	2	65.70	179.84	292.81	65.70	75.72	95.72
64	4	69.10	87.69	107.07	69.10	69.10	82.94
64	8	75.90	76.51	78.04	75.90	75.90	81.69
128	1	364.45	5316.85	12487.72	128.00	237.65	289.07
128	2	164.58	886.40	1614.38	129.70	172.43	192.42
128	4	133.10	247.24	360.21	133.10	143.12	163.12
128	8	139.90	158.49	177.87	139.90	139.90	153.74

Table 3. Corpus-median latency quantiles in μs for each service time T and stage count N at $h = 1.7 \mu s$, on the real NQ arrivals and on a Poisson stream with the same packet count per window. What the table shows: the median is $T + (N - 1)h$ in almost every cell, so each stage costs one hop. On the real arrivals the single-thread p_{99} equals T up to $4 \mu s$ and then grows much faster than T , to about forty times T at $128 \mu s$; on the Poisson stream it stays at T up to $16 \mu s$. One cut removes most of the real stream's tail for one hop, and further cuts keep helping until each stage is shorter than the publisher period. What follows: the tail comes from how the real packets are bunched, not from load. A server queues only when consecutive gaps are shorter than its service time, and the real stream has many times the Poisson share of such gaps (Table 6) and arranges them in runs.

and splitting helps it too. The $p_{99.9}$ column shows the same pattern at utilisations ten times lower (Section 6). Under Poisson arrivals, then, splitting is not useless in general; it is useless over the service times an HFT hot path occupies.

On the real stream a tail appears only above a threshold. At $T = 2$ and $4\mu\text{s}$ there is none, and the real stream cannot be told apart from the Poisson one. Between 4 and $8\mu\text{s}$ a tail appears, and it then grows much faster than the service time. Splitting into two stages removes most of it at every service time from $16\mu\text{s}$ up, for one hop on the median, and wins on every window there. The best stage count rises with T , from one at $8\mu\text{s}$ and below to eight at $64\mu\text{s}$ and above (Table 3). Section 4.8 locates the onset more precisely and Section 4.9 explains what sets it.

Findings.

- *No tail below the publisher period.* At $T \leq 4\mu\text{s}$ the p_{99} equals T on real and Poisson arrivals alike.
- *Above it, the real stream's tail grows much faster than T ,* to about forty times T at $128\mu\text{s}$.
- *The Poisson stream at the same rate has no tail up to $16\mu\text{s}$,* and above that only a much smaller utilisation tail.
- *One cut removes most of the tail for one hop* and wins on every window from $16\mu\text{s}$.
- *The best stage count rises with T ,* from one at $8\mu\text{s}$ to eight at $64\mu\text{s}$ and above.

4.2 Gap-shuffle and 1 s-binned nulls

The Poisson null of Table 3 redraws every arrival uniformly over the whole window. That removes three things at once: the microsecond-scale self-excitation, the slower rate variation across the window, and the shape of the gap distribution, including the publisher period of Section 4.9. The tail difference in Table 3 is credited to the first of these, but the mechanism of Section 4.9 runs on the third: a queue forms when consecutive gaps are shorter than the service time, and that is a property of the gap distribution, which a reordering of the same gaps leaves untouched. Two further nulls separate the three. The *gap shuffle* permutes each window's interarrival gaps at random (the renewal null of Section 3.2): same count, same gap distribution, same publisher period, no ordering. The *1 s-binned Poisson* keeps each one-second bin's own packet count and redraws the arrivals uniformly inside the bin: same second-scale rate variation, no microsecond clustering.

T	N	real	uniform	gap shuffle	1 s-binned
2	1	2.00	2.00	2.00	2.00
2	2	3.70	3.70	3.70	3.70
2	4	7.10	7.10	7.10	7.10
2	8	13.90	13.90	13.90	13.90
4	1	4.00	4.00	4.00	4.00
4	2	5.70	5.70	5.70	5.70
4	4	9.10	9.10	9.10	9.10
4	8	15.90	15.90	15.90	15.90
8	1	8.61	8.00	8.54	8.00
8	2	9.70	9.70	9.70	9.70
8	4	13.10	13.10	13.10	13.10
8	8	19.90	19.90	19.90	19.90
16	1	34.59	16.00	30.75	16.00
16	2	18.31	17.70	18.24	17.70
16	4	21.10	21.10	21.10	21.10
16	8	27.90	27.90	27.90	27.90
32	1	146.14	42.02	99.61	47.36
32	2	52.29	33.70	48.45	33.70
32	4	37.71	37.10	37.64	37.10
32	8	43.90	43.90	43.90	43.90
64	1	820.70	106.73	340.62	112.62
64	2	179.84	75.72	133.31	81.06
64	4	87.69	69.10	83.85	69.10
64	8	76.51	75.90	76.44	75.90
128	1	5316.85	237.65	1306.63	246.03
128	2	886.40	172.43	406.32	178.32
128	4	247.24	143.12	200.71	148.46
128	8	158.49	139.90	154.65	139.90

Table 4. Corpus-median p_{99} in μs under the real arrivals and the three packet-level null streams of Section 3.6, for every service time T and stage count N . Uniform is the Poisson null; the gap shuffle keeps every gap and destroys their order; the 1 s-binned stream keeps each second’s packet count and destroys the microsecond structure. What the table shows: at $T \leq 4 \mu\text{s}$ every stream sits at the no-queue latency, because no gap is that short. For a single thread above that, the uniform and binned streams lose almost all of the real stream’s tail, while the gap shuffle keeps most of it at short service times and less as T grows (Figure 3). Once a cut takes each stage below the publisher period, all four streams meet at the no-queue latency. What follows: the tail is not busy-second load. At short service times it is set by how many gaps are shorter than T , since a single short gap costs the next packet one wait whether or not its neighbours are short. At long service times it needs runs of short gaps, and runs are what the shuffle destroys. Session-bootstrap 95% intervals on the shares are within about two points of the reported values.

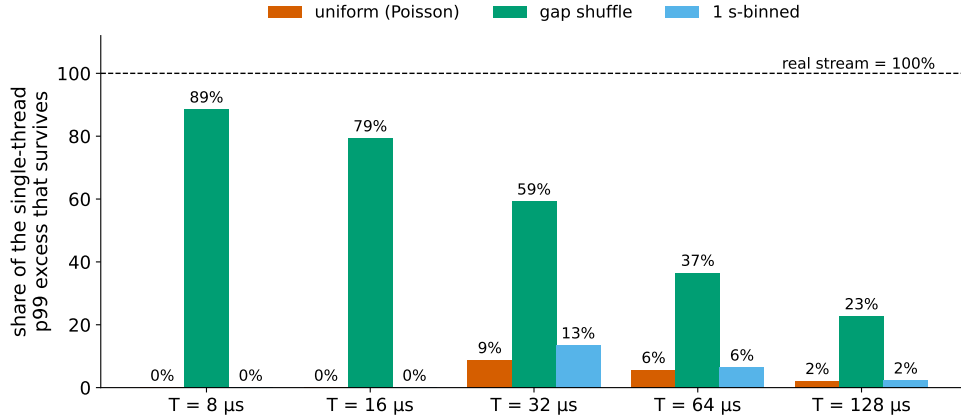


Figure 3. Table 4 as bars: the share of the single-thread p_{99} excess ($p_{99} - T$) that survives each null stream, the real stream being 100%. The gap shuffle (green), which keeps every gap and destroys their order, keeps most of the tail at 8–16 μ s and less as T grows: at short service times the tight gaps carry the tail, at long ones their arrangement into runs. The uniform and 1 s-binned streams keep almost none at any T .

Table 4 and Figure 3 give the result. The 1 s-binned stream keeps none of the tail at 8 and 16 μ s and little above, so the tail is not busy-second load. The gap shuffle keeps about 80% of it at 16 μ s and less as the service time grows, about a quarter at 128 μ s. At the service times of an HFT hot path, then, the tail is set mainly by how many gaps are shorter than the service time, and the arrangement of those gaps into runs matters only from about 64 μ s. Section 4.3 traces both to the timing of matching-engine transactions: the short gaps are consecutive transactions sent back to back at the publisher period (Table 14), and the runs are the self-exciting component. Which process places two transactions within microseconds of each other at the engine is not identified by any stream run here (Section 13.3).

Findings.

- *The tail is not busy-second load.* Keeping every second’s packet count but placing packets at random within it leaves almost no tail.
- *At short service times the number of short gaps sets the tail.* Keeping every gap but shuffling their order keeps about 80% of it at 16 μ s.
- *At long service times the runs of short gaps set it.* The same shuffle keeps only about a quarter at 128 μ s.

4.3 Grouping packets into matching-engine transactions

The nulls of Section 4.2 act on packet times. A packet is the exchange’s unit, not the market’s. CME MDP3 publishes each matching-engine event — a *transaction*, identified by the common `transactTime` carried by every message it generates — as one or more UDP datagrams, and a datagram may carry the end of one transaction and the start of the next. Two accounts of the tight-gap surplus are therefore open. Either the exchange splits transactions across back-to-back packets, in which case the clustering is an artefact of packetisation and a receiver that reassembled transactions would see none of it; or the transactions themselves arrive clustered and the packets relay them. The corpus decides between the two. Every message in the corpus was grouped by `transactTime` and every packet labelled with the transactions it carries, about 3×10^9 of each.

About 3% of packets carry parts of two transactions: the exchange’s publisher packs transactions that reach it while it is busy into one packet (Section 5.5).

Consecutive transactions and the gap between them. A *transaction* is one matching-engine event: all messages that carry the same `transactTime`. One incoming order that trades against three resting orders is one transaction, however many messages it produces. Within a session the NQ front-month transactions are put in the order the engine processed them, $A, B, C, D, \dots$; two transactions are *consecutive* when one directly follows the other in that order, with no other NQ front-month transaction between them (A and B , B and C , and so on). The *gap between consecutive transactions*, their inter-arrival time, is the time from one to the next. It is measured on one of two clocks, always named: the matching engine’s (the difference of the two `transactTimes`) or the publisher’s (the difference of the `sendingTimes` of the packets that carry them, zero if one packet carries both). A session with m transactions has $m - 1$ such gaps. Every statement in the paper about how close transactions are is a statement about these gaps; a *tight* gap is one below $16\mu\text{s}$.

	1	2	3	4	5	6–10	11–20	21+
messages per packet	95.897	2.439	0.869	0.401	0.188	0.171	0.027	0.009
packets per transaction	98.295	1.689	0.011	0.002	0.001	0.001	0.000	0.000
messages per transaction	95.362	2.729	0.822	0.558	0.195	0.280	0.038	0.016

Table 5. Size distributions, percent of the pooled corpus (about 3×10^9 packets and as many transactions). What the table shows: almost every packet carries one message and almost every transaction fits in one packet; a transaction split across three or more packets is very rare. The message count per transaction is the message count per packet with the split transactions folded back in.

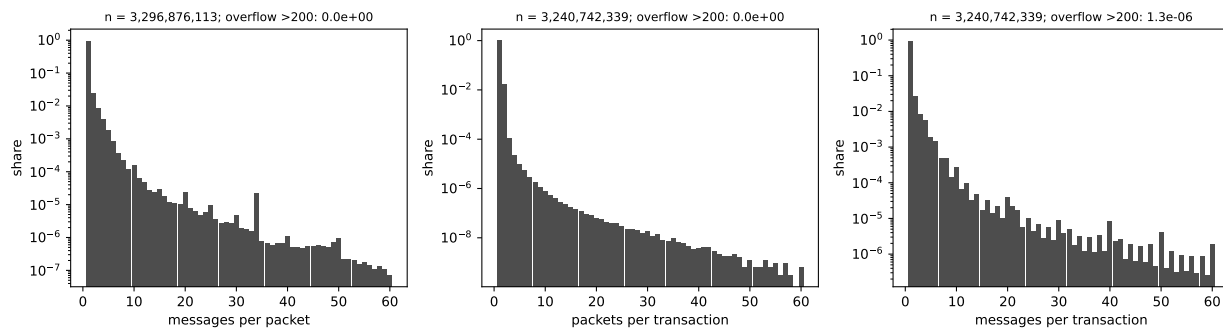


Figure 4. The three size distributions of Table 5 on a logarithmic share axis, counts 1–60. Messages per packet and per transaction have the same geometric decay; packets per transaction has almost none beyond two.

Packetisation and the tight gaps. Table 5 gives the three size distributions, and Figure 4 plots them. A split transaction is rare, and its packets are not sent back to back: they arrive about $15\mu\text{s}$ apart, above the publisher period, and almost never closer (Figure 6, left, and Figure 8, right). The tight gaps lie between different transactions (Table 6).

gap shorter than	share of all packet gaps	of which inside one transaction	Poisson at the same rate
7.9 μ s (spacing)	2.15%	0.1%	0.56%
17.8 μ s	20.6%	5.0%	1.24%
35.5 μ s	34.1%	4.6%	2.46%

Table 6. Short packet gaps, pooled over the corpus. The rows are the bin edges of the logarithmic grid of Figure 6 nearest to the publisher spacing and to the 16 and 32 μ s service times. What the table shows: the real stream has several times the Poisson share of gaps below each of these service times, about one gap in five below 16 μ s against about one in a hundred, and almost none of these short gaps lie inside a single transaction. What follows: the short gaps that make a server queue are gaps between different matching-engine events, not the exchange’s division of one event into several packets, so a receiver that reassembled transactions before decoding would remove almost none of them. Section 5 shows where they come from: the engine processes many transactions within a microsecond of each other, and the publisher sends them back to back at its spacing (Table 14).

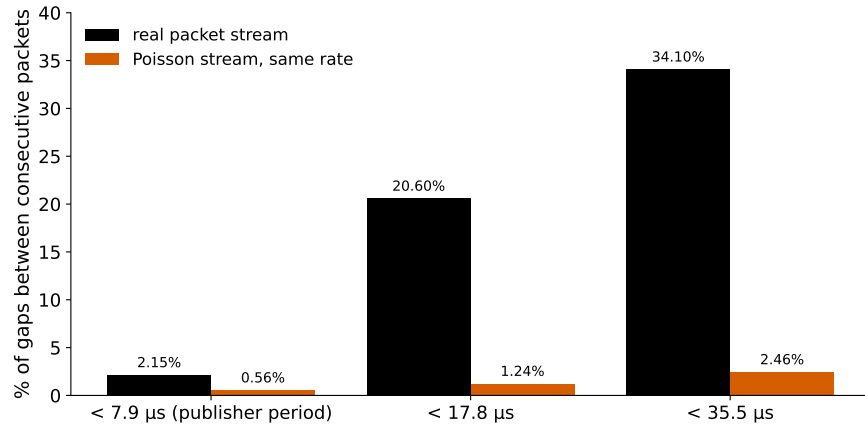


Figure 5. Table 6 as bars: the share of gaps between consecutive packets shorter than the publisher period, than about 16 μ s and than about 32 μ s, on the real stream (black) and on a Poisson stream with the same packet count (vermillion). Below 16 μ s the real stream has about one gap in five against about one in a hundred. A server with a service time in this range queues on each of these gaps, which is why the real stream has a tail and the Poisson stream does not.

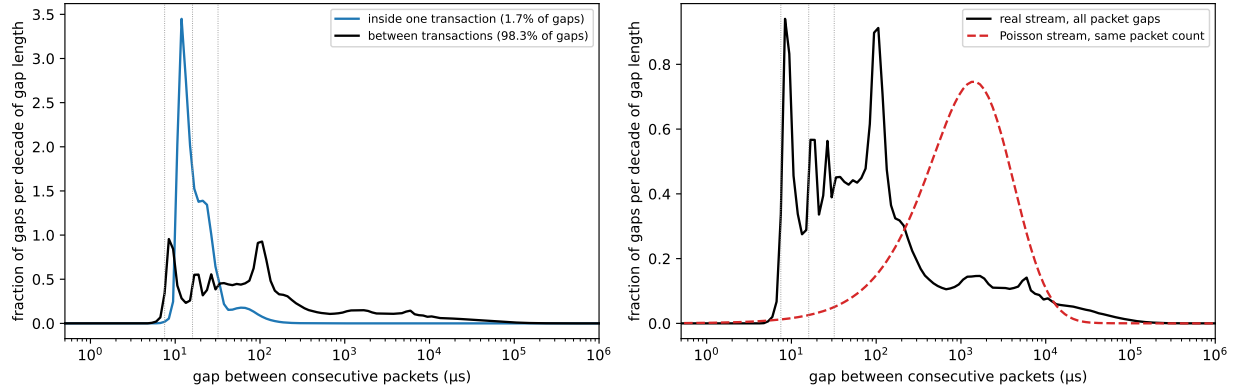


Figure 6. Packet gaps, pooled over the corpus. Left: gaps between the packets of one split transaction (blue) against gaps between packets of different transactions (black). The pieces of a split transaction arrive about $15 \mu\text{s}$ apart and almost never closer than the publisher period; the region from the publisher period to about $12 \mu\text{s}$ is filled by the black curve alone. So the gaps that make a receiver queue are between packets of different events. Right: all packet gaps of the real stream (black) against a Poisson stream with the same packet count (red, dashed), the null of Table 3 drawn in gap space. The Poisson density is one hump near 1 ms ; the real density puts about a third of its gaps below $32 \mu\text{s}$, with an edge at the publisher period and a mode near $100 \mu\text{s}$ that the null lacks. That region of short gaps is why the real stream has a tail at HFT service times and the Poisson stream does not. Axes: logarithmic gap axis; vertical axis the fraction of gaps per decade of gap length, so the area under a curve over a range is the fraction of its gaps in that range. Dotted lines at 7.5 , 16 and $32 \mu\text{s}$.

Self-excitation of the transaction stream. Treating each transaction as one event at the time of its first packet, the transaction stream carries the same clustering as the packet stream (Table 7): the variability of transaction counts grows with the counting window just as that of packet counts does, shuffling the idle gaps between transactions removes most of the growth, and the Hawkes fit gives the same branching ratio, about 0.8 , on both. The gap density of transaction starts (Figure 8, left) is the same as that of packets and unlike the Poisson curve at every scale. The exchange publishes each matching event as it occurs, and the packet stream inherits the clustering of the events. Why the events cluster is a question about the market, outside the scope of this paper (Section 13.3).

count window τ	transactions	transactions, idle gaps shuffled	packets
1 ms	7.6	5.5	7.8
10 ms	21.6	10.8	22.2
100 ms	50.5	15.8	51.8
1 s	105.4	17.1	108.4
branching ratio n	0.795	—	0.798

Table 7. Fano factor $\text{Var}[C_\tau]/\text{E}[C_\tau]$ of event counts, corpus median over 3512 windows, for transaction starts, for the same starts with the idle gaps between transactions randomly permuted (transactions kept intact), and for packets; Poisson gives 1 at every τ . Last row: exponential-kernel Hawkes branching ratio, corpus median, all fits converged. The within-window Fano values agree with those of Section 3.2 at the same bin width.

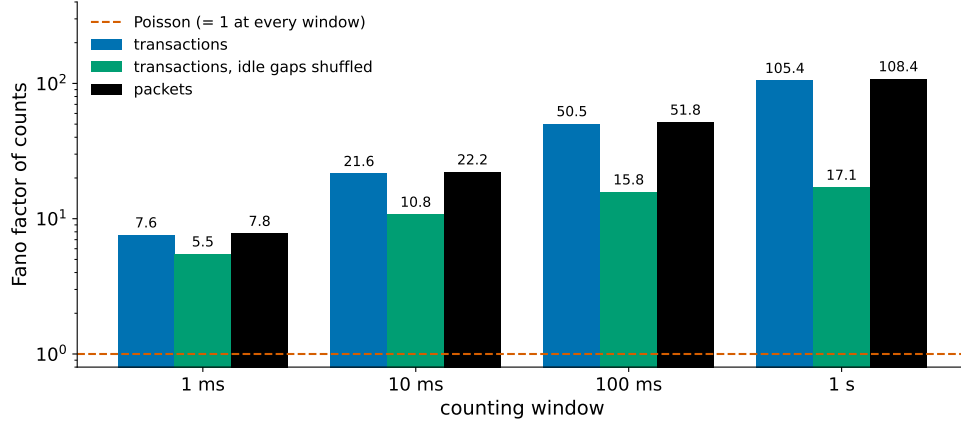


Figure 7. Table 7 as bars, log scale: how much more variable the counts are than Poisson (Fano factor, 1 for Poisson, dashed) in counting windows from 1 ms to 1 s. Transactions (blue) and packets (black) are almost identical and grow from about 8 to over 100: the packet stream inherits its clustering from the transactions. Shuffling the idle gaps between transactions (green) removes most of the growth: what is left is the over-dispersion of the gap lengths themselves.

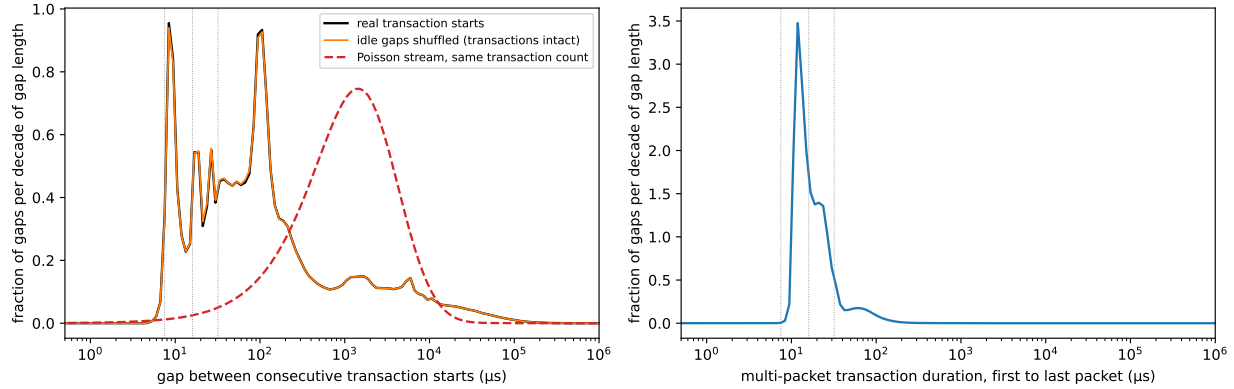


Figure 8. Gaps between consecutive *transaction starts*, to be read against Figure 6, whose gaps are between consecutive *packets*. Left: each transaction collapsed to one event at the time of its first packet, for the real stream (black), for the same events with the idle gaps between transactions shuffled (orange), and for a Poisson stream with the same transaction count (red, dashed). The black curve is the black curve of Figure 6: the same edge at the publisher period and the same mode near $100 \mu\text{s}$, because almost every transaction is one packet. The orange curve lies on the black one by construction, since shuffling keeps the gaps. The Poisson curve has a different shape. What follows: the clustering seen at the packet level is present unchanged at the level of matching-engine events, which is what Table 7 shows numerically and Table 8 shows in the simulator. Right: time from the first to the last packet of a split transaction, about $15 \mu\text{s}$ at the median. Axes as in Figure 6.

The tail under the transaction-level nulls. To separate the parts of the transaction process that build the queue, each window is re-simulated at $N = 1$ with service charged per packet at its measured absolute cost, $S_i = T(1 + r(\sigma_i - 1))$ with σ_i the packet’s message count and r the live decoder’s ratio of per-message cost to decode floor (Section 7.3), so that a large packet adds work rather than redistributing it. Five counterfactual streams are built from the real one (H), each removing one ingredient. TG keeps every transaction intact — its packets, their sizes and their

spacing — and permutes the idle gaps between transactions, removing the clustering of transactions. TP keeps the transactions intact and redraws their start times uniformly over the window. TS keeps the real start times and permutes the transactions’ shapes among them, decoupling size from timing. TM merges each transaction into a single packet at its start. TW keeps the starts and spreads a split transaction’s packets by the median idle gap instead of their real spacing. Table 8 reports the corpus-median p_{99} of each stream and the share of the real stream’s excess over T that it retains.

T	H	TG	TP	TS	TM	TW
16	34.8	31.0 (80)	20.7 (25)	37.4 (114)	34.9 (101)	35.6 (104)
32	146.8	100.5 (60)	52.9 (18)	155.9 (108)	146.3 (100)	148.1 (101)
64	832	345 (37)	118 (7)	839 (101)	795 (95)	833 (100)
128	5460	1319 (22)	251 (2)	5218 (95)	5035 (92)	5459 (100)

Table 8. Which part of the transaction process builds the queue. Each row is one service time T ; each column is the single-thread p_{99} in μs (corpus median, service charged per packet at NQ’s cost per message) when the simulator is driven by the real stream H or by one of five rebuilt streams, each identical to the real one except for one removed ingredient. The number in parentheses is the share, in percent, of the real stream’s tail ($p_{99} - T$) that the rebuilt stream still produces: 100 means the ingredient did not matter, 0 that it was the whole tail. The columns ask: TG, if the transactions are kept intact but no longer come in bunches, does the tail survive? TP, if each transaction is put at a random moment? TS, if big and small transactions swap places? TM, if every transaction is sent as one packet? TW, if the pieces of a split transaction are spread apart? What the table shows: TS, TM and TW keep about all of the tail in every row, so the size of transactions and how the exchange packs them do not build the queue. TP removes most of it at every service time, so the timing of transactions does. TG removes little at $16 \mu\text{s}$ and most at $128 \mu\text{s}$: a short server is hurt by each short gap on its own, a long server only by sustained runs, and runs are what TG breaks up. This is the profile the gap shuffle gave at the packet level (Table 4). At $T \leq 8 \mu\text{s}$ the real tail is under $1 \mu\text{s}$ and the shares are not meaningful. Using ZN’s cost per message instead of NQ’s moves every cell by at most a few points, and session-bootstrap intervals are within about two points.

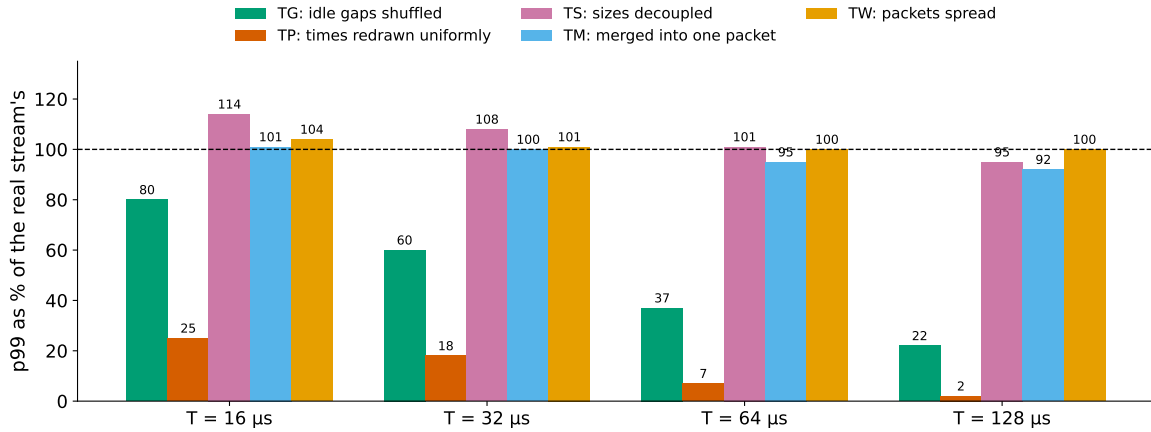


Figure 9. Table 8 as bars: each transaction-level arm’s p_{99} as a percentage of the real stream’s. Redrawing transaction times uniformly (TP, vermillion) removes the tail; shuffling idle gaps (TG, green) removes more of it the longer the service time. Decoupling sizes (TS), merging each transaction into one packet (TM) or spreading its packets (TW) leaves it near 100% from $16 \mu\text{s}$ up.

Summary in plain terms. Each rebuilt stream changes one thing about the real transactions and keeps everything else. What removes the tail, and at which service time T :

- *Putting each transaction at a random moment (TP) removes the tail at every T :* about three quarters of it at $16\mu\text{s}$ and almost all of it from $64\mu\text{s}$. When transactions happen is what builds the queue.
- *Keeping every gap between transactions but shuffling their order (TG) removes little at short T and most at long T :* about 80% of the tail remains at $16\mu\text{s}$, about a fifth at $128\mu\text{s}$. A fast server is hurt by each short gap on its own, and the shuffle keeps every short gap. A slow server is hurt by long runs of transactions close together, and the shuffle breaks the runs up.
- *Nothing about what the transactions contain or how the exchange packs them removes the tail from $16\mu\text{s}$ up.* Swapping big and small transactions around (TS), sending each transaction as one packet (TM), or spreading a transaction's packets apart (TW) leaves the tail about as it was.
- *At $T \leq 8\mu\text{s}$, the sweep points at or just above the publisher period, these rebuilt streams say nothing.* The real stream's tail there is too small to split into shares. At the decode floor of a production receiver the tail comes mostly from decoding long packets one message after another (Section 7.6), which these streams were not built to test.

Three things follow. The tail is the transaction process: putting transactions at random moments removes most of it. From $16\mu\text{s}$ up, sizes and packetisation do not enter, at ZN's cost per message as at NQ's. And the ordering of transactions into runs, which TG destroys, carries a small part of the tail at $16\mu\text{s}$ and most of it at $128\mu\text{s}$, the same profile the packet-level gap shuffle gave. The rest, dominant at HFT service times, is the surplus of short gaps between transactions over a Poisson stream at the same rate (Table 6), which on the publisher's clock sit at its spacing (Table 14). That surplus says the gap distribution is far from exponential; it does not say what put the mass there, and a stream with the same gaps and no self-excitation (the gap shuffle) reproduces it and most of the tail at $16\mu\text{s}$. The attribution is therefore the following. The queueing tail behind a single-threaded receiver on this feed is produced by the timing of matching-engine transactions; from $16\mu\text{s}$ up it is not produced by packetisation, by the message count per packet, or by the packet rate (at the decode floor of a production receiver the message count carries most of it, Section 7.6). It acts on a short service time through how often two transactions land within one service time of each other, and on a long one through how long the runs of them last, which is the self-excitation. Why transactions reach the engine within microseconds of each other is a question about the market and is left open (Section 13.3).

Findings.

- *The exchange does not split transactions into bursts.* Almost every transaction fits in one packet; the pieces of a split one arrive about $15\mu\text{s}$ apart; the short gaps lie between different transactions.
- *The transaction stream carries the clustering:* transactions and packets have the same count variability and the same branching ratio, about 0.8.
- *When transactions happen builds the tail.* Placing them at random moments removes most of it.
- *From $16\mu\text{s}$ up, what they contain and how they are packed does not:* swapping sizes, merging into one packet or spreading packets leaves the tail about as it was.

4.4 Tandem, sharding and dispatch on the same N cores

The tandem of Table 3 runs on N cores and the single thread on one, so part of what it buys is capacity: its throughput is N/T against $1/T$, and more capacity shortens queues under any arrival law. The design question is whether a chain of stages is the right way to spend the same N cores, and the two alternatives are the ones HFT systems use. *Sharding* runs N independent single-thread loops, each owning a disjoint slice of the stream, by instrument or by channel: full service T per message and no hops, so it has the lower median, and its tail depends on whether a burst lands on one shard or spreads over several. *Dispatch*, an M/D/ N queue, puts one queue in front of N identical servers each taking a whole packet at service T : the same capacity N/T as the tandem and no hops, so a burst of k packets drains in about the same time, while each packet’s own latency stays T rather than $T + (N - 1)h$.

Sharding is not available on this corpus: a CME burst is one instrument’s order flow, so sharding by instrument sends the whole burst to one core at service T and relieves nothing, and the corpus is a single instrument. Dispatch is available for any stage that holds no state from one packet to the next — SBE decode is such a stage, order-book application is not — provided its output is put back in order. Two comparisons follow. Table 9 puts dispatch beside the tandem with no ingress hop, no egress hop and no resequencing charged, which is the lowest latency dispatch could reach. Table 10 charges all three.

T	N	tandem		M/D/ N dispatch	
		p_{50}	p_{99}	p_{50}	p_{99}
2	1	2.00	2.00	2.00	2.00
2	2	3.70	3.70	2.00	2.00
2	4	7.10	7.10	2.00	2.00
2	8	13.90	13.90	2.00	2.00
4	1	4.00	4.00	4.00	4.00
4	2	5.70	5.70	4.00	4.00
4	4	9.10	9.10	4.00	4.00
4	8	15.90	15.90	4.00	4.00
8	1	8.00	8.61	8.00	8.61
8	2	9.70	9.70	8.00	8.00
8	4	13.10	13.10	8.00	8.00
8	8	19.90	19.90	8.00	8.00
16	1	16.00	34.59	16.00	34.59
16	2	17.70	18.31	16.00	16.00
16	4	21.10	21.10	16.00	16.00
16	8	27.90	27.90	16.00	16.00
32	1	32.00	146.14	32.00	146.14
32	2	33.70	52.29	32.00	48.04
32	4	37.10	37.71	32.00	32.00
32	8	43.90	43.90	32.00	32.00
64	1	98.88	820.70	98.88	820.70
64	2	65.70	179.84	64.00	171.38
64	4	69.10	87.69	64.00	72.50
64	8	75.90	76.51	64.00	64.00
128	1	364.45	5316.85	364.45	5316.85
128	2	164.58	886.40	128.00	868.98
128	4	133.10	247.24	128.00	223.06
128	8	139.90	158.49	128.00	128.00

Table 9. Real arrivals, corpus medians in μs : N cores as an N -stage tandem against the same N cores as one queue feeding N whole-packet servers, with no hop and no resequencing charged to the latter. What the table shows: uncharged, the pool is at or below the tandem everywhere, because it adds no hops. This is the most dispatch could achieve; Table 10 charges its costs.

T	N	tandem		dispatch, costs charged		reordered packets
		p_{50}	p_{99}	p_{50}	p_{99}	
16	2	17.70	19.59	19.40	20.78	0.00%
16	4	21.10	22.48	19.40	20.78	0.00%
16	8	27.90	29.28	19.40	20.78	0.00%
32	2	33.70	52.84	35.40	51.61	0.00%
32	4	37.10	39.86	35.40	38.16	0.01%
32	8	43.90	46.66	35.40	38.16	0.01%
64	2	65.70	181.08	67.40	175.74	0.01%
64	4	69.10	89.10	67.40	79.99	0.03%
64	8	75.90	81.42	67.40	72.92	0.03%
128	2	166.37	902.82	131.40	883.83	0.08%
128	4	133.10	250.40	131.40	227.94	0.14%
128	8	139.90	162.28	131.40	142.45	0.14%

Table 10. The same N cores as a tandem and as a dispatch pool with every cost charged: a hop into the servers, a hop out of them, and an in-order resequencer that holds each packet until its predecessors have left. Corpus medians in μs at $h = 1.7 \mu s$, service growing with message count at NQ’s cost per message; the last column is the share of packets that wait in the resequencer. What the table shows: dispatch’s median is $T + 2h$ at every N , against the tandem’s $T + (N - 1)h$, so dispatch is one hop slower at $N = 2$, level at three stages and faster from four. At $N = 2$ the two have about the same p_{99} ; from $N = 4$ dispatch is lower on both quantiles in every cell. Resequencing costs almost nothing. Constant service gives the same picture. What follows: for a stage that carries no state, a pool of four or more cores is at least as good as cutting it into as many stages (three cores were not simulated); at two cores the two are level. For a stage that carries state, the order book above all, dispatch is not available and cutting the chain is how extra cores are spent.

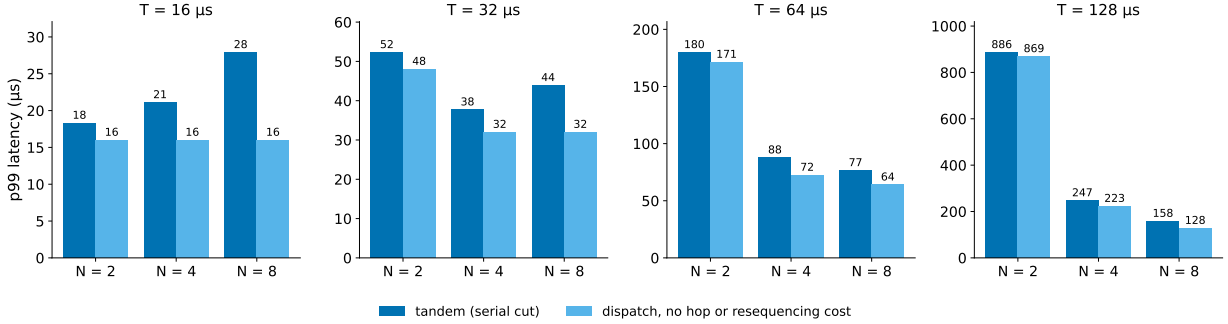


Figure 10. Table 9 as bars: p_{99} of a serial cut into N stages (dark blue) against a pool of N cores that each take a whole packet, with no hop and no resequencing charged (light blue). Uncharged, the pool is at or below the tandem everywhere, and at $16 \mu s$ it has no tail at all. This is the most dispatch could achieve; Figure 11 charges its costs.

Without its costs, dispatch’s p_{99} is at or below the tandem’s in every cell from $16 \mu s$ (Table 9), and its median is T against the tandem’s $T + (N - 1)h$.

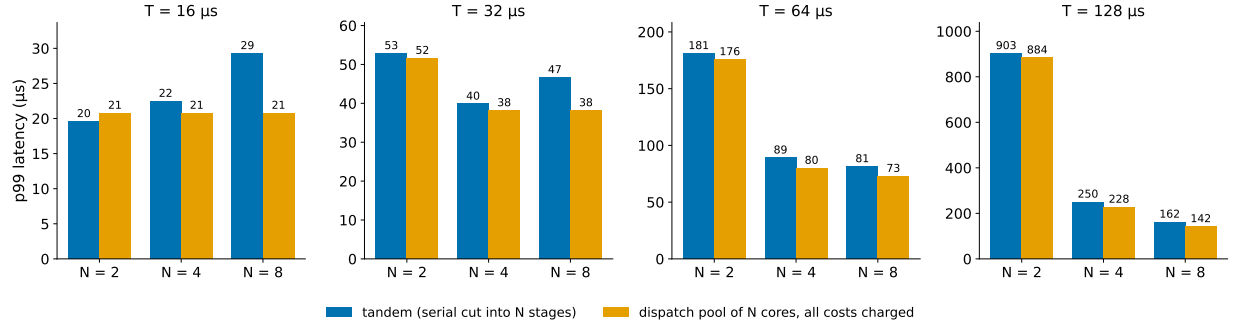


Figure 11. Table 10 as bars: p_{99} of a serial cut into N stages (blue) against a pool of N cores with the ingress hop, egress hop and resequencing charged (orange). At $N = 2$ the two are level; from $N = 4$ the pool is lower at every service time, and at $16 \mu s$ its p_{99} does not grow with N while the tandem's does.

With every cost charged (Table 10), dispatch pays two hops on every packet, in and out, and nothing else measurable: the resequencer holds almost no packets, because a packet that waited behind a burst in the shared queue leaves in the order it arrived. Its median is therefore $T + 2h$ whatever N is: one hop more than a two-stage tandem, the same as three stages, and less than four or more. Its tail is lower than the tandem's from $N = 4$ in every cell. The design consequence is a division by state. A stateless stage above the publisher period, decode being the natural candidate, is best served by a dispatch pool of four or more cores (three were not simulated); at two cores a cut and a pool are level. A stateful stage above the publisher period is served by cutting the chain, which is the case the rest of the paper analyses.

Findings.

- *Sharding by instrument does not help here:* a burst is one instrument's order flow and lands on one shard.
- *Dispatch with every cost charged* (hop in, hop out, resequencing) has median $T + 2h$ at any N , and almost no packet waits to be put back in order.
- *Against a serial cut on the same cores* the pool is level at two cores and has the lower p_{99} from four; three was not simulated.
- *Only a stage without state can be dispatched.* Decode can; the order book cannot.

4.5 Median cost of an N -stage tandem

This section looks at the typical message rather than the slow one. Its point is that splitting a task into more stages makes the typical latency worse by a fixed, predictable amount, exactly one mailbox hop per added stage, and nothing more. The reason is that the typical message arrives when the queue in front of it is empty, so it pays only the service time plus the hops. The simulation confirms this to the reported precision in almost every cell of the sweep. The few exceptions are at the longest service times, where the queue never empties and even the typical message waits; those sit outside the range the paper recommends, and splitting cures them anyway.

The median columns of Table 3 match the single-message formula of Proposition 1

$$p_{50}(N) = T + (N - 1)h \quad \text{with } h = 1.7 \mu s,$$

in every cell of the Poisson stream and in all but three cells of the real stream. Under Poisson the identity holds because the queue is empty; on the real stream it holds because the median message

arrives in a quiet stretch between clusters, where the queue is also empty. Each boundary therefore costs exactly one hop on the median, with no hidden component, as Theorem 1 says it must at every quantile.

The three exceptions are the longest service times: one stage at 64 and 128 μ s, and two stages at 128 μ s. There clusters merge into a persistent backlog and even the typical message waits. This is the regime boundary showing in the median rather than in the tail, and it lies outside the range the paper’s recommendations address. Splitting restores the identity once each stage is short enough; at 128 μ s the median is back at the no-queue latency by four stages.

Findings.

- *Each stage costs exactly one hop at the median:* $p_{50} = T + (N - 1)h$ in almost every cell, on real and Poisson arrivals.
- *The three exceptions* are the longest service times, where even the typical message waits; splitting restores the identity.

4.6 The design equation: hop cost against tail excess

This section turns the measurements above into a rule for choosing how many stages to split a task into. The idea is simple. Splitting a task across more threads has a fixed price: every added stage costs one mailbox hop, and that hop is paid on every message, fast or slow. It also has a benefit: when packets arrive in bursts, messages queue behind one another, and shorter stages drain those queues faster, so the slow tail shrinks. The question is when the benefit outweighs the price.

The section does four things. It writes the median and the tail latency as two short formulas, a hop-cost term plus a tail term. It tabulates the tail term from the corpus and shows that it depends only on how long the slowest stage is, and that it disappears entirely once every stage is short enough. It combines cost and benefit into one expression the operator can minimise, with a table of where the optimum falls. And it draws the practical conclusion: the goal is not many threads but a short slowest stage, and a task already shorter than the publisher period, about seven and a half microseconds here, should not be split at all.

From the corpus:

$$p_{99}^{\text{Hawkes}}(N) = T + (N - 1)h + \Delta(N), \quad (11)$$

$$p_{50}^{\text{Hawkes}}(N) = T + (N - 1)h, \quad (12)$$

where $\Delta(N) \equiv p_{99}^{\text{Hawkes}}(N) - p_{50}^{\text{Hawkes}}(N)$ is the “clustering tail excess”. Table 11 lists it, together with the ratio $\Delta(N)/\Delta(1)$ that Theorem 1 bounds by $1/N$.

T	$\Delta(N)$ (μs)				$\Delta(N)/\Delta(1)$			γ
	$N=1$	$N=2$	$N=4$	$N=8$	$N=2$	$N=4$	$N=8$	
2	0.00	0.00	0.00	0.00	—	—	—	—
4	0.00	0.00	0.00	0.00	—	—	—	—
8	0.61	0.00	0.00	0.00	0.000	0.000	0.000	—
16	18.59	0.61	0.00	0.00	0.033	0.000	0.000	—
32	114.14	18.59	0.61	0.00	0.163	0.005	0.000	—
64	721.82	114.14	18.59	0.61	0.158	0.026	0.001	3.32
128	4952.41	721.82	114.14	18.59	0.146	0.023	0.004	2.68
bound					0.500	0.250	0.125	≥ 1

Table 11. Corpus-median tail excess $\Delta(N) = p_{99}(N) - p_{50}(N)$ on the real arrivals at $h = 1.7 \mu\text{s}$, its ratio to the single-stage value, and the fitted exponent γ in $\Delta(N) \approx \Delta(1)N^{-\gamma}$; the last row is the Theorem 1 bound. What the table shows: each extra halving of the stage length moves the excess to the single-thread value at half the service time, so the values repeat along the diagonals; once each stage is below the publisher period the excess is zero. Every ratio is far below the bound.

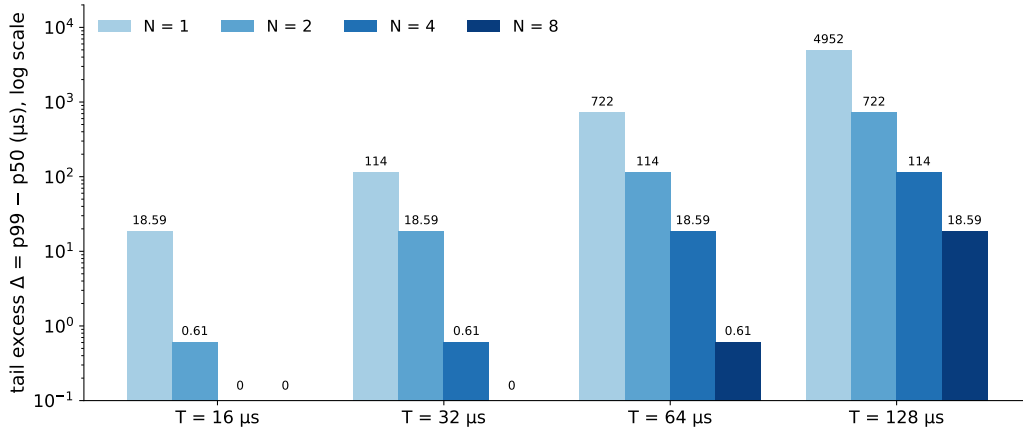


Figure 12. Table 11 as bars, log scale: the tail excess $\Delta = p_{99} - p_{50}$ for one, two, four and eight stages at each service time. Each extra halving of the stage length moves the excess down to the single-thread value at half the service time, because only the largest stage matters. Once each stage is below the publisher period the excess is zero (bars marked 0).

Two features of Table 11 follow from the theorems. First, Δ depends on (T, N) only through the per-stage service T/N : every diagonal of constant T/N carries one value, to the reported precision, without fitting. This is the reduction identity, Equation (4), on real arrivals: the tandem's tail is the single-server tail at $s_{\max}$, and the diagonals of the table are $\Delta_{\text{single}}(s)$ on a power-of-two grid of s .

Second, every ratio is at or below the bound $1/N$, and the pathwise check of Section 6 finds no message on any window above it. The margin is large, because of the threshold. Since $\Delta_{\text{single}}(s)$ is zero for s below the publisher period (Section 4.8), a split that pushes the per-stage service below that spacing does not merely divide the tail, it removes it. At $32 \mu\text{s}$, for example, four stages leave about half a percent of the single-thread excess, where the bound allows a quarter, and eight leave none. Where a power law $\Delta(N) \approx \Delta(1)N^{-\gamma}$ can be fitted, its exponent is about three, far steeper than the $\gamma = 1$ the bound alone would permit. The decay of the excess is therefore not a $1/N$

division on this corpus; it ends at zero once each stage is below the publisher period. The design consequence is that the objective is not a large N but an $s_{\max}$ below the publisher period. The utilisation slack described after Theorem 1 contributes as well, and Appendix B reproduces it on synthetic clusters.

The operator’s stage-count optimum is

$$N^* = \arg \min_N \left[(N-1)h + \kappa \Delta(N) \right]$$

for the operator’s tail weight κ (the number of microseconds of median the operator will spend to remove one microsecond of p_{99} excess). Over the simulated grid $N \in \{1, 2, 4, 8\}$ at $h = 1.7 \mu\text{s}$ the breakpoints are:

T (μs)	$N^* = 1$	$N^* = 2$	$N^* = 4$	$N^* = 8$
2	always	—	—	—
4	always	—	—	—
8	$\kappa < 2.8$	$\kappa \geq 2.8$	—	—
16	$\kappa < 0.095$	$0.095 \leq \kappa < 5.6$	$\kappa \geq 5.6$	—
32	$\kappa < 0.018$	$0.018 \leq \kappa < 0.19$	$0.19 \leq \kappa < 11$	$\kappa \geq 11$
64	$\kappa < 0.0028$	$0.0028 \leq \kappa < 0.036$	$0.036 \leq \kappa < 0.38$	$\kappa \geq 0.38$
128	$\kappa < 0.0004$	$0.0004 \leq \kappa < 0.0056$	$0.0056 \leq \kappa < 0.071$	$\kappa \geq 0.071$

A dash means that stage count is never optimal at any κ : at $T \leq 8 \mu\text{s}$ the tail excess is already zero by $N = 2$, so further splitting only adds hop cost. An operator who weights tail and median equally ($\kappa = 1$) keeps one stage up to $8 \mu\text{s}$ and uses more stages as T grows, which is also the ordering of the raw p_{99} minima in Table 3. Over the service times an HFT hot path occupies, an operator must weight the tail several times more than the median before splitting pays; above the threshold the required weight is small.

At real cut points the stages are unequal, and by Remark 2.3 the tail excess depends on the partition P of the servicing chain only through its largest stage, $\Delta(P) = \Delta_{\text{single}}(s_{\max}(P))$, i.e. the single-stage tail excess measured at service $s_{\max}$. The optimum over partitions is then

$$P^* = \arg \min_P \left[\text{hops}(P)h + \kappa \Delta_{\text{single}}(s_{\max}(P)) \right],$$

which Table 11 lets the operator evaluate directly, since its diagonals are $\Delta_{\text{single}}(s)$ on a power-of-two grid of s . Two rules follow. A cut is worth its hop only if it lowers $s_{\max}$ by enough that $\kappa [\Delta_{\text{single}}(s_{\max}) - \Delta_{\text{single}}(s'_{\max})] > h$; and among partitions with the same $s_{\max}$ the one with the fewest hops is preferred, so sub-tasks downstream of the bottleneck should be merged onto one thread until their sum reaches $s_{\max}$.

Design principle (only $s_{\max}$ matters). For a servicing chain cut at fixed boundaries, the tail-latency question is not “how many threads?” but “what is the smallest achievable largest stage $s_{\max}$?” By Theorem 1 the clustering tail excess of any partition is $\Delta_{\text{single}}(s_{\max})$, independent of the stage count; a cut that does not lower $s_{\max}$ compresses no tail and still costs one hop h . Choose the partition with the smallest reachable $s_{\max}$, then the fewest hops that realise it.

Minimum service time for splitting. Equations (11)–(12) imply a lower bound on the tasks the recommendation applies to. Splitting a task of total service T into N stages spends $(N - 1)h$ at every quantile to compress a tail excess $\Delta(1)$ that is zero at $4\mu\text{s}$ and below and grows fast above (Table 11). When T is below the publisher period there is nothing to remove, and the design equation prescribes one stage; a hop there adds cost only. The sweep confirms this: at 2 and $4\mu\text{s}$ a split removes nothing and raises every quantile by the full hop, and at $16\mu\text{s}$ it removes almost the entire excess for one hop, on every window.

Two separate necessary conditions apply. The first is a property of the *framework*: a split cannot pay unless the tail excess it removes exceeds the hop cost it adds, which on this framework’s shipping BQueue path ($h \approx 1.7\mu\text{s}$) sets a minimum at roughly $T \gtrsim 2h \approx 3.4\mu\text{s}$. The second is a property of the *feed*: there is no tail excess to remove at all until T exceeds the publisher period, about s_{pub} on this corpus (Section 4.9). The binding constraint is whichever is larger, and on CME MDP3 channel 318 with a BQueue mailbox it is the feed’s, by roughly a factor of two; this is why $T = 4\mu\text{s}$, although above $2h$, has $\Delta(1) = 0$ and should not be split. A framework with a much more expensive mailbox would find its own minimum binding instead. On CME the binding minimum is the publisher period, and the rest of the paper uses that term. Sub-microsecond tasks, for instance a single SBE integer decode or a small handler that fires from a strategy fast path, belong on `fast_send` rather than on a cross-thread mailbox, regardless of feed statistics.

Findings.

- *Only the largest stage matters.* The tail excess depends on (T, N) only through T/N , as Theorem 1 predicts.
- *A cut that takes every stage below the publisher period removes the tail rather than dividing it;* the decay is far steeper than the $1/N$ bound.
- *Weighting tail and median equally* gives one stage up to $8\mu\text{s}$ and more as T grows, up to eight at $64\mu\text{s}$ and above.
- *The binding minimum is the publisher period s_{pub} , not the framework’s $2h$.*

4.7 Per-window results

The results so far are for a typical half-hour window. This section asks whether they hold on every window, including the busiest ones, or only on average. The answer matters because a busy window has a far worse single-thread tail than the typical one, so an operator cannot size a pipeline from the average alone. The finding is that the split wins on essentially every window from about ten microseconds up, and on no window at eight microseconds or below, where there is no tail to remove and the split only adds hop cost, with a sharp crossover between. The recommendation is therefore not an artefact of averaging.

The corpus medians understate the tail on the busiest windows. Table 12 gives the median and the 95th percentile over windows of the single-thread p_{99}/T . At short service times the busiest windows are close to the median one; at long service times one window in twenty has a tail up to twice the median window’s. The tandem’s win is not confined to the median window: two stages and four stages both beat one on every window from $16\mu\text{s}$, and on no window at $8\mu\text{s}$ or below, where there is no tail to compress and the split pays only hop cost. The crossover is sharp and lies between 8 and $10\mu\text{s}$ (Section 4.8). The fitted branching ratio is about 0.8 in every window (Section 3.2); how the tail depends on it, and on the window’s packet rate, is examined in Section 4.9. The p_{95} is recorded per window and omitted from the tables only for brevity; it lies between p_{50} and p_{99} by

construction. The windows are nested in 276 sessions and are not independent, so uncertainty on the aggregates is clustered at the session level.

T (μs)	2	4	8	16	32	64	128
p_{99}/T , median window	1.0	1.0	1.1	2.2	4.6	12.8	41.5
p_{99}/T , 95th-percentile window	1.0	1.0	1.1	2.5	6.1	19.9	81.7

Table 12. Single-thread p_{99} relative to the service time, in the median window and in the window at the 95th percentile, over the corpus. What the table shows: below the publisher period no window has a tail; above it the busiest windows are worse than the median one, by up to a factor of two at long service times. What follows: a pipeline sized from the median window underestimates the tail it will meet in a busy one.

Findings.

- *The split wins on every window from 16 μs and on none at 8 μs or below.*
- *The crossover is sharp, between 8 and 10 μs .*
- *Busy windows are worse than the median one, by up to a factor of two at long service times (Table 12).*

4.8 Location of the tail onset

The coarse sweep brackets the onset of the clustering tail between $T = 8$ and $T = 16 \mu\text{s}$, too wide for a design rule. We reran the single- and two-stage arms on the same 3512 windows at $T \in \{5, 6, 7, 8, 9, 10, 12, 14, 16\} \mu\text{s}$:

T (μs)	p_{99} Hawkes	excess over T	windows with a tail	$p_{99}(2) < p_{99}(1)$
5	5.00	0.00	0.0%	0.0%
6	6.00	0.00	0.0%	0.0%
7	7.00	0.00	4.5%	0.0%
8	8.61	0.61	98.6%	0.0%
9	10.85	1.85	100.0%	67.0%
10	13.52	3.52	100.0%	99.7%
12	19.83	7.83	100.0%	100.0%
14	26.68	12.68	100.0%	100.0%
16	34.59	18.59	100.0%	100.0%

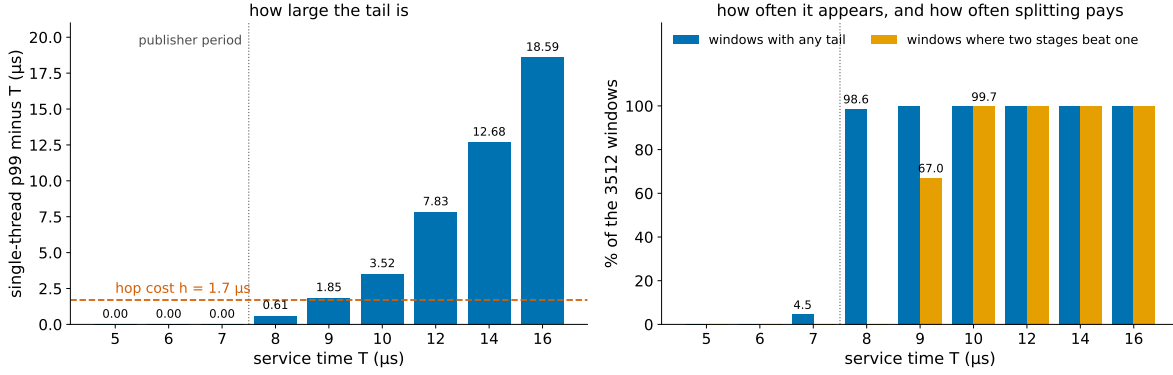


Figure 13. The table above as bars; the dotted line marks the publisher period s_{pub} . Left: how far the single-thread p_{99} sits above the service time. Below the spacing there is no tail at all; above it the tail grows fast and passes the hop cost of one split (dashed) between 8 and 9 μs . Right: the share of windows that have any tail (blue) jumps from almost none to almost all between 7 and 8 μs ; the share where two stages beat one (orange) follows about 1.5 μs later, once the tail is larger than the hop cost.

Two distinct thresholds are visible. The first is the *tail* threshold: between $T = 7$ and 8 μs the share of windows with any tail jumps from almost none to almost all. The second is the *split* threshold, about 1.5 μs later, between 8 and 10 μs : a tail must exist *and* exceed the hop cost before a split repays it. At 8 μs nearly every window has a tail, but it is well under the 1.7 μs hop, and splitting loses on every window. By 10 μs the tail is about twice the hop, and splitting wins almost everywhere.

The first threshold is set by the feed’s tight-gap structure rather than by its mean rate; the next section establishes this.

Findings.

- *The tail appears between 7 and 8 μs :* the share of windows with any tail jumps from almost none to almost all.
- *Splitting starts to pay about 1.5 μs later,* once the tail exceeds the hop cost, and pays on almost every window from 10 μs .

4.9 Dependence of the tail on packet rate and branching ratio

One would expect the slow tail, and with it the case for splitting, to depend on how busy the market is and on how strongly one transaction triggers the next. If it did, the rule for splitting would have to change with market conditions. This section tests that expectation and finds that, over the task lengths a trading hot path uses, it does not hold: a window with four to five times the packet rate has the same tail, relative to the task length, as a quiet one. The reason is that a busy market produces more bursts, not tighter bursts. The tightest spacing between two packets on this feed is about seven and a half microseconds, the spacing at which the exchange’s publisher sends packets (Section 5.4); the two packets on either side of such a gap are, almost always, two different transactions (Section 4.3), and neither the publisher period nor the spacing of transactions inside a burst changes with the market state. Since it is the tightest spacing that starts a queue, the point at which a tail appears is a constant of the feed. Market load only matters for very long tasks, where the ordinary effect of a busy server takes over. The practical consequence is that the rule for splitting needs no live estimate of market conditions; the section closes by confirming the result on

five-minute windows, and by noting that a rule based on the average gap between packets, rather than the tightest, would be wrong by two orders of magnitude.

The results above are conditioned on the service time T alone. A Hawkes tail depends on two further quantities that the corpus median averages over: the window's mean packet rate $\bar{\lambda}$, and its branching ratio $n = \alpha/\beta$.

We sort the 3512 windows into quintiles, first by $\bar{\lambda}$ and then by the fitted n , and report the corpus median of the normalised single-stage tail p_{99}/T within each quintile. Normalising by T removes the service time, so a value of 1.0 means no tail at all and the quintile-to-quintile ratio isolates the conditioning effect.

quintile	median	p_{99}/T , corpus median				
		$T=8$	$T=16$	$T=32$	$T=64$	$T=128$
<i>by packet rate $\bar{\lambda}$ (pkt/s)</i>						
Q1	202	1.08	2.29	4.74	12.25	31.76
Q3	451	1.07	2.12	4.48	12.68	42.81
Q5	942	1.08	2.11	4.51	13.92	55.67
Q5/Q1		1.00	0.92	0.95	1.14	1.75
<i>by branching ratio n</i>						
Q1	0.769	1.08	2.18	4.51	11.61	30.73
Q3	0.798	1.08	2.16	4.61	13.17	42.83
Q5	0.825	1.07	2.15	4.61	14.65	58.73
Q5/Q1		1.00	0.98	1.02	1.26	1.91

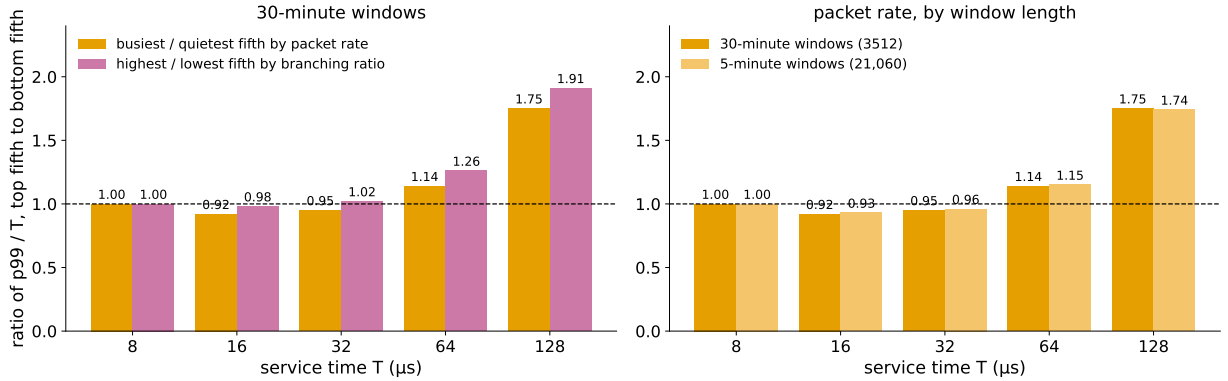


Figure 14. The quintile table above and the five-minute table below as bars: how much larger the normalised tail p_{99}/T is in the top fifth of windows than in the bottom fifth. Left: by packet rate (orange) and by branching ratio (pink). Up to $32 \mu\text{s}$ every bar is within 8% of 1 (dashed): a busier or more self-exciting market does not change the tail relative to the service time. From $64 \mu\text{s}$ both matter, as ordinary utilisation queueing sets in. Right: the packet-rate ratio on 30-minute and on five-minute windows; the two agree to 0.01.

The result divides between 32 and $64 \mu\text{s}$ (Figure 14). Up to $32 \mu\text{s}$ the normalised tail is flat in both variables: a more than fourfold range of packet rate moves it by less than a tenth, and at 16 and $32 \mu\text{s}$ the busiest fifth of windows even has the slightly smaller tail; the range of branching ratios moves it by at most a few percent. From $64 \mu\text{s}$ both variables matter in the expected direction, and

at $128\mu\text{s}$ the busiest and the most self-exciting fifths have roughly twice the tail of the quietest. This is the boundary between the clustering regime and the utilisation regime found elsewhere in the paper: from $64\mu\text{s}$ the Poisson stream has a tail too (Section 4.1), utilisation $\rho = \bar{\lambda}T$ is the governing quantity, and ρ is proportional to the packet rate. The rate matters where utilisation queueing applies and not below it.

Simulator check of the invariance. The Poisson stream, run on the same windows through the same simulator, does show the dependence on rate that utilisation queueing requires: at 16 and $32\mu\text{s}$ its normalised tail is clearly larger in the busiest fifth of windows than in the quietest, because utilisation crosses the 1% switch-on of Corollary 2 within the range. The real stream on the same windows shows none. The simulator therefore resolves the rate; the clustered tail does not depend on it at these service times. The result is the same without binning into fifths, by rank correlation over all windows, and between the top and bottom 2% of windows by rate, a range of more than twentyfold. The mechanism is that at low utilisation clusters do not overlap. Raising the rate raises the number of clusters per window, each with the same internal spacing and, at a fixed branching ratio, the same size distribution, so where the p_{99} packet sits within its cluster, and hence p_{99}/T , is unchanged. The rate enters once clusters begin to overlap, and the branching ratio once T is long enough for many members of one cluster to fall within one service time; on this corpus both happen from $64\mu\text{s}$.

The mechanism is that at $\rho \ll 1$ clusters do not overlap. Raising $\bar{\lambda}$ raises the number of clusters per window, each with the same internal spacing and, at fixed n , the same size distribution, so the position of the p_{99} packet within its cluster, and hence p_{99}/T , is unchanged. Intensity enters once clusters begin to overlap, and n enters once T is long enough for many members of one cluster to fall within a single service time; on this corpus both occur at $T \geq 64\mu\text{s}$.

Packet rate and the tight gaps below $64\mu\text{s}$. The null result has a mechanical explanation in the arrival stream. Taking the same $\bar{\lambda}$ quintiles and reporting the corpus median of the per-window interarrival-gap quantiles:

quintile	$\bar{\lambda}$ (pkt/s)	p_1 gap	p_5 gap	p_{25} gap	p_{50} gap	mean gap
Q1	202	7.45	8.85	21.30	87.10	4944
Q2	320	7.52	8.75	21.87	84.52	3128
Q3	451	7.53	8.70	22.70	82.16	2215
Q4	620	7.51	8.66	23.36	81.98	1612
Q5	942	7.43	8.58	22.55	76.52	1062
Q5/Q1	4.66×	0.997×	0.969×	1.059×	0.878×	0.215×

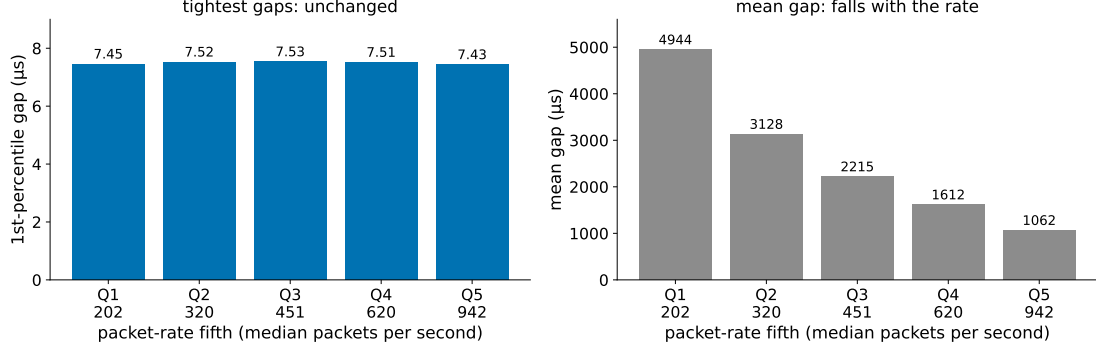


Figure 15. The gap-quantile table above as bars, by packet-rate fifth. Left: the tightest gaps (1st percentile) stay at $7.43\text{--}7.53\ \mu\text{s}$ while the rate more than quadruples. Right: the mean gap falls in proportion to the rate. A busy market has more bursts, not tighter ones, and the tightest gap is what starts a queue.

All gaps in microseconds. The mean gap falls in proportion to the rate, which is definitional. Every other column is nearly constant, and the tightest gap is the most constant of all: it stays at the publisher period while the rate more than quadruples (Figure 15).

Raising the intensity of a self-exciting process raises the rate at which clusters are generated; it does not change the spacing of events inside a cluster. The tight end of that spacing is bounded by the publisher period s_{pub} on this corpus, a property of the exchange’s publisher rather than of the market state (Section 5.4); the events on either side of a gap at the publisher period are distinct transactions arriving within that separation of each other (Section 4.3, Table 6), and how closely successive transactions within a burst follow one another is likewise invariant to the window’s rate. A busy window contains more bursts, not tighter bursts.

The gap statistics on the publisher’s clock are censored at the publisher period: two transactions closer together than s_{pub} reach the receiver in one packet (Section 5.4). A faster self-excitation would therefore show in the packets rather than the gaps. It does not scale with the branching ratio: from the lowest to the highest branching-ratio fifth, the share of multi-message packets, the mean message count per packet and the share of packets carrying more than one transaction do not rise, and across packet-rate fifths they barely move.

The queueing consequence: a deterministic server of service time T begins to build a queue when it is handed two arrivals separated by less than T , so the service time at which a tail appears is set by the *tight* end of the gap distribution, not by its mean. The tight end is invariant to $\bar{\lambda}$; therefore the threshold is invariant to $\bar{\lambda}$. The two numbers agree: the tightest gap, s_{pub} , the same in every rate fifth, matches the tail onset of Section 4.8, between 7 and $8\ \mu\text{s}$.

Robustness to window length. A 30-minute window averages the rate over a long interval, and the comparison above may have smoothed away the variation that matters. We therefore reran the whole pipeline, from packet extraction and Hawkes fits to the Poisson null and the full (T, N) grid, on five-minute windows, six times as many. Shorter windows do resolve more variation in rate and branching ratio, as expected. The conditioning result is unchanged to two decimal places:

	$\bar{\lambda}$ Q5/Q1	$T=8$	$T=16$	$T=32$	$T=64$	$T=128$
30-min windows ($n = 3512$)	$4.66\times$	1.00	0.92	0.95	1.14	1.75
5-min windows ($n = 21,060$)	$4.98\times$	1.00	0.93	0.96	1.15	1.74

Entries are the ratio of the normalised single-thread tail p_{99}/T in the top fifth of windows to that in the bottom fifth. Resolving the rate six times more finely moves no entry by more than 0.01. The result is a property of the arrival process, not of the window length.

Consequence for the design rule. On NQ the rule can therefore be stated as a constant of the feed rather than as a function of market state.

Design principle (on NQ, the threshold is a constant of the feed). Split a stage whose service time exceeds the publisher period; do not split one below it. Compare the publisher period with the upper quantiles of the stage’s per-packet service under load, not with its median (Section 13.1). The publisher period is the tight end of the feed’s interarrival-gap distribution (the p_1 gap, about s_{pub}), not its mean, which is two orders of magnitude larger. It is measured once per feed: on this corpus it is the same in every packet-rate fifth and every branching-ratio fifth, so the rule requires no runtime estimate of market intensity or clustering and does not change between a quiet and a volatile session. Both the publisher period and the spacing of transactions inside a burst were measured on one instrument, NQ front-month on CME channel 318; recheck them on any other instrument or feed rather than assume them.

Appendix D constructs a feed on which both dependences appear at $T = 8\mu\text{s}$, and identifies the packed-gap fraction as the quantity that separates it from CME.

The publisher period is not the mean interarrival gap. On this corpus the mean gap is about 2 ms in the median window, some three hundred times the publisher spacing. A rule stated on the mean gap would recommend never splitting an HFT stage, and would be wrong by two orders of magnitude.

Findings.

- *Up to $32\mu\text{s}$ market conditions do not change the tail relative to T :* a more than fourfold range of packet rate moves it by less than a tenth, the range of branching ratios by a few percent.
- *From $64\mu\text{s}$ both matter,* as utilisation queueing sets in: at $128\mu\text{s}$ the busiest windows have about twice the tail of the quietest.
- *A busy market has more bursts, not tighter ones:* the tightest gap stays at the publisher period while the rate more than quadruples.
- *The same holds on five-minute windows,* and packet size does not rise with the branching ratio.
- *A rule based on the mean gap* instead of the tightest would be wrong by two orders of magnitude.

4.10 Sensitivity to the hop cost h

The rule depends on the cost of a mailbox hop, and this section asks what changes if that cost is different. A cheaper hop makes splitting worthwhile over a wider range of task lengths, but it does not move the point below which there is no tail to remove, since that point is set by the feed and not by the framework. Any other framework can therefore measure its own hop cost, put it into the same equation, and read off its own answer; a framework with a very expensive hop should not split at all. The section also explains why the hop cost used throughout is a cautious overestimate. It was

measured with the receiving thread asleep, so it includes the cost of waking it. Inside a burst the receiver never sleeps, because messages arrive faster than it can process them, and the true hop cost there is a few hundred nanoseconds rather than nearly two microseconds. The benefit of splitting during a burst is therefore, if anything, understated.

The design equation is monotone in h : at fixed T and fixed corpus $\Delta(N)$, every κ breakpoint in the table above scales linearly with h , so a smaller h can only raise N^* . At a hypothetical $0.5\mu\text{s}$ busy-poll mailbox the best stage count changes at one service time only, $8\mu\text{s}$, where it rises from one to two; below the publisher period no reduction in h can make a split worth taking when there is no tail to remove. A cheaper hop widens the band of service times over which splitting pays; it does not move the threshold, which is set by the feed. For any other framework the procedure is the same: measure the one-way asynchronous hop cost h on its own dispatch path, put it into Equation (11) with the corpus Δ of Table 11, and read off N^* ; a framework whose h exceeds the tail excess a split could remove gets $N^* = 1$, and the recommendation reverses.

Findings.

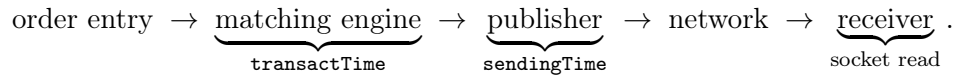
- *A cheaper hop widens the range where splitting pays but does not move the threshold:* at a third of the hop cost the best stage count changes only at $8\mu\text{s}$.
- *The measured hop cost is an upper bound:* inside a burst the receiver never sleeps and a hop costs a few hundred nanoseconds.

5 The exchange side: matching engine, publisher and bursts

The receiver’s arrivals are the output of the exchange. This section looks one step upstream, at the two CME systems that produce them, through the only window the public feed opens on them: the timestamps it carries. It treats the exchange as a queueing system observed from outside and makes no claim about CME’s internal design beyond what the MDP3 specification states. All figures are for NQ front-month on channel 318.

5.1 Three timestamps

Every message on the feed carries two exchange timestamps, and the receiver adds a third. **transactTime** is the start of event processing at the matching engine: when the engine began to match, add or cancel the order behind the message. **sendingTime** is written into each packet by CME’s market-data publisher when it sends the packet. Both come from CME’s synchronised clocks. The receiver’s own timestamp, taken at the socket read, is the third. The path is



The simulator of Part II uses **sendingTime** as each packet’s arrival time, since the receiver sees the publisher period plus a network delay that is close to constant. The feed does not carry the time an order entered the engine, and nothing in it shows the engine’s or the publisher’s internal queues directly. The MDP3 end-of-event flag marks the last message of each matching event.

5.2 Transactions and events

Section 4.3 grouped packets into blocks whose **transactTime** ranges overlap. Regrouping by the exact **transactTime** value, and checking against the end-of-event flag, on 40 sessions, confirms the

grouping. A `transactTime` group almost never contains or straddles more than one end-of-event flag, so one `transactTime` is one matching event. The flag itself cannot serve as the grouping key on a single-instrument tape, since most NQ events end on a message for another security on the channel. Under exact grouping the share of transactions that fit in one packet is the same to within a tenth of a percent, and there are a few percent more transactions than blocks, the difference being transactions that share a packet. The results of Section 4.3 are unchanged at the precision reported.

Findings.

- *One `transactTime` is one matching event:* groups almost never conflict with the end-of-event flag.
- *Regrouping changes nothing reported:* the share of transactions that fit in one packet is unchanged.

5.3 Gaps between transactions on the engine and publisher clocks

gap between consecutive transactions	$< 7.5 \mu s$	$< 16 \mu s$	$< 32 \mu s$	p_1 gap
engine clock (<code>transactTime</code>)	17.1%	25.4%	35.4%	$0.18 \mu s$
range across sessions	14.4–22.9	22.3–31.8	31.5–40.7	0.15–0.21
publisher clock (<code>sendingTime</code>)	1.04%	17.8%	32.1%	$7.46 \mu s$
range across sessions	0.44–1.52	15.5–22.2	28.6–36.9	7.23–7.88

Table 13. The same NQ transactions timed by the matching engine and by the publisher; median session and full range over 40 sessions. What the table shows: one consecutive transaction in six leaves the engine less than one publisher period after the previous one, and the tightest one percent are $0.18 \mu s$ apart; almost none leave the publisher that close, and the publisher’s tightest one percent are $7.46 \mu s$ apart. The difference holds in every session. What follows: the shortest gap the receiver sees is the publisher period, not a property of the market’s event process; events the engine produces faster than that are either held back or packed into one packet.

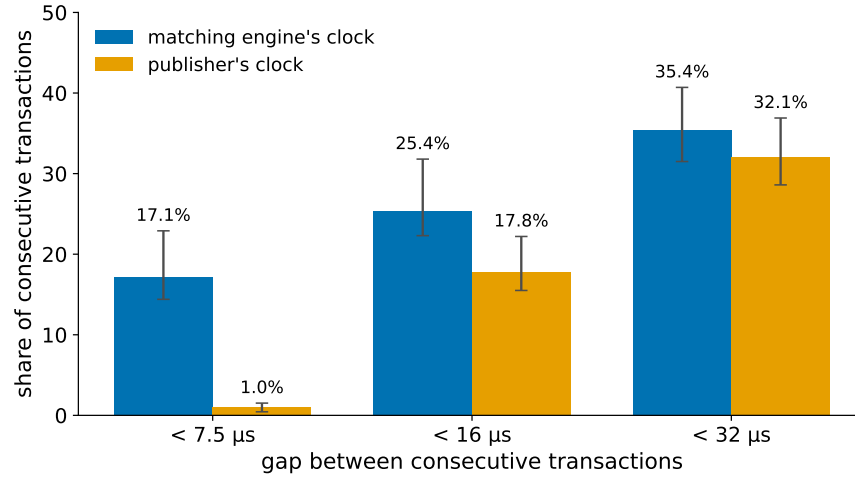


Figure 16. Table 13 as bars: share of consecutive NQ transactions closer than 7.5, 16 and 32 μ s, on the engine's clock (blue) and the publisher's (orange); whiskers span the 40 sessions. Below the publisher period the engine has one gap in six and the publisher almost none: the publisher does not send faster than its spacing. By 32 μ s the two clocks nearly agree.

Finding. On NQ, one consecutive transaction in six (17.1%) is processed by the matching engine less than one publisher period after the previous one, against 1.04% of consecutive packets sent by the publisher. The publisher period of the receiver's arrival stream is set by the exchange's publisher. The engine's own gaps reach 0.18 μ s at the first percentile, which means the engine went directly from one event to the next with the next order already waiting: at those moments it was working through a backlog of orders that reached it within a fraction of a microsecond of each other.

The receiver does not see these transactions together. Table 14 follows every gap between consecutive NQ transactions from the engine's clock to the publisher's (the two clocks are defined in Section 4.3): for the same two transactions, the table gives the gap at the engine and the gap between the packets that carry them.

gap on the engine's clock	share of gaps	gap on the publisher's clock (row %)					
		same packet	< 7.5	7.5–10	10–16	16–100	> 100 μs
< 1 μs	5.3%	15.7	3.7	43.0	16.1	19.9	1.7
1–7.5 μs	11.8%	10.0	3.8	32.1	20.2	29.9	4.1
7.5–16 μs	8.5%	8.8	2.1	21.5	13.6	48.7	5.3
16–100 μs	31.4%	4.9	0.5	4.9	4.0	71.1	14.7
> 100 μs	43.0%	0.8	0.0	0.1	0.2	15.1	83.7
all gaps	100%	4.6	1.0	9.5	5.7	37.5	41.6
of which engine gap < 7.5 μs		43.3	66.2	64.0	56.4	12.2	1.4

Table 14. The gap between consecutive NQ transactions, measured on two clocks for the same two transactions, over 20 sessions. Rows: the gap on the matching engine's clock. Columns: the gap between the packets that carry the two transactions, on the publisher's clock, as a percentage of the row; "same packet" means both were sent in one packet. The last row gives, for each publisher gap, the share of cases in which the engine processed the two transactions less than s_{pub} apart. What the table shows: when the engine processes a transaction less than 1 μs after the previous one, the two are sent in separate packets 84% of the time, most often 7.5 to 10 μs apart. Read the other way, most of the short gaps the receiver sees separate transactions that the engine processed almost together. What follows: transactions do not reach the receiver together. The publisher spaces them out to its own interval of about s_{pub} and packs only a few into one packet, so the short gaps that load the receiver are, for the most part, near-simultaneous engine events delivered back to back at the publisher period.

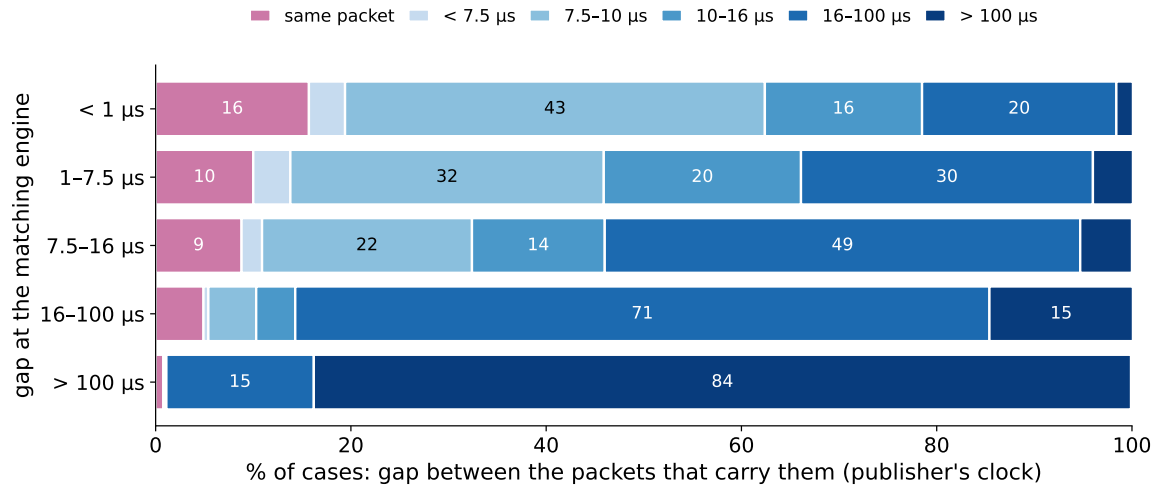


Figure 17. Table 14 as bars. Each bar is the set of consecutive transactions with the engine gap on the left, split by the gap between the packets that carry them; the numbers are percent of the bar. The top bar reads: when the engine processes a transaction less than 1 μs after the previous one, the two share a packet 16% of the time (pink) and otherwise arrive in separate packets, most often 7.5–10 μs apart (43%). Near-simultaneous at the engine becomes back to back at the publisher period at the receiver.

Finding. Transactions the engine processes a fraction of a microsecond apart do not reach the receiver together: 84% of them arrive in separate packets, most of them 7.5 to 16 μ s apart. What the receiver sees is a train of packets at the publisher period. Its queue grows when its service time exceeds that spacing, which is why the threshold of Section 4.8 sits at about s_{pub} . The open market question is why orders reach the engine within a microsecond of each other; what reaches the receiver is set by the publisher.

5.4 The publisher as a queue

Definition (publisher period, s_{pub}). The minimum period between two consecutive packets from the exchange’s market-data publisher: the publisher’s service time per packet, and the inverse of its maximum send rate. On NQ it is about 7.5 μ s, measured as the 1st-percentile gap between packets (7.43–7.53 μ s, Section 4.9). Two packets never arrive closer together than s_{pub} ; during a burst they arrive exactly s_{pub} apart. It is the publisher’s period, not the matching engine’s: the engine processes transactions much closer together (Table 13). The paper refers to it by name throughout.

If the publisher sent events on a fixed schedule, the time from engine processing to packet send, $\text{sendingTime} - \text{transactTime}$, would not depend on how busy the engine is. It does. Its median is about 100 μ s and its far tail several milliseconds, and it rises steeply with the number of transactions the engine processed in the preceding 50 μ s (Table 15).

other NQ transactions in the previous 50 μ s (engine clock)	share of transactions	publisher delay		published sharing a packet with another transaction
		p_{50}	p_{99}	
0	57.9%	86 μ s	2.9 ms	4.2%
1	24.1%	113 μ s	3.5 ms	10.0%
2–3	13.8%	205 μ s	4.4 ms	15.2%
4–7	3.8%	406 μ s	6.8 ms	20.8%
8–15	0.4%	912 μ s	15.0 ms	26.6%

Table 15. Delay from engine processing to packet send, $\text{sendingTime} - \text{transactTime}$, by how busy the engine was in the 50 μ s before each transaction, over 20 sessions. What the table shows: the median delay grows about tenfold from a transaction that follows no other to one that follows eight to fifteen, the p_{99} grows with it, and the share of transactions sent in a packet with another rises several times over. What follows: a fixed publishing schedule would keep the delay flat. Delay and packing growing together is the signature of a queue: the publisher sends about one packet every 7.5 μ s, events the engine produces faster than that wait, and the backlog is packed into fuller packets. The burst measure counts NQ front-month transactions only, while the publisher serves the whole channel, so the table understates the load on the publisher.

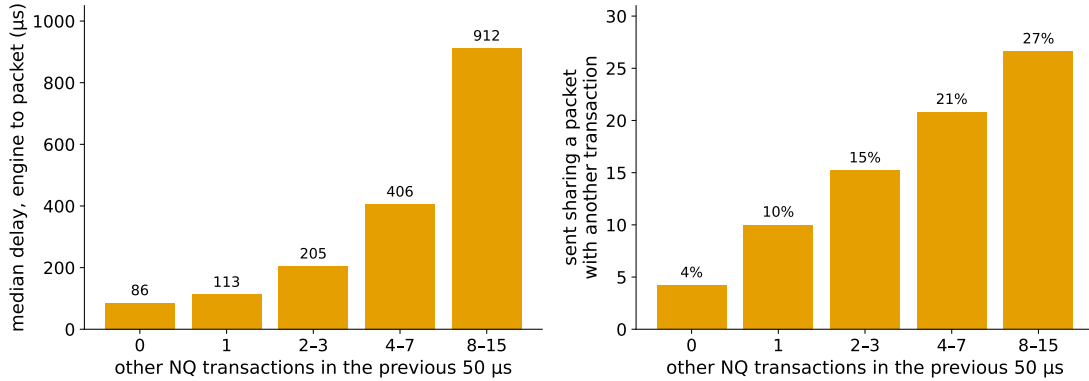


Figure 18. Table 15 as bars. Left: median delay from engine to packet grows tenfold as the engine gets busier. Right: the share of transactions sent in a packet with another transaction grows from 4% to 27%. Both rising together is what a queue at the publisher looks like.

A fixed publishing policy would keep the delay flat. Here it grows roughly tenfold with engine burst intensity, and coalescing grows with it. That is a queue: the publisher has a service time of one publisher period, s_{pub} . When the engine produces events faster than that, they wait, and the publisher packs the backlog into fuller packets.

This accounts for the publisher period from first principles. The output of a deterministic server cannot contain two departures closer together than its service time, and while the server works through a backlog its departures are spaced by exactly that time. The edge of the receiver’s gap distribution at s_{pub} (Figure 6) and the mass of gaps just above it are the publisher draining its backlog.

Findings.

- *The delay from engine to packet grows with engine load*, about tenfold from a quiet moment to a burst.
- *Packing grows with it*: the share of transactions sent in a shared packet rises several times over.
- *The publisher is a queue* with service time s_{pub} per packet; the shortest gap the receiver sees is its period.

5.5 Packet size under publisher backlog

If the publisher’s service time is fixed per packet, a backlog can be drained in two ways: by keeping the spacing and sending many packets, or by packing more into each packet. The end-of-event tapes separate the two. On 20 sessions (2.43×10^8 packets) we take, for each packet, the NQ front-month messages it carries (its span), the number of distinct transactions in it, the gap to the previous NQ packet on the publisher’s clock, and the publisher delay of its first transaction, `sendingTime` – `transactTime`, as the measure of how far behind the publisher is.²

²Channel 318 carries other instruments besides NQ front-month, and the publisher packs their messages into the same packets. The span here counts NQ front-month messages only, so it is not the packet’s size in bytes, and the publisher’s backlog includes work for instruments we do not decode. Doing this properly means measuring the whole channel and modelling how activity in one instrument excites activity in the others (cross-excitation, a multivariate Hawkes process), rather than treating NQ as a stream on its own. On NQ the other instruments are thin. On ZN

publisher delay of the packet's first transaction	share of packets	mean span (NQ messages)	packets with ≥ 2 transactions
$< 100 \mu s$	52.3%	1.000	0.02%
100–200 μs	21.3%	1.016	1.2%
200–500 μs	14.9%	1.237	8.9%
0.5–1 ms	5.9%	1.278	11.3%
1–5 ms	5.2%	1.440	18.2%
> 5 ms	0.4%	1.603	20.6%

Table 16. Packet size against the publisher's backlog; 20 sessions, 2.43×10^8 packets. What the table shows: when the publisher is less than 100 μs behind, packets carry exactly one NQ message; when it is more than 5 ms behind, they carry 1.6 on average and one in five holds two or more transactions. What follows: packets get bigger when the publisher falls behind. Table 17 shows which packets.

gap to previous NQ packet	share of packets	mean span (NQ messages)	packets with ≥ 2 transactions	adjacent on the channel	median other packets between
$< 7.5 \mu s$	1.0%	1.006	0.08%	100%	0
7.5–10 μs (spacing)	9.9%	1.007	0.40%	100%	0
10–16 μs	6.8%	1.096	2.8%	95.7%	0
16–32 μs	14.2%	1.141	6.7%	36.2%	1
32–100 μs	25.2%	1.073	3.4%	20.6%	2
0.1–1 ms	26.9%	1.106	4.5%	12.9%	4

Table 17. Packet size against the gap to the previous NQ packet on the publisher's clock; same packets as Table 16. The last two columns count the channel's other packets sent between two consecutive NQ packets, from the packet index of the tape, which numbers every channel packet that carries order or trade records for any instrument; packets carrying only other message types are not counted. What the table shows: packets sent at the publisher period carry one NQ message (1.007 on average) and in every case follow the previous NQ packet with no other order or trade packet of the channel between them. Bigger packets appear after gaps of 10 to 32 μs , which from 16 μs usually contain at least one packet for another instrument. What follows: the publisher period is the period of the channel's order and trade packets, not of NQ alone, and the publisher does not answer a backlog with bigger packets sent at the publisher period.

publisher delay (packets at the spacing only, gap 7.5–10 μs)	mean span (NQ messages)	packets with ≥ 2 transactions
$< 100 \mu s$	1.000	0.04%
100–500 μs	1.006	0.50%
0.5–5 ms	1.026	1.0%
> 5 ms	1.114	2.0%

Table 18. Packets sent at the publisher period, by the publisher's backlog. What the table shows: even more than 5 ms behind, packets sent back-to-back at the publisher period carry 1.1 NQ messages on average and 98% of them hold a single transaction. What follows: pacing holds under load. The publisher keeps its spacing when it is far behind and does not switch to fuller packets at the same rate.

they are not: the Treasury futures along the yield curve trade against each other, a move in one is likely to trigger quotes and trades in the others within microseconds, and where they share ZN's channel the publisher would carry those reactions in the same packets. Cross-excitation along the curve is therefore a candidate cause of the large ZN packets of Section 7.6. It is untested here and listed among the open questions of Section 14.1.

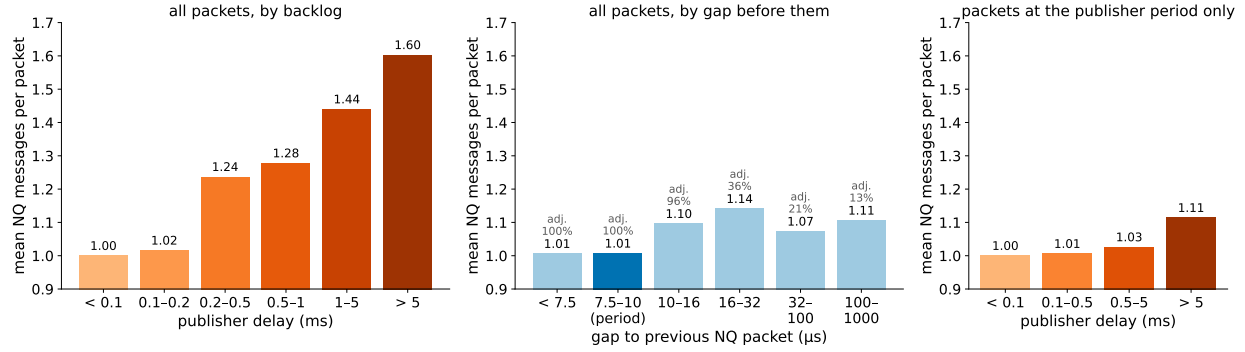


Figure 19. Tables 16–18 as bars: mean NQ messages per packet. Left: packets get bigger as the publisher falls further behind. Middle: packets sent at the publisher period, 7.5–10 μ s after the previous NQ packet (dark blue), are single-message packets; the grey “adj.” figures are the share with no other order or trade packet of the channel between the two NQ packets, 100% at the spacing. Right: packets at the spacing stay single-message even when the publisher is more than 5 ms behind. The publisher drains a backlog with more packets, not bigger ones.

Three things follow. First, pacing holds under load: even when the publisher is milliseconds behind, the packets it sends back-to-back are single-message packets at about publisher period (Table 18), and that period holds across the channel’s order and trade packets (Table 17). Second, coalescing happens elsewhere. Bigger NQ packets come after gaps of 10 to 32 μ s, so when packets are bigger they also leave more slowly, which fits a service time per packet that grows with its size, the span-dependent service of Section 7.6. Third, a receiver sees a burst at the engine as more packets, not bigger ones: a long run of small packets at the publisher period, with an occasional larger packet after a longer gap. This is the arrival pattern that makes the receiver’s service time, compared with s_{pub} , the quantity that sets its tail (Section 5.6).

Findings.

- *Packets get bigger as the publisher falls behind*, from one NQ message each to about one and a half when it is milliseconds behind.
- *Packets sent at the publisher period stay single-message even far behind*: the publisher drains a backlog with more packets, not bigger ones.
- *The publisher period holds across the channel’s order and trade packets*: packets at the period always follow the previous NQ packet with no other such packet between.
- *Only NQ front-month messages are counted*; the full channel’s packet size is an open question.

5.6 The receiver as the second stage of a tandem starting at the exchange

The publisher and the receiver are two servers in series. When a burst at the engine builds a backlog at the publisher, the publisher emits packets back-to-back at its own period s_{pub} . A receiver whose service time is shorter than that keeps up and queues nothing; a receiver whose service time is longer falls behind by the difference on every packet of the drain, and its backlog grows for as long as the drain lasts. This is the threshold of Section 4.8, now explained: the service time at which a receiver’s tail appears equals the service time of the exchange stage upstream of it. It is the tandem of Theorem 1 read in the other direction: a downstream stage slower than the upstream one is the bottleneck, and it is the receiver’s stage, not the publisher’s, that sets the tail the receiver pays. The

two regimes of Section 4.2 follow. At service times just above the publisher period, a short drain is enough to build a queue, and what matters is how often the publisher drains, which the gap shuffle preserves. At long service times the queue needs a sustained drain, which is a run of engine bursts, and that is what the shuffle destroys.

Findings.

- *The receiver's tail threshold is the publisher's service time:* a receiver faster than s_{pub} per packet keeps up with any train the publisher sends; a slower one falls behind on every packet of the train.
- *This explains the two regimes of the nulls:* short service times queue on each short drain, long ones on sustained runs of engine bursts.

5.7 Bursts at the matching engine

Section 5.3 shows that the engine often processes events a fraction of a microsecond apart, which means orders were waiting for it; the publisher then delivers them to the receiver one publisher period apart (Table 14). Why orders reach the engine within a fraction of a microsecond of each other is the market question of Section 13.3. The $0.18\mu\text{s}$ spacing fits many participants sending orders in response to the same public signal, the *races* that Aquilina et al. [33] measured on the London Stock Exchange with order-entry times the CME public feed does not carry. It is consistent with that mechanism and does not prove it: a reaction to the previous transaction that is faster than the engine's backlog would look the same from here.

Table 19 tests what transactions separated by a tight gap are. It takes every gap between consecutive transactions on the engine clock in 40 sessions (512 million gaps) and compares the two transactions on either side of a tight gap (below $16\mu\text{s}$) with those on either side of a loose gap (0.1 to 1 ms). A transaction is labelled by the first matching kind among trade, delete, new and change. Transactions across a tight gap are ordinary book traffic: their mix is close to that across a loose gap, and only 2.0% of tight gaps follow a trade. After a trade, the two differ: the next transaction is another trade in 11.7% of tight gaps against 2.2% of loose ones, and a delete in 37.4% against 23.5%. A trade followed within microseconds by a further trade or a cancellation is the pattern a race would leave, and it is more frequent across tight gaps. The table cannot measure how much of the engine's activity is racing: the losers of a race to take liquidity are rejected and never reach the public feed, and a race to cancel appears only as deletes, which dominate across tight and loose gaps alike. What the feed shows is that transactions separated by tight gaps are mostly quote placements and cancellations, and that those following a trade carry the race signature.

Consecutive transactions, engine clock	gap < 16 μ s	gap 0.1–1 ms
share of all gaps	25.8%	27.2%
first is a trade	2.0%	3.5%
first is a delete	41.2%	39.8%
first is a new order	46.0%	39.5%
first is a change	10.8%	17.2%
after a trade: next is a trade	11.7%	2.2%
after a trade: next is a delete	37.4%	23.5%
after a trade: next is a new order	45.5%	68.2%
same side	52.5%	53.9%
same price	20.5%	17.3%

Table 19. The two consecutive transactions on either side of a gap, by gap length on the engine clock. Forty sessions, 512 million gaps; the middle bin (16–100 μ s, 31.5% of gaps) is omitted. “First” is the earlier transaction, “next” the later. Transactions across a tight gap are mostly quote placements and cancellations, in about the same mix as across a loose gap; only 2% of tight gaps follow a trade. After a trade, a tight follow-up is five times as likely to be another trade and more likely to be a cancellation, the signature of a race. Rejected race orders do not appear on the public feed, so the table does not measure the share of racing.

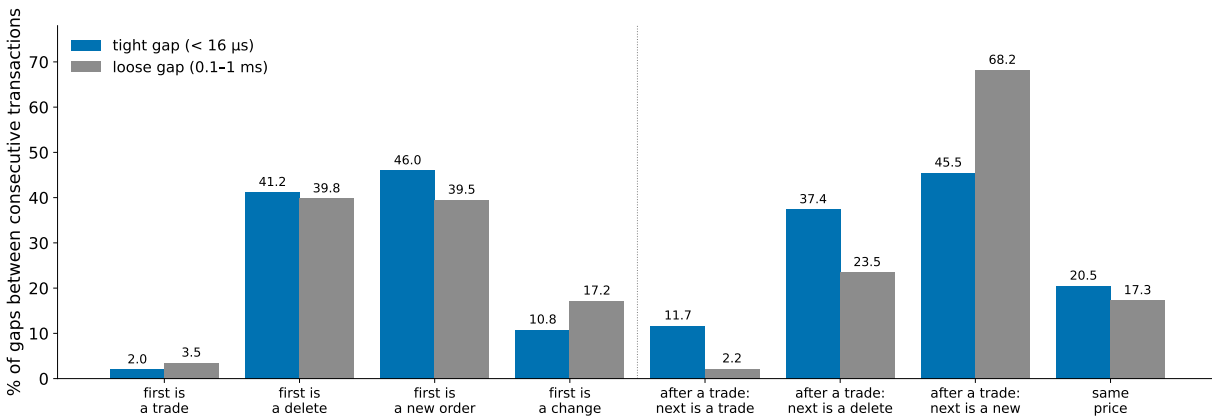


Figure 20. Table 19 as bars: the kind of transaction on either side of a tight gap (blue, under 16 μ s) and of a loose gap (grey, 0.1–1 ms). Left of the dotted line the two mixes are alike: tight gaps are ordinary quote traffic. Right of it, after a trade the next transaction is far more often another trade or a delete across a tight gap, the pattern a race would leave.

The fitted self-excitation is a different phenomenon at a different scale. The Hawkes fit on the engine clock gives the same branching ratio as on the publisher’s clock, about 0.8, and a kernel that decays over about 160 μ s, so it places only a few percent of its triggering within one publisher period and nearly all of it within a millisecond. That is the scale of the mode near 100 μ s in the gap distribution (Figure 6) and of the runs that carry the tail from 64 μ s. The self-excitation the fit measures is one transaction making another more likely over the following hundred microseconds or so; it accounts for the runs and does not account for transactions a fraction of a microsecond apart, which reach the engine together rather than one triggering the next.

Findings.

- *Transactions across a short engine gap are ordinary quote traffic:* the mix of new orders and deletes matches that across long gaps, and few short gaps follow a trade.
- *After a trade, the race pattern shows:* the next transaction is far more often another trade or a delete. The share of racing is not measurable on the public feed.
- *The fitted self-excitation acts over about $160\mu\text{s}$* and explains the runs, not transactions a fraction of a microsecond apart.

5.8 Conjecture: the publisher’s capacity as an information-side speed limit

Conjecture. Because the publisher sends at most one packet per s_{pub} and queues whatever the engine produces faster than that, it bounds the rate at which market information reaches participants, and during bursts it delivers that information 0.4 to 15 ms late (Table 15). Any feedback loop in which participants react to published data therefore cannot run faster than the publisher, and a cascade driven by public information unfolds more slowly than the engine could process it. In this sense the publisher acts as a speed limit on the information side of the market, whether or not it was designed as one, and protects downstream receivers from the engine’s peak rate.

Three qualifications bound the conjecture. First, delaying feedback does not reduce the branching ratio of a self-exciting process: each event still triggers the same number of follow-on events on average, later, so the publisher cannot make a critical cascade subcritical; it bounds the cascade’s speed, not its size. Second, it does not bound simultaneous reactions to a common signal that reach the engine through order entry, which are what the engine-clock gaps of Table 13 suggest; those do not pass through the publisher before they are submitted. Third, whether the publisher’s capacity is a design choice or an engineering limit cannot be read from the feed; the queueing signature of Table 15 points to a limit. The conjecture is related to proposals that slow markets deliberately to blunt speed races, such as frequent batch auctions [29]; the difference is that here the delay is on the information leaving the exchange, not on orders entering it.

5.9 Summary of the findings on the exchange side

The section follows each transaction from the matching engine, through the publisher, to the receiver. What it finds, in order:

Findings.

- *The matching engine processes transactions far closer together than the receiver sees them.* On the engine’s clock one consecutive transaction in six is under s_{pub} apart, the tightest a fraction of a microsecond; on the publisher’s clock 1% are (Table 13).
- *The publisher is a queue with service time s_{pub} per packet.* Its delay grows about tenfold with engine load (Table 15).
- *Near-simultaneous transactions reach the receiver as a train of packets at that spacing, not together.* Of transactions processed within $1\ \mu\text{s}$ of the previous one, 84% are sent in a separate packet, most $7.5\text{--}16\ \mu\text{s}$ later (Table 14).
- *Under backlog the publisher sends more packets, not bigger ones.* Packets grow when it is far behind, but those sent at the publisher period stay single-message (Tables 16–18).
- *The receiver is the second queue in a tandem that starts at the exchange.* Its tail threshold is the publisher period, which is why the threshold sits at s_{pub} and does not move with market load.
- *Transactions across a tight engine gap are ordinary quote traffic; after a trade, the race pattern shows* (Table 19). Why orders reach the engine within a microsecond of each other is left open.
- *The fitted self-excitation acts over about $160\ \mu\text{s}$.* It explains the runs that carry the tail at long service times, not the sub-microsecond engine gaps.
- *Conjecture:* the publisher’s capacity acts as a speed limit on information leaving the exchange (Section 5.8).

6 The analytical framework checked against the simulations

This section checks the closed-form predictions of the analytical framework (Section 2) against the results of the simulations on the recorded corpus (Section 4): each proposition or theorem is stated with the prediction it makes and the simulated numbers that test it.

Proposition 1 (burst-limit throughput). The single-message identity $p_{50}(N) = T + (N - 1)h$ is predicted exactly by Proposition 1 for any arrival law when the median message finds the queue empty. Table 3 confirms it in every cell of the Poisson stream and in all but three cells of the real stream, to the reporting precision; the three exceptions are the longest service times, where the median message itself waits (Section 4.5).

Theorem 1 (reduction and scaling bound). The prediction is $\Delta(N) \leq \Delta(1)/N$ on every window, and more strongly $W_j^{(N)} \leq W_j^{(1)}/N + (N - 1)h$ for every individual message. The corpus-median ratios in Table 11 are at or below $1/N$ in every cell, with a wide margin: the closest any cell comes is about a third of the bound. The margin is largest where a split drives the per-stage service below the publisher period and removes the tail outright rather than dividing it, which is why the fitted decay exponent is about three rather than one. The per-message form was checked directly by the simulator on every window, every service time, all four packet-level streams and every stage count: no message violates it. The reduction identity, Equation (4), is visible in Table 11

without any fitting: Δ is constant along every diagonal of constant T/N , so the tandem’s tail is the single-server tail at $s_{\max}$ on real arrivals exactly as on synthetic ones (Appendices B and C).

Corollary 2 (Poisson null). The prediction is $p_q^P(N) = T + (N - 1)h$ wherever the per-stage utilisation satisfies $\rho/N < 1 - q$, and a visible wait wherever it does not. Table 3 matches both halves: the Poisson p_{99} exceeds the no-queue latency in exactly the six cells whose per-stage utilisation is above 1%, all at long service times, and in no other cell. The Poisson $p_{99.9}$ likewise exceeds it exactly where the per-stage utilisation is above 0.1%. The Poisson stream is therefore a working queue whose small wait appears where theory says it must, and splitting under it costs $(N - 1)h$ at every reported quantile.

The renewal null against the cluster picture. The gap shuffle of Section 4.2 is a renewal stream with the real gap distribution, so under it the chance that a short gap is followed by another equals the share of short gaps $F(T)$, and runs of short gaps are as long as independent draws make them. The cluster picture of Section 2 predicts that the real stream has longer runs, and more so at longer service times, where many members of one cluster fall within one service time. Table 20 compares the two on 254 windows from 20 sessions. The real runs are longer than the renewal ones at every service time, and the excess grows with T from $16\mu\text{s}$: at $16\mu\text{s}$ the runs are within a few percent of renewal and the shuffle keeps most of the tail, while at $128\mu\text{s}$ they are a quarter longer and the shuffle keeps little. At $8\mu\text{s}$ short gaps are strongly bunched, six times the renewal chance, yet the shuffle keeps most of the tail there too, because the tail at that service time is a fraction of a microsecond and consists of single waits that need no run.

T (μs)	8	16	32	64	128
chance a short gap is followed by another, real	0.120	0.209	0.363	0.534	0.725
same, renewal stream (share of short gaps $F(T)$)	0.020	0.183	0.325	0.456	0.655
mean run of short gaps, real	1.14	1.26	1.57	2.14	3.63
mean run of short gaps, renewal	1.02	1.22	1.48	1.84	2.90

Table 20. Runs of short gaps (gaps shorter than T) on the real stream against a renewal stream with the same gaps in random order; per-window medians over 254 windows from 20 sessions. What the table shows: short gaps follow one another more often than chance at every service time, and the real runs grow longer than the renewal ones as T grows. What follows: the runs are what the gap shuffle destroys, which is why it removes more of the tail at long service times (Table 4).

The cluster representation against Section 4.3. The Hawkes–Oakes picture of Section 2.2 is one of immigrant events each generating a cluster of offspring; Section 4.3 identifies the events as matching-engine transactions. Two of its findings are what the picture predicts if the feed relays events one packet each: the branching ratio fitted on transaction starts equals the one fitted on packets (0.795 against 0.798), and merging each transaction into one packet or decoupling transaction sizes from their timing leaves the tail unchanged (Table 8). The corpus also shows that the exchange does not add clustering of its own, since the pieces of a split transaction are paced at $15\mu\text{s}$, above the publisher period. What the corpus does not test is the cluster-size law itself: the Borel distribution of Appendix A is a statement about offspring counts, which the data do not label, and its check in Appendix A remains indirect.

Summary. All three analytical statements hold on the full corpus. The per-message bound of Theorem 1 holds on every message of every window under all four packet-level streams, and the tail

falls far faster than the bound requires: where a split takes each stage below the publisher period, it removes the tail rather than dividing it. The renewal stream behaves as the cluster picture says it should, keeping the one-gap tail at short service times and losing the run-driven tail at long ones, and the transaction analysis places the clusters in the matching engine's event stream.

Findings.

- *Proposition 1 holds:* $\text{median} = T + (N - 1)h$ in all but three cells, the exceptions at the longest service times.
- *Theorem 1 holds on every message of every window,* and the tail falls far faster than its $1/N$ bound.
- *Corollary 2 holds:* the Poisson wait appears exactly where utilisation per stage exceeds $1 - q$.
- *Runs of short gaps are longer than a renewal stream's,* increasingly so with T , which is why shuffling the gaps removes more of the tail at long service times.

7 Cross-validation against a live production receiver

The results of Part II so far are measurements on recorded data: pcap-derived message tapes replayed offline. That design gives scale and repeatability at the cost of external validity. A replayed tape cannot show whether the fitted clustering survives the network path to a receiver, whether it is specific to the 2025–2026 sample, or whether the service times the sweep assumes correspond to production decode times.

This section addresses all three with instrumentation of the KASPAR-HFT production system. The two measurements were built for different purposes and share only the exchange they observe. (The live measurement was taken seven months after the corpus ends, so it also tests whether the clustering is specific to the period the corpus covers.) One limit is stated at the outset: the live per-message record carries the receive and publish timestamps, the ring depth and the position in the datagram, but not the message’s `transactTime`, so the transaction-level identification of Section 4.3 is a corpus result that the live data cannot repeat; what the live data cross-validate is the packet process, and the transaction result rests on the corpus alone.

7.1 The recorded corpus and the live measurement compared

The live measurement instruments the KASPAR-HFT market-data path in production: a timestamp `t0` when a UDP datagram is handed up from the socket, a timestamp `t1` when the resulting book update is published, and one 16-byte record per admitted message carrying the pair together with the ring-buffer depth and the message’s position inside its datagram. It covers 6.81 million messages on six streams — ES, NQ and ZN futures, book and trade — over 53 minutes of a single afternoon. Its latency results are reported in Maciejewski [36]; we draw on them here, and on the underlying per-message records, as a control on Part II.

	Part II corpus	production measurement
source	pcap tapes, replayed offline	live multicast, production recorder
period	2025-01 to 2026-02	2026-09-16 (7 months out of sample)
instruments	NQ front month	ES, NQ, ZN $\times$ book, trade
arrival clock	CME gateway <code>sendingTime</code>	receiver <code>t0</code> at socket read
estimator	per-window, median of 3512 windows	pooled over one 53-min capture
sample	3512 windows, 276 sessions	6.81 M messages, 6 streams

Table 21. The two measurements compared in this section: different instruments, a period that does not overlap, a different arrival clock, and a different estimator.

The arrival-clock difference is the most significant. Part II keys arrivals on `sendingTime`, the timestamp CME’s publisher writes into the packet header as the packet leaves the exchange. The live measurement keys them on `t0`, taken by our own process after the packet has crossed the network, the NIC and the kernel. If the clustering reported in Section 3.2 were an artefact of exchange-side emission scheduling, the intervening path, with its own queueing, coalescing and interrupt moderation, would be expected to attenuate it. It does not; and Section 4.3 rules out the exchange’s packetisation as the source from the other side, by showing that the clustering is already present in the matching engine’s transaction stream.

7.2 Clustering statistics in the corpus and on the live receiver

This section checks that the bursty arrival pattern the paper is built on is real, and not something produced by replaying recordings. It takes four numbers that each measure how far packet arrivals are from evenly spread. The first is the share of gaps between packets that are tiny, under a tenth of the average gap. The second is how much the variability of packet counts drops when the order of the packets is scrambled, which removes any bunching. The third is a measure of whether busy stretches persist rather than fading at once. The fourth is how strongly one event tends to trigger the next. Each of the four is computed twice, independently: once on the recorded corpus that the rest of the paper uses, and once on the live production receiver, seven months later, on different instruments, with a different clock. Three of the four match directly, to within the spread seen across the live streams. The fourth is measured in two different ways, one an estimate and one an upper bound, and the estimate falls inside the bound. The conclusion is that the bursts are a property of the feed as received, not of replay: they show up at the same strength whether one looks at a recording or at the live wire. Where they come from is the question Section 4.3 answers on the corpus: the matching engine’s transaction stream.

Table 22 puts the four statistics that both measurements compute side by side. The NQ column is the like-for-like comparison, since Part II’s corpus is NQ front month; the third column gives the range across the three live book streams.

	Part II (NQ, 3512 win)	live NQ book	live, three books
$P(\text{gap} < \text{mean}/10)$	73.2%	63.1%	63.1–84.8%
Fano(5s) collapse under shuffle	14.8×	12.2×	12.2–18.5×
Hurst exponent H	0.740	0.702	0.702–0.743
branching ratio n	0.798 (MLE)	$\leq 0.850\text{--}0.972$ (Fano ceiling)	

Table 22. The same four Poisson-rejection statistics, computed independently on recorded and on live data. The Poisson values are 9.52%, 1×, 0.5 and 0 respectively.

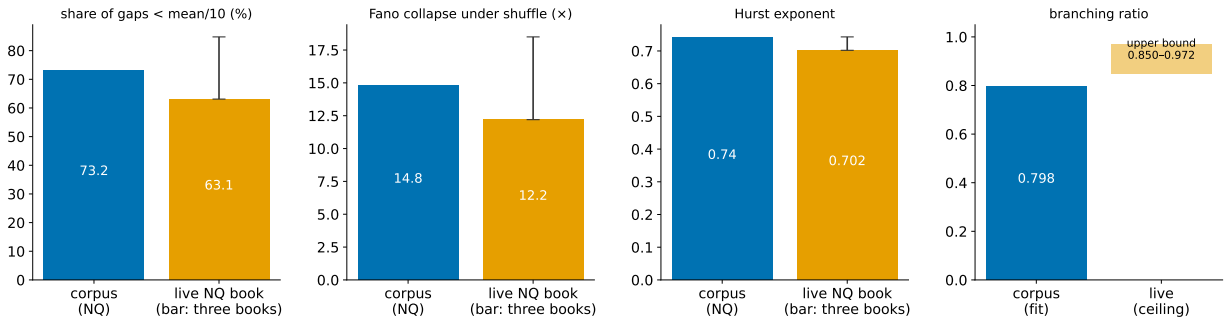


Figure 21. Table 22 as bars: the clustering statistics on the recorded corpus (blue) and on the live receiver’s NQ book stream (orange; whiskers span the three live book streams). The share of tiny gaps, the collapse of the Fano factor under shuffling and the Hurst exponent agree within the spread of the live streams. The corpus branching ratio lies below every upper bound the live streams give (shaded).

Three of the four are direct comparisons and agree to within the spread across live streams. The fourth compares different estimators. Part II fits the branching ratio by maximum likelihood, per window, using the Ogata recursion (Section 3.2); the live measurement does not fit a Hawkes process, and bounds n from above by inverting the Fano ceiling, $n \leq 1 - 1/\sqrt{F}$. The MLE’s 0.798 lies within

every bound the live streams imply, which run from 0.850 on NQ trade to 0.972 on NQ book. The bound is loose, and the agreement is weak evidence on its own; its value is that it is independent evidence and is consistent.

The arrival process modelled in Part II is therefore not an artefact of replay, of the sample period, or of the exchange-side clock; it is present at the same strength in a live receiver seven months later on other instruments.

Findings.

- *The clustering is not an artefact of replay:* the share of tiny gaps, the collapse under shuffling and the Hurst exponent agree between the corpus and the live receiver, on other instruments and a different clock.
- *The branching ratio* fitted on the corpus (0.798) lies inside every upper bound the live streams give (0.850–0.972).

7.3 The production decode floor and the tail threshold

The live system supplies one number that a recording cannot: how long the production decoder actually takes to handle a message when nothing is queued in front of it. This section measures that time three independent ways and finds they agree, at about seven microseconds. The notable result is where that number falls. Part II found, from the recorded corpus, that a tail appears once a stage takes longer than the tightest gap between packets, about seven and a half microseconds. The production decoder sits just under that, within a few hundred nanoseconds, by two measurements that share nothing. The production system therefore operates right at the edge of the tail. The section then shows what pushes it over: a packet carrying a single message is handled below the threshold, and a packet carrying two or more at or above it, on every stream measured. Whether a packet’s service crosses the threshold is decided by whether it carries one message or more.

Isolating messages that arrive to an empty ring and first in their datagram, so that neither queueing nor in-packet serialisation contributes, gives the hot path directly. Three estimators agree (Table 23):

method	ES	NQ	ZN
direct median, empty ring and first in packet	7.01	7.24	6.89
fitted intercept of the median position ladder	6.98	7.23	6.83
single-message packets during an FOMC release	7.1	7.5	7.0

Table 23. Three independent estimates of the KASPAR-HFT decode floor (socket to book), in μs , on the book streams. The three differ in sample, estimator and load regime. The third was taken while the packet rate rose 4.6 to $9.8\times$ within a second; the decode floor rose by at most $0.27\mu s$ against either other estimator.

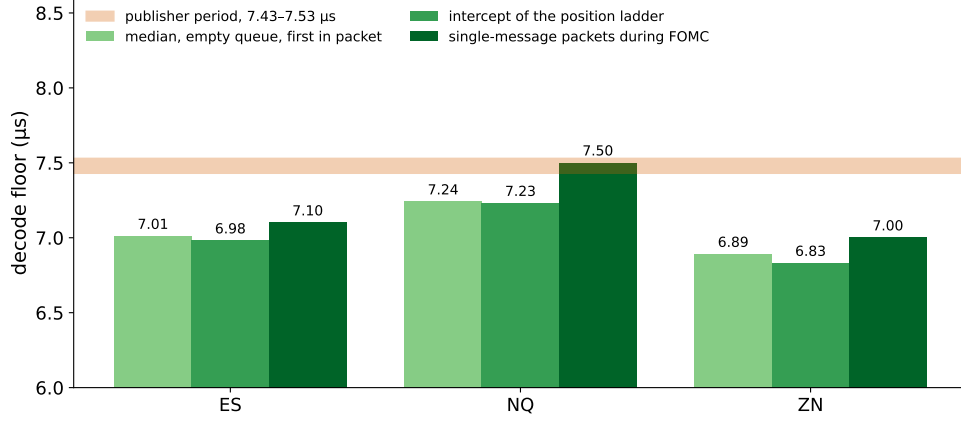


Figure 22. Table 23 as bars: the production decode floor by three independent estimates on each instrument, against the tail threshold measured on the corpus (shaded band). All nine estimates lie in or just below the band: the production receiver runs right at the edge of the tail.

Across all six streams the fitted decode floor lies within half a microsecond of $7 \mu s$, on six multicast channels on one host and one afternoon.

Compare this with Section 4.9. The service time at which a stage on this feed begins to build a tail is the publisher period, the tightest gap between packets, s_{pub} , the same at every level of market activity; the refined sweep of Section 4.8 puts the switch-on between 7 and $8 \mu s$. The production decode floor is about $7 \mu s$.

The production system operates at the threshold, to within a few hundred nanoseconds, by two measurements made on different data with different estimators. The agreement is close to definitional for a p_{99} target, since a deterministic server leaves its no-queue latency at p_{99} once about one packet in a hundred arrives within T of the previous one; for a $p_{99.9}$ target the relevant spacing is the 0.1st-percentile gap, which is lower, and Table 3 already shows a $p_{99.9}$ excess at $T = 8 \mu s$.

The live measurement locates the crossing. Per-packet service is the decode floor plus one slope for each message beyond the first (Section 7.5), so the service time of a datagram carrying span messages is decode floor + $(\text{span} - 1) \times \text{slope}$. Evaluating that on each stream’s own fitted pair:

	span 1	span 2	span 3
ES book	6.98	7.55	8.11
NQ book	7.23	7.54	7.85
ZN book	6.83	7.80	8.76
ES trade	6.81	7.52	8.24
NQ trade	7.14	7.67	8.19
ZN trade	6.81	7.77	8.74
range	6.81–7.23	7.52–7.80	7.85–8.76

Table 24. Per-packet service time in μs by datagram span, from each stream’s measured decode floor and slope, against the publisher period s_{pub} (Section 4.9). Every stream is below the threshold at span 1 and at or above it at span 2; ES trade, at 7.52, sits inside the 7.43–7.53 band.

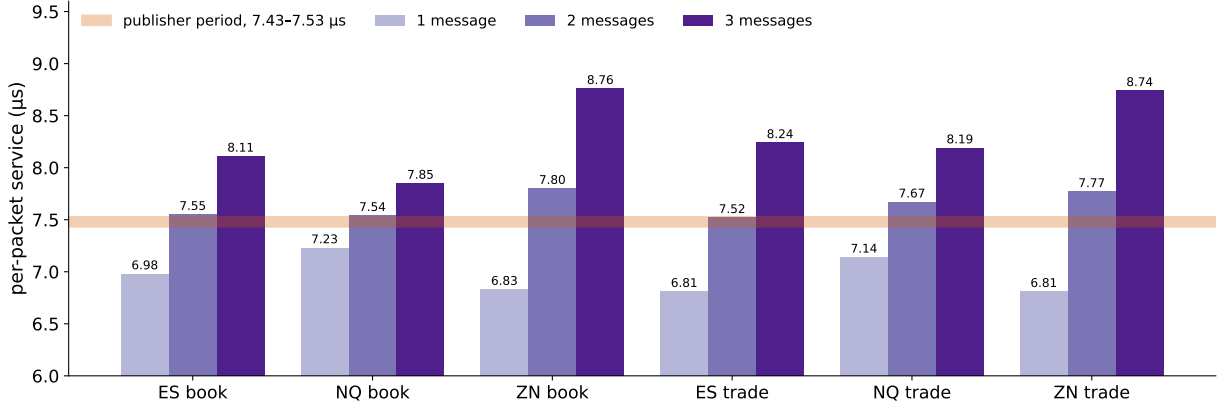


Figure 23. Table 24 as bars: per-packet service for one, two and three messages on each live stream, against the tail threshold (shaded band). A one-message packet is served below the threshold on every stream; a two-message packet is at or above it on every stream. Whether a packet can start a queue is decided by whether it carries more than one message.

All six streams are below the threshold when a datagram carries one message and at or above it when it carries two (Table 24). All six crossings fall in the same unit interval of span, from decode floors and slopes fitted separately on six multicast channels against a threshold measured on a different corpus. For this system, whether a packet’s service is above or below the threshold is determined by whether it carries one message or more.

Findings.

- *The production decode floor is about 7 μs by three independent estimates, just under the publisher period.*
- *A packet with one message is served below the publisher period, one with two or more at or above it, on all six live streams.*

7.4 Statistics that differ between the corpus and the live receiver

Two numbers do not match between the recorded and the live measurements, and this section explains why neither is a cause for concern. In both cases the disagreement is about how the number was computed, not about the feed, and in both cases one of the two measurements can diagnose the other. The first is the raw level of burstiness, which reads about four times higher live. That is because the live figure was taken over one uncut stretch that mixes a quiet period and a busy one, and a rate that drifts across the stretch inflates the measure whether or not the arrivals are genuinely bursty; the recorded corpus computes the same quantity inside each half-hour window, which removes that drift. The change in burstiness when the packets are scrambled, which is the effect the paper uses, agrees closely between the two. The second is a control value that should come out at exactly one half after scrambling, and reads a little above it live. That is a known small-sample bias of the estimator on a short capture; on the much larger recorded corpus the same control comes out at one half, as it should. Neither discrepancy touches any quantity the paper’s argument rests on, and for the level of burstiness the recorded corpus gives the more conservative value, which is the one reported.

The absolute Fano factor and the length of the stretch. Two estimators are in play. The *windowed* estimator cuts the data into 30-minute windows, computes the Fano factor of the bin counts inside each window, and reports the median across windows; the *pooled* estimator treats one long uncut stretch as a single series and computes the Fano factor over all of it. (This is a different sense of “pooled” from the histograms of Section 4.3, which pool gap *lengths* across windows into one distribution; a distribution of lengths is indifferent to slow rate drift, a count-variance statistic is not.) Part II, windowed, measures a corpus-median Fano at a 5-second bin of 259.7; the live streams, pooled over the 53-minute capture, give 1024 to 1260. The ratio under the gap shuffle agrees closely (Table 22), so the discrepancy is in the level, not in the effect. The cause is the estimator row of Table 21. The live figure is computed over one uncut 53-minute capture that spans a quiet stretch and a busy one; a rate that drifts between the two inflates the count variance and reads as clustering whether or not the process is self-exciting. Part II computes the Fano factor *within* each 30-minute window and then takes the corpus median, which removes between-window rate variation by construction: a slow drift from a quiet stretch to a busy one adds count variance that the estimator cannot distinguish from bursts, so the pooled level reads roughly four times higher. The gap-shuffle ratio cancels the drift, since the drift is present in the numerator and the denominator alike, which is why the ratio agrees between the two measurements and why the paper uses the ratio, not the level. The windowed estimate is the conservative one and is the value reported.

The shuffled Hurst null and finite-sample bias. A shuffle of a gap sequence destroys the ordering while keeping every gap, so the shuffled series is a renewal process and its Hurst exponent must be 0.5. The live measurement recovers somewhat more, attributes the excess to finite-sample bias in the Hurst estimator on a short capture, and states its result as a difference from the unshuffled value rather than a distance from 0.5. Part II confirms the attribution: over the whole corpus the same shuffle, and the Poisson null, both return 0.50 (Section 3.2). The bias vanishes with more, shorter series.

Findings.

- *The live Fano level reads four times higher* because it is computed over one long stretch whose rate drifts; the shuffle ratio the paper relies on agrees.
- *The live shuffled Hurst exponent sits above 0.5* from small-sample bias; on the corpus it is 0.50.

7.5 Per-packet service time, message count and coalesced transactions

The simulator in Part II assumed that every packet takes the same time to process. The live measurement shows that it does not, and this section sets out what that means. A packet on this feed carries a variable number of messages, usually one but sometimes dozens, and they are decoded one after another, so a packet takes longer the more it carries. Large packets are rare but expensive: on the most fragmented stream, ZN trade, one packet in nine carries two thirds of the messages and consumes six sevenths of the decode time. The task length used throughout the paper is therefore the work for a typical packet, not a fixed constant of the system. Two points follow, set out at the end of the section: the results survive, because the cost per message is constant and only the bundling varies; and the number of messages per packet is a second place where the same clustering is paid: transactions that reach the publisher while it is busy can be packed into one packet rather than sent in separate packets (Section 5.5). It acts mainly where bursts across packets do not, on tasks at or just below the tightest gap between packets, which is where production receivers run, and

adds a few percent above $16\mu\text{s}$. The next section reruns the sweep with the message count included and measures the share of each channel.

The sweep of Section 3.5 treats each packet as one arrival with a constant service time T . Per-packet service is not constant. A UDP datagram on this feed carries a variable number of SBE messages — call it the packet’s *span* — which are decoded in order, so a message at position k waits for k decodes before its own. Median latency follows

$$\text{median} = \text{decode floor} + \text{slope} \times \text{position},$$

with the decode floor of Section 7.3 and a slope of 0.31 to $0.97\mu\text{s}$ per message depending on the stream. Span is not close to constant (Tables 25 and 26):

stream	packets	messages	p_{50}	p_{90}	p_{99}	max
ES book	2,123,042	2,415,610	1	1	7	45
NQ book	2,506,205	3,010,036	1	2	11	36
ZN book	824,536	965,334	1	1	6	45
ES trade	83,140	235,601	3	8	17	83
NQ trade	50,279	83,233	2	7	16	85
ZN trade	22,864	100,659	2	11	71	85

Table 25. Messages per UDP datagram (span), packet-weighted quantiles. The median book packet carries one message; the 99th percentile carries 6 to 11 on the book streams and 71 on ZN trade.

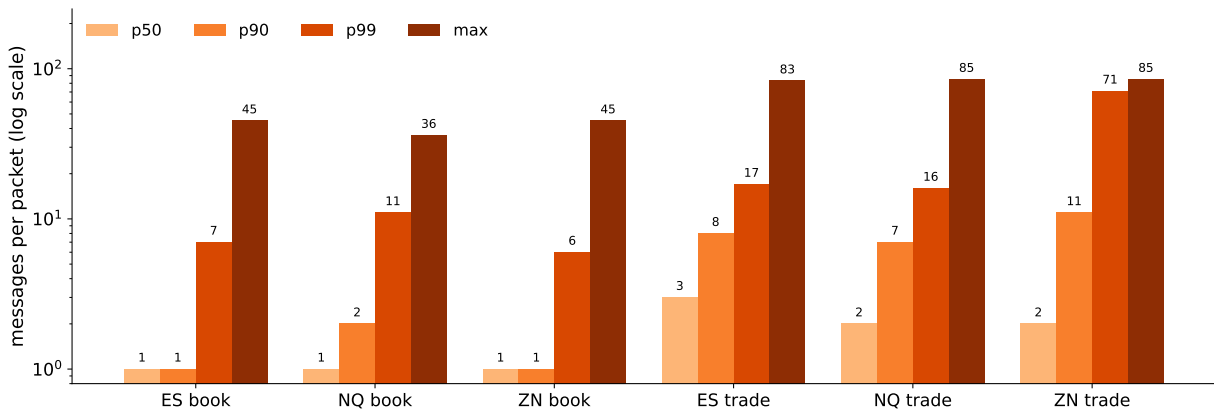


Figure 24. Table 25 as bars, log scale: messages per packet at the median, 90th and 99th percentile and the maximum, per live stream. Book streams carry one message in most packets and up to 45; trade streams carry more, and ZN trade reaches 71 at p_{99} .

Because service is charged per message, large packets account for a disproportionate share of processing time:

span ≥ 10	% of packets	% of messages	% of decode time
ES book	0.52	4.55	5.34
NQ book	1.68	4.87	2.57
ZN book	0.47	7.29	7.29
NQ trade	5.34	19.31	16.16
ES trade	6.69	26.53	40.10
ZN trade	11.31	66.18	85.54

Table 26. Concentration of work in large datagrams. On ZN trade, 11.3% of packets carry 66% of the messages and consume 86% of decode time.

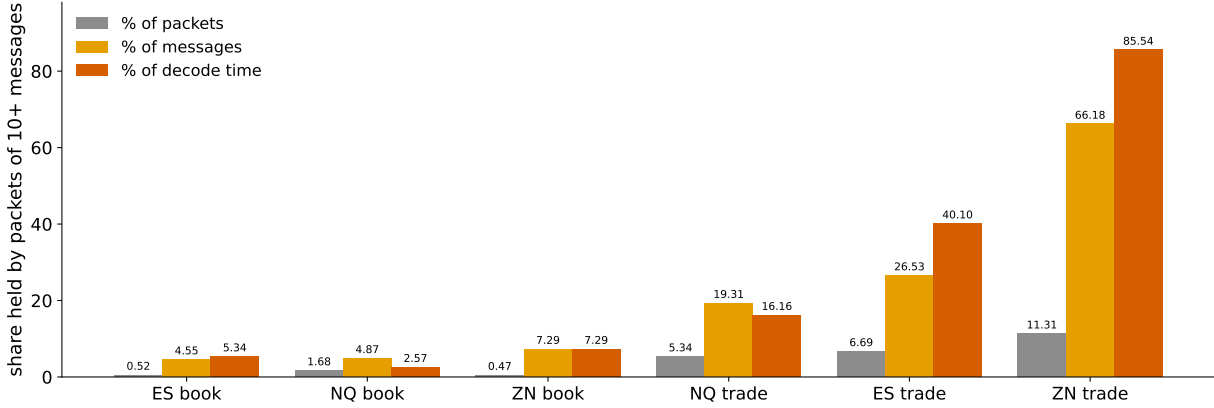


Figure 25. Table 26 as bars: the share of packets, of messages and of decode time held by packets of ten or more messages. On ZN trade 11% of packets hold 66% of the messages and 86% of the decode time; on the book streams long packets are under 2% of packets but several times that share of messages.

The constant-service assumption is therefore violated at the packet level, and Part II’s T is the per-packet work at a typical span rather than a constant of the system. Two points follow.

First, at the message level the assumption approximately holds: the slope is fixed within a stream to the precision of the fit, and it is the composition of packets, not randomness in per-message work, that makes per-packet service vary. Theorem 1 is pathwise and arrival-law independent, and Remark 2.3 already handles unequal stages through $s_{\max}$; what the live data adds is a measured magnitude for the variability that Section 2.3 lists as an assumption.

Second, span is a second channel for the same clustering on the same stream, active where the queueing channel is not. Section 7.6 reruns the sweep with span-dependent service. The serialisation channel supplies the tail below the publisher period, where queueing across packets supplies none, and adds a few percent above $16\mu s$, where queueing across packets supplies nearly all of it.

Spans and transactions. On the corpus, where every message carries its `transactTime`, a multi-message packet is usually not one large transaction but several transactions delivered together. About 4% of packets carry two or more messages, and about three quarters of those carry more than one transaction. The publisher, that is, packs transactions that reach it while it is busy into one packet (Section 5.5). This is consistent with the publisher period: two transactions that the engine processes closer together than s_{pub} never appear as two packets closer together than that; they appear either as one packet with more messages or, more often, as two packets at the publisher period (Table 14), which is why so few packet gaps fall below the publisher spacing (Table 6). Two

consequences follow for the rest of this section. The in-packet tail that a large packet produces is, for the most part, the same clustering of transactions as the queueing tail, delivered inside a packet instead of across packets; and the message count per packet is set by how closely transactions follow one another, not by how large individual transactions are. The two-mechanism description below, queueing across packets and decoding within them, stands as a description of where the wait is paid; both originate in the transaction stream.

Findings.

- *Service is not constant per packet:* each further message adds a fixed cost, from about a third of a microsecond to about one depending on the stream.
- *Long packets are rare but expensive:* on ZN trade one packet in nine carries two thirds of the messages and most of the decode time.
- *A long packet is usually several transactions packed together:* about three quarters of multi-message packets hold more than one transaction.

7.6 The sweep rerun with span-dependent service

The previous section found that a packet takes longer to process the more messages it carries. This section reruns the whole simulation with that fact built in, to find out what it changes. The processing time is modelled three ways: fixed per packet, as in Part II; growing with message count at the rate measured on the live system; and, as a deliberately exaggerated upper bound, fully proportional to message count. All three do the same total work on average, so any difference in the slow tail comes from how the work is spread across packets. There are four findings. First, message count and bursts produce slow messages in different places. Message count matters most for short tasks, at or just above the tightest gap between packets, which is where production receivers run, where bursts build little or no queue; for longer tasks the bursts dominate and message count adds a few percent at $16\mu\text{s}$ and one or two beyond. Second, at the simulated task length nearest the production system's seven microseconds, eight, message count accounts for roughly two thirds of the slow tail and bursts for one third, and the live system reports the same ordering. Third, the recommendation on how many stages to split a task into is unchanged at every task length. The in-packet part of that tail is the plain cost of decoding several messages one after another, which splitting the work into stages does not remove; the queue a large packet opens behind itself is taken up in Section 7.7. Fourth, on NQ the two channels do not compound at the packet level: a large packet is no more likely to arrive after a tight gap than after a loose one, checked two ways, so the tables of Part II, which measure the queueing channel alone, stand as they are at service times above the publisher period. On ZN the live data suggest the opposite, and the test has not been run there.

We re-extracted the per-packet span for all 3512 windows — aligned element-wise against the cached arrival arrays, with every window's span count checked against the packet count already in the cache — and re-ran the entire sweep with service that depends on it.

Three service models are compared. All three hold the *mean* service per window at T , so the utilisation ρ is identical across them and only the distribution of work across packets differs; any difference in the tail is attributable to the span dependence.

const :	$S_i = T$	(the Part II assumption)
span :	$S_i = A(1 + r(\sigma_i - 1))$, $r = 0.0432$	(measured slope / decode floor)
prop :	$S_i = T\sigma_i/\bar{\sigma}$	(upper bound)

with A set so that the window mean of S_i is T . The *span* model uses the ratio of slope to decode floor measured on NQ book in Section 7.3, 0.312/7.23. The *prop* model removes the per-packet fixed cost, making service proportional to the message count; this overstates the dependence, since the measured decode floor exceeds the slope by more than twenty to one, and it is included as an upper bound. As a correctness check the *const* arm reproduces the published constant-service grid of Table 3 to zero difference in all 28 cells.

Message-weighted quantiles. On this corpus, pooled over its 3.30×10^9 packets, 95.9% of packets carry one message and the mean span is 1.078 (per-window medians 96.0% and 1.077), so a packet-weighted quantile is dominated by singletons and changes little under any service model. Weighting by message, which counts a 20-message datagram twenty times, the in-packet position distribution is concentrated at zero with a long upper tail:

message quantile	p_{50}	p_{90}	p_{95}	p_{99}	$p_{99.9}$	max
in-packet position	0	0	1	4	19	84

89.0% of messages are alone in their packet and pay no serialisation; the remaining 11.0% constitute the tail. The live measurement reports the same shape. The comparison below is message-weighted.

T	const			span			span/const		prop/const	
	p_{50}	p_{99}	$p_{99.9}$	p_{50}	p_{99}	$p_{99.9}$	p_{99}	$p_{99.9}$	p_{99}	$p_{99.9}$
2	2.00	2.00	2.00	1.99	2.34	3.54	1.17	1.77	4.64	23.35
4	4.00	4.00	4.00	3.99	4.67	7.08	1.17	1.77	4.69	26.73
8	8.00	8.56	10.10	7.97	9.63	14.87	1.12	1.47	5.21	24.64
16	16.00	34.00	53.55	15.95	35.08	56.62	1.03	1.06	3.36	11.53
32	32.00	143.74	256.51	31.89	144.41	260.14	1.00	1.01	2.37	5.88
64	98.80	815.22	1557.72	100.02	820.05	1573.97	1.01	1.01	2.37	3.94
128	366.78	5440.22	12844.51	371.91	5550.42	13159.71	1.02	1.02	2.51	3.10

Table 27. Effect of the number of messages per packet (the packet’s *span*, not its byte length) on the single-stage tail. Each row is one service time T ; every entry is the corpus median over 3512 windows of a message-weighted latency quantile, in μs , under real (Hawkes) arrivals with a single stage ($N = 1$). The *const* block charges every packet the same service T , the Part II assumption. The *span* block charges service that grows with the number of messages in the packet, at the slope measured on the live system. Both do the same mean work per window, so any difference between the two blocks is due to message count alone. The *span/const* columns give that difference directly as a ratio; a value near 1 means message count adds nothing. The *prop/const* columns repeat the ratio for a deliberately exaggerated model in which service is fully proportional to message count, as an upper bound. Read across: message count inflates the tail only at $T \leq 8 \mu\text{s}$, at or just above the publisher period, where clustering builds little or no queue, and is negligible above it.

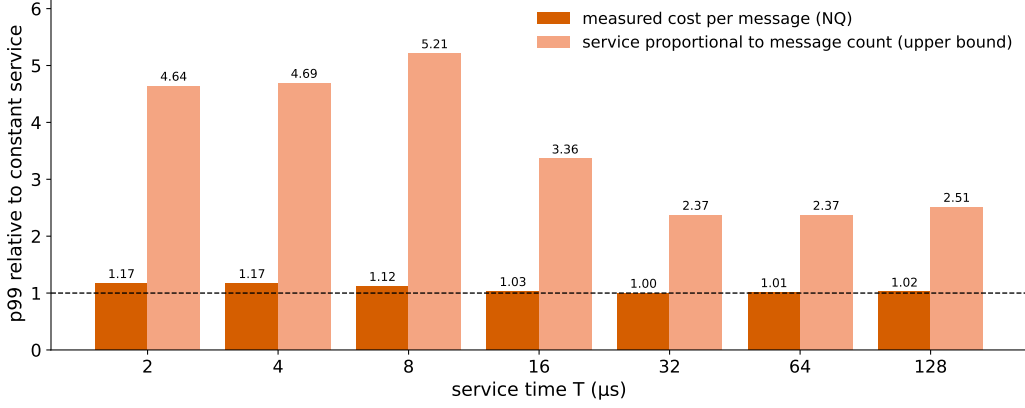


Figure 26. Table 27 as bars: p_{99} under service that grows with the message count, relative to constant service with the same mean. With the measured cost per message (vermillion) the tail is 12–17% larger at 8 μs and below and within 3% from 16 μs . The exaggerated model in which service is proportional to message count (light) bounds the effect from above.

Regimes of the two tail mechanisms. The span/const ratio of Table 27 is largest where the queueing tail is absent and smallest where it dominates. At 2 and 4 μs the constant-service model has no tail at all, while the span-dependent model has a small one: below the publisher period there is a tail, but it is not a queueing tail. From 32 μs , where queueing across packets makes the p_{99} several times T , span adds at most a few percent.

Decomposition at the production operating point. The live system’s measured decode floor is 6.81–7.23 μs (Section 7.3), so the comparable row is $T = 8 \mu s$. Decomposing the message-weighted p_{99} excess over the median (p_{50}) into the part the constant-service model already produced (queueing) and the remainder introduced by span-aware service (serialisation):

T	p_{50}	queueing excess	serialisation excess	serialisation share
2	1.99	0.00	0.34	100%
4	3.99	0.00	0.69	100%
8	7.97	0.56	1.09	66%
16	15.95	18.00	1.14	6%
32	31.89	111.74	0.78	1%
64	100.02	716.42	3.61	1%
128	371.91	5073.43	105.07	2%

At $T = 8 \mu s$ serialisation supplies 66% of the p_{99} excess and queueing 34%. The live measurement reports the same ordering: evaluating the position ladder at each stream’s own p_{99} position accounts for 71% to 80% of the observed p_{99} on four of its six streams, with the remainder left to queueing and to stalls. Applying that definition to the simulator at $T = 8$ gives 97%, higher than the live figure because the only residual the model contains is queueing; it has no stall population. Both measurements attribute the larger share of the tail at the production service time to in-packet serialisation.

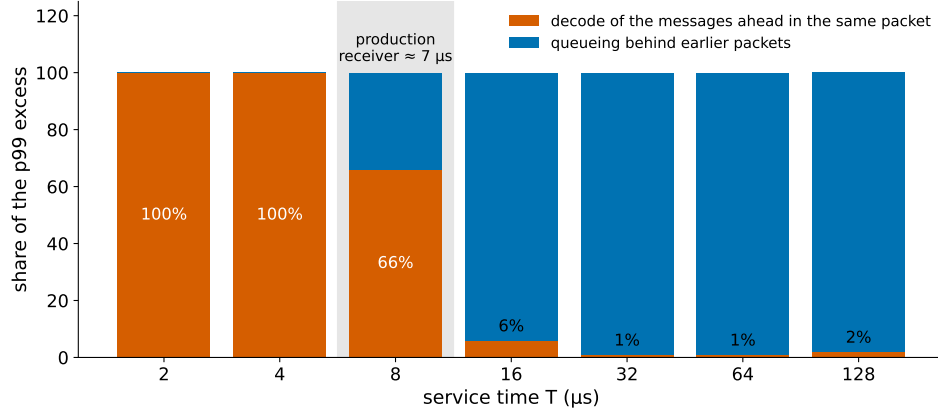


Figure 27. The decomposition above as bars: of the simulated p_{99} excess at each service time, the part due to decoding the messages ahead in the same packet (vermillion) and the part due to queueing behind earlier packets (blue). At and below the publisher period the decode of long packets is the tail; the shaded bar, $T = 8 \mu s$, is the nearest simulated service time to a production receiver’s decode floor, and there it is two thirds. From $16 \mu s$ queueing is almost all of it.

This is consistent with Part II. The queueing claim stands: below about s_{pub} of service, arrival clustering does not build a queue and splitting the servicing chain removes nothing. The span-dependent run adds that a second mechanism, in-packet serialisation, is active in the same regime; it is deterministic within the packet and a stage cut does not shorten it, while the queue that a large packet opens behind itself is taken up in Section 7.7. N^* , the p_{99} -minimising stage count, is unchanged at all seven service times when service is made span-dependent. The design rule of Section 12 is unaffected.

The in-packet ladder. Median latency by in-packet position at $N = 1$ reproduces the linear form of the live measurement. At $T = 8 \mu s$ the corpus-median latency by position fits

$$\text{median} = 7.98 + 0.344 \times \text{position} \quad (\mu s),$$

recovering the $0.345 \mu s$ per-message cost that was input to the model, which confirms that queueing contributes nothing at the median at this service time. The live NQ-book ladder is $7.23 + 0.312 \times \text{position}$; the forms agree and the levels differ by the difference between the measured decode floor and the $8 \mu s$ nominal service time.

Compounding of the two channels on NQ. There are two places a message can be made slow. One is queueing: it waits behind a run of earlier packets. The other is serialisation: it sits deep inside a large packet and waits for the messages ahead of it in that same packet to be decoded first. Section 7.5 showed that a large packet is usually several transactions packed together by the publisher, so both are deliveries of the same clustering. The question here is narrower: whether a large packet also tends to arrive behind a tight gap, so that one message pays both delays at once. If it did, the two would compound and the Part II tables, which model queueing alone, would understate the tail above the publisher period. If a large packet arrives after a tight gap no more often than after a loose one, the tables stand. Two tests find the second case. Both compare a packet’s span with the gap that precedes it, so they cannot see the coalescing itself, which removes a gap rather than tightening one.

The first is a control arm in which each window's span sequence is randomly permuted across packets, preserving the span marginal and the arrival times exactly and destroying only the association between them. If large packets arrived preferentially inside clusters, breaking that association would lower the tail. It does not: the real and the shuffled p_{99} agree within a couple of percent at every service time, with no systematic sign. Second, measuring the association directly, a packet's size is almost uncorrelated with the gap before it, and packets arriving after the shortest gaps are, if anything, slightly smaller than those after the longest. The association is weakly negative.

The live measurement reports the same instrument dependence: mean span rises with queue depth on ZN, where a quiet book is filled only by genuine bursts, but *falls* with queue depth on ES and NQ, whose rings back up with many small packets. Part II's corpus is NQ, and it behaves like the live NQ streams. The transaction-level arms of Section 4.3 give the same answer a third way: decoupling transaction sizes from their timing (TS) and merging each transaction into one packet (TM) leave the tail within $\pm 15\%$ at every service time from $16\mu\text{s}$ up (Table 8). Above the publisher period, then, the serialisation channel adds a few percent to a tail the queueing channel produces; at the publisher period, where the production receiver runs, it carries most of the tail, and the constant-service tables of Part II measure that channel; at the publisher period, where serialisation carries most of the excess, the coalescing of transactions into one datagram described in Section 7.5 means the two channels are two deliveries of the same clustering. On an instrument like ZN, where the far tail is a few large packets, the second delivery dominates; Section 7.7 takes this up.

Findings.

- *At the production service time, long packets carry most of the tail:* in the simulation at the sweep point just above the publisher period about two thirds of it, and on the live receiver in-packet position reproduces about three quarters of the p_{99} on most streams.
- *From $16\mu\text{s}$ up, message count adds a few percent* to a tail set by queueing across packets.
- *The best stage count does not change:* a stage cut does not shorten the decode of a long packet.
- *On NQ long packets do not arrive preferentially after short gaps;* on ZN this is untested.

7.7 The production handler's tail

The live records also give the production handler's own tail, on the shipping serial configuration: one thread decoding every packet inline. The question is the one the paper opened with, put to a real handler. On the live feed, why is the slowest message in a thousand so much slower than the typical one? Read with Sections 4.3 and 7.5, the answer is that the clustering of transactions reaches this handler in two forms. Most transactions that the engine processes close together reach it as a train of packets one publisher period apart (Table 14); some are packed by the publisher into one packet and reach it as a single packet carrying several messages, which the handler decodes one message after another, so that the packet holds the thread for many times the decode floor and everything behind it queues. On the corpus, a multi-message packet is several transactions delivered together far more often than it is one large transaction (77% of packets with two or more messages carry more than one transaction, Section 7.5), so a large packet is a transaction cluster in its own right. On the ZN channel the second form dominates the far tail. What follows is argued from the fitted decode floor and slope of Section 7.3 and the reported percentiles, not from a message-by-message decomposition of the window.

Setup. Channel 344 (ZN), book stream, one 900 s window, recovery windows excluded, the shipping serial configuration (`cme_decode_workers 0`). Latency is the leg-1 socket-read-to-book-publish time of Section 7.3, per message. ZN is the instrument on which Section 7.6 left the coupling of span and bursts open. Two populations are reported: every message, and the *hot path*, the messages that arrived to an empty queue and were first in their packet, which is the decode with neither queueing nor in-packet serialisation in it.

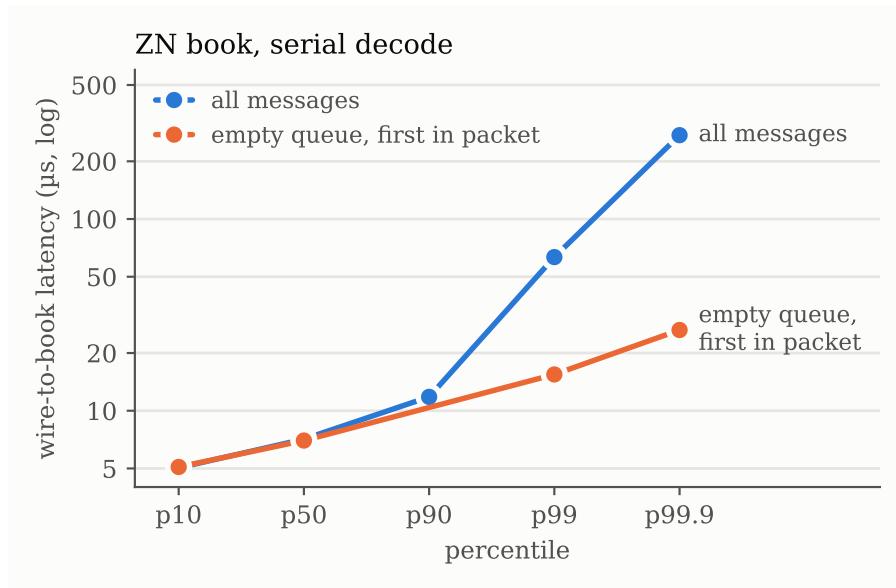


Figure 28. Wire-to-book latency by percentile on the ZN book stream, serial decode, for every message and for the hot path alone (empty queue, first in packet; its p_{90} was not reported). Log latency axis. The two populations share a minimum near $5\text{--}7\ \mu\text{s}$ and separate only in the tail: the hot path reaches $26\ \mu\text{s}$ at $p_{99.9}$, all messages $274\ \mu\text{s}$.

population	messages	p_{10}	p_{50}	p_{90}	p_{99}	$p_{99.9}$	mean
all messages	157,699	5.04	7.11	11.81	63.30	273.69	9.95
empty queue, first in packet	122,500	5.09	6.99	—	15.46	26.37	—

Table 28. The data behind Figure 28, in μs . The hot path is 78% of all messages. The window maximum for all messages was $3046\ \mu\text{s}$; the report that records this measurement (`tech_reports/serial_vs_parallel_decode.md` in the KASPAR-HFT repository) flags a few such isolated spikes as possibly residual recovery, so it is not used here.

The spread from the median to $p_{99.9}$. The slowest message in a thousand takes $274\ \mu\text{s}$, against $7\ \mu\text{s}$ for the median, some forty times as long. Restricting to messages that met an empty queue and were first in their packet removes nine tenths of that far tail, down to $26\ \mu\text{s}$, without changing the fastest messages. The difference between the two is the contribution of queueing and of position inside the packet. The restricted messages are not at the decode floor in their own tail either: about twice the decode floor at p_{99} and nearly four times at $p_{99.9}$. That residual, present with neither queueing nor in-packet decode, is not resolved by these records (Section 14.1).

How the live tail forms. The three findings of this paper meet here. Packets reach the handler as trains one publisher period apart (Section 5.5). The handler’s median service, about $7\ \mu\text{s}$ on the

book streams (Table 29), is just under that spacing, so a train of single-message packets served at the median builds no queue; this is why its p_{10} and p_{50} sit at the decode floor. Two things push a service over the spacing. A packet with two or more messages costs the decode floor plus one per-message slope for each further message, which crosses s_{pub} on every stream (Section 7.3). And a single-message packet sometimes takes far longer than the median for reasons these records do not show: on ZN, messages with nothing queued reach about twice the decode floor at p_{99} . Either way the packets still arriving behind it queue, and the queue lasts until the handler catches up with the train. The large-packet mechanism is a matter of arithmetic on the measured decode floor and slope; the slow-service mechanism is measured only as the empty-queue percentiles above. How the ZN $p_{99.9}$ divides between the decode inside long packets, the queue behind them and slow services was not decomposed, because the per-message records were not retained. What the records do show is that the part of the tail not explained by queueing across packets is large at this service time: the decode floor plus the decode cost of the messages ahead in the same packet reproduces about three quarters of the observed p_{99} on four of the six streams (Section 7.6).

Large packets and the queue behind them. The message count of a packet is set by how closely transactions followed one another, not by how large any one transaction was: few transactions carry more than one message (Table 5), while a packet of ten or more messages is usually ten or more transactions that the publisher packed into one packet. Section 7.3 gives what such a packet costs this handler on ZN: the decode floor of about $7\mu\text{s}$ plus about $1\mu\text{s}$ for each further message. The largest packet seen, forty-five messages (Section 7.5), therefore holds the thread for about $50\mu\text{s}$, seven times the decode floor. That is the first cost: the last message of that packet waits over $40\mu\text{s}$ before its own decode. Yet the in-packet decode accounts for only a fifth of the $p_{99.9}$. The rest is the queue the packet opens behind it, the second cost. At one message per packet the service is about $7\mu\text{s}$, just under the publisher spacing measured on NQ (ZN’s own spacing was not measured), and a queue almost never forms; that is the empty-queue population of Table 28. While a forty-five-message packet holds the thread, every packet that arrives waits behind it, and each adds its own service to the wait of the ones behind it: the wait of the k -th packet in the queue is the sum of the service times ahead of it, so it grows with the number queued, not with any one packet’s size. ZN packets of ten or more messages are fewer than one in two hundred, but each is a burst of transactions that this handler pays for twice, once inside the packet and once in the queue behind it, and the hundreds of microseconds at $p_{99.9}$ are those queues.

Across the six streams. The same measurement covers three instruments and, on each, a book stream and a trade stream. Figure 29 puts the six serial tails side by side and Table 29 adds, for each stream, its packet rate, the size of its packets and the cost of each extra message. Three things are visible at once. First, the tail does not follow the packet rate: NQ book is by far the busiest stream and has the smallest tail, ZN book carries a third of its rate and a tail an order of magnitude larger, and the trade streams, twenty-five to fifty times quieter than the books, have larger tails than the books on every instrument. Second, on every instrument the trade stream carries more messages per packet than the book stream and has the larger tail: a trade is a transaction that touches several resting orders and is followed by the reaction to it, so it is where transactions arrive closest together and are coalesced most. Third, ZN, with the largest trade packets and the highest cost per extra message, has the largest tails on both streams; its book packets are not large at the 99th percentile, but at three times NQ’s cost per message and a maximum span of 45 the excursions they produce are the longest of any book stream. The clustering statistics are similar across the three instruments (the Fano-implied branching ceilings of Section 7.2 span 0.85–0.97 across all six streams, and the

NQ publisher period is assumed to hold on the others). What separates a twenty-fold spread of tails across streams with similar clustering statistics is not the arrival rate and not those statistics; it is how much of the clustering the publisher delivers inside a packet, the number of coalesced transactions, times the cost this handler pays for each, and the queue that opens behind the packet. On the live system the far tail is the clustering of transactions delivered as span.

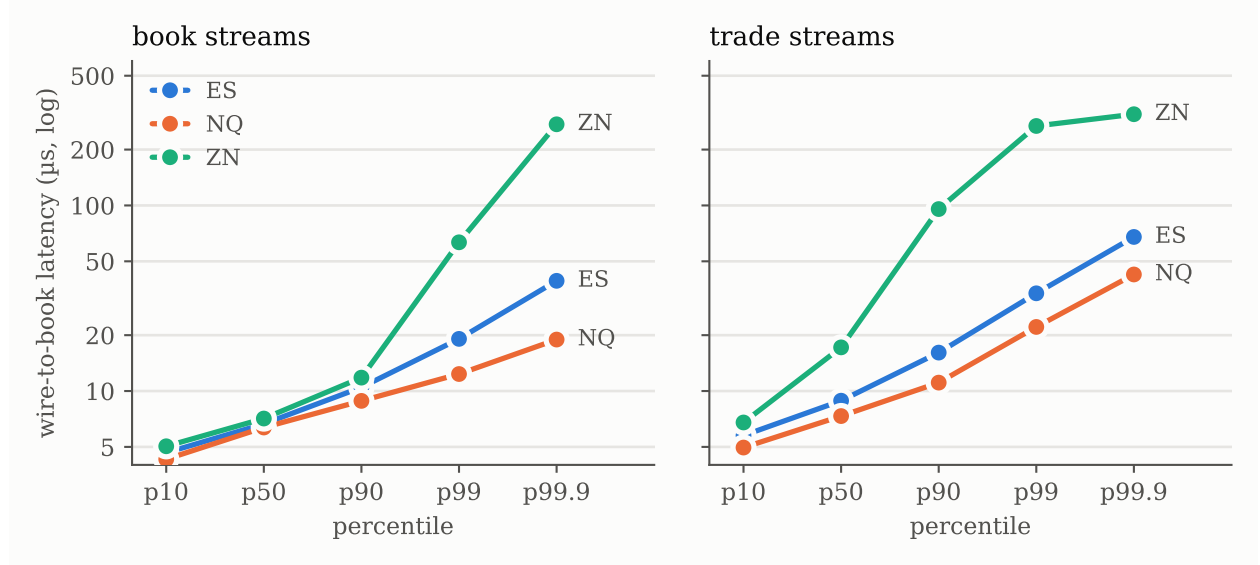


Figure 29. Serial wire-to-book latency by percentile on the six live streams, one 900 s window each, recovery excluded (book streams left, trade streams right, log latency axis). The tails order by messages per packet and cost per message, not by packet rate: the busiest stream (NQ book) has the smallest tail.

stream	packets	span p_{99}	$\mu\text{s}/\text{msg}$	p_{50}	p_{99}	$p_{99.9}$
NQ book	2,506,205	11	0.31	6.36	12.35	18.94
ES book	2,123,042	7	0.57	6.65	19.10	39.25
ZN book	824,536	6	0.97	7.11	63.30	273.69
ES trade	83,140	17	0.71	8.87	33.62	67.65
NQ trade	50,279	16	0.53	7.33	22.15	42.47
ZN trade	22,864	71	0.96	17.19	268.04	310.13

Table 29. The six streams of Figure 29, ordered by packet rate. Packets and span p_{99} are from the 53-minute capture of Section 7.5; the cost per extra message is the fitted slope of Section 7.3; the latency quantiles, in μs , are from the 900 s serial windows. Rate falls down the table while the tail rises; span p_{99} and cost per message, taken together, track the tail.

Consequences for the design rule. The live handler’s tail is the mechanism of Sections 4.3 and 7.5, seen from the receiver’s side. Section 7.5 measured that a packet’s decode time is the decode floor plus one slope for each message beyond the first, with packets of up to 45 messages on the book streams and 85 on the trade streams, and that such packets are coalesced transactions. Section 7.6 re-ran the sweep with that span-dependent service and found that the message count inflates the tail at service times at or just above the publisher period, where it supplies about two thirds of the p_{99} excess at $T = 8 \mu\text{s}$, adds a few percent above $16 \mu\text{s}$ where the queue across packets takes over, and leaves N^* unchanged; the two are one clustering delivered two ways, inside and across

datagrams. The ZN records put the production handler in the first regime: at a $7\mu\text{s}$ decode floor the typical packet queues behind nothing and the verdict of Section 9 for it (do not split at the publisher period) stands, while the far tail is paid on the coalesced packets, whose effective service is several times the decode floor. That is the regime in which the design rule of Section 4.6 admits a cut, and a cut would shorten the queue such a packet opens behind it but not the serialisation inside it; the span-aware sweep of Section 7.6, calibrated on NQ's slope, a third of ZN's, found N^* unchanged, so whether a cut pays on ZN is for the measurement to decide. The other lever is the per-message cost itself (Section 12), which scales the whole excursion. Which lever to pull is decided by the end-of-build measurement of Section 10.3.

Findings.

- *On ZN the far tail is large:* about $270\mu\text{s}$ at $p_{99.9}$, against about $26\mu\text{s}$ for messages with nothing queued and first in their packet.
- *Across the six live streams the tail follows packet size times cost per message, not packet rate:* the busiest stream, NQ book, has the smallest tail.
- *Long packets and occasional slow services each start a queue in the publisher's train;* how the ZN tail divides between them was not measured.
- *Messages with no queue still reach twice the decode floor at p_{99} ,* for reasons the records do not show.

8 Summary of Part II: findings and what they mean for HFT system design

Part II recorded the packet arrivals of NQ front-month on CME, ran a simulated receiver over them at a range of service times and stage counts, followed the same transactions upstream through the exchange's matching engine and publisher, and checked the results against a live production receiver. This section collects what it found and states, for each finding, what it means for the design of an HFT receiver. Part III turns these consequences into a design procedure for a concrete system. All results are for one instrument.

The arrival stream. Packets do not arrive at random. They come in clusters that a Hawkes process describes as near-critical, with a branching ratio of about 0.8 (Section 3.2). The clustering belongs to the matching engine's transactions, not to how the exchange packs them into packets (Section 4.3). At the engine, one transaction in six follows the previous one by less than s_{pub} , often by a fraction of a microsecond. The publisher does not pass that on: it is a queue that sends about one packet every s_{pub} , so a burst at the engine reaches the receiver as a train of packets one period apart, mostly one message each (Section 5). *For design:* a receiver cannot be sized from the average packet rate, which is hundreds of times too slow to matter. What it must survive is a train of packets at the publisher period, and that spacing, not the rate, is the number to measure on a feed.

What makes the receiver's tail. A single-threaded receiver queues only when two packets arrive closer together than its service time. Below the publisher period it has no queueing tail at all; above it, the tail grows much faster than the service time (Section 4.1). Up to about $32\mu\text{s}$ what sets it is how many gaps are shorter than the service time; beyond that it is how long the runs of such gaps last (Sections 4.2 and 4.3). The tail relative to the service time does not change with market

load or branching ratio in that range, because a busier market brings more bursts, not tighter ones (Section 4.9). *For design:* the question to ask of each stage is whether its service time is above or below the publisher period. The answer does not depend on the session being quiet or volatile, so it can be settled once per feed and does not need a live estimate of market conditions.

The production operating point. A production receiver runs at about $7\mu\text{s}$, just under the publisher period. There the queue across packets is small, and most of the tail comes instead from long packets, whose messages are decoded one after another while the packets behind them queue, and from service times that vary for reasons the records do not show (Sections 7.6 and 7.7). A one-message packet is served below the spacing and a packet of two or more messages above it (Section 7.3). *For design:* at this operating point the levers are the cost of each extra message and the spread of service times, not the number of threads. A stage cut does not shorten the decode of a long packet. What matters is the upper quantiles of per-packet service under load, measured on the real feed, compared with the publisher period.

What reduces the tail. Cutting the servicing chain into stages on separate threads costs one hop per stage on every message and shrinks the queueing tail, because only the slowest stage queues (Section 4.6). The tail appears between 7 and $8\mu\text{s}$; a cut starts to pay between 8 and $10\mu\text{s}$, once the tail is larger than the hop, and from $16\mu\text{s}$ it pays on every window (Sections 4.8 and 4.7). The best number of stages grows with the service time. For a stage without state, such as decode, a pool of cores that each take whole packets does as well as a cut at two cores and better from four, all costs charged (Section 4.4). A cheaper hop widens the range where cutting pays but does not move the threshold (Section 4.10). *For design:* cut a chain whose service time is above the split threshold, at the natural boundary that makes the slowest stage shortest; a cut that leaves the slowest stage as it was only adds a hop. Merge short stages onto one thread. Give a stateless stage a pool of cores instead of a cut when four or more cores are available; the order book, which carries state, can only be cut.

The theory and the live system. The two analytical results of Part I hold on every message of every window, and the Poisson stream behaves exactly as the theory predicts (Section 6). A live production receiver, on three instruments and a different clock, shows the same clustering (Section 7.2). *For design:* the design equation of Section 4.6 can be applied with confidence to a real pipeline: its terms are exact, and its inputs, the publisher spacing and the service time of each stage, can be measured directly.

Findings.

- *The receiver's tail threshold is the publisher period s_{pub} . Design:* measure it once per feed and compare every stage with it.
- *Above it, transaction timing sets the tail and only the slowest stage matters. Design:* cut chains above the split threshold of about $10\mu\text{s}$ at the boundary that shortens the slowest stage.
- *At a production receiver's decode floor, long packets and variable service carry most of the tail. Design:* reduce per-message cost and service spread; do not add threads.
- *A pool of whole-packet servers beats a cut from four cores for a stage without state. Design:* pool decode, cut the order book.
- *The threshold does not move with market load. Design:* no runtime estimate of market conditions is needed.

What Part II leaves open. Only NQ front-month was measured, and only its messages were counted in each packet; the full channel, ZN and other instruments are open (Section 14.1). Service time in the simulation is constant or grows with message count; its spread on a live receiver is not modelled. Why orders reach the matching engine within a microsecond of each other is a question about the market and is not answered here (Section 13.3). A designer applying these results to another feed should measure the publisher period and the per-packet service there rather than assume them.

Part III

Best practices: implications for HFT systems design

9 The reference instance: KASPAR-HFT and its fast_send pipeline

Part III applies the design consequences of Part II, collected in Section 8, to a concrete system. This section introduces the system that the rest of Part III works on, and applies the design rule of Section 4.6 to it: cut a chain only where the cut shortens its slowest stage, and never once that stage is already below the feed's publisher period, because below it there is no tail left to remove. It first says what KASPAR-HFT is and names the two ways its actors hand messages to one another, one synchronous and one queued, because the choice between those two at each boundary is exactly what the design rule decides. It then lays out how the production pipeline is actually wired: which boundaries are queued and why, and which stages run as one unbroken chain on a single thread. Finally it runs the design equation on that chain. On the chain as it stands, about seven microseconds long, the rule says do not split it. On a longer chain, sixteen microseconds, it says make one balanced cut and no more. The section after this one turns those verdicts into a procedure for changing the real system one boundary at a time and measuring the result.

The recommendations in this Part are stated for HFT systems in general and worked out on one system. KASPAR-HFT [36, 37] is an open-source C++20 trading system for CME futures built on a shared-nothing actor framework, available at <https://github.com/vincent212/kaspar-hft>:

every component (socket reader, feed arbitrator, SBE decoder, order book, strategy, order manager, iLink session) is an actor with private state and a mailbox, and components interact only by passing messages. It is at once a production system (MDP3 multicast in, full order books reconstructed order by order, authenticated iLink 3 sessions out) and, with the same actor code driven from recorded packet captures, a deterministic replay simulator in its own right. Part II does not run on that simulator: its measurements are made on a standalone tandem-Lindley model of the pipeline (Section 3.3), driven by packet-arrival timestamps extracted from the same class of capture. The framework supplies the architecture under study and the calibration of h ; it is not in the measurement loop. The **fast_send** paper documents the four extensions that make the actor model usable at HFT latencies: **fast_send** itself, a synchronous delivery in which the sending thread runs the receiver’s handler inline and takes the reply as a value; actor **Groups**, which co-schedule a set of actors on one thread behind a single mailbox; mailbox queue implementations selectable per actor; and a memory pool that removes per-message allocation and its tail. The defining property of **fast_send** is *receiver transparency*: a handler is written once and has no way to tell whether it was invoked synchronously or asynchronously, on which thread, or whether its sender is blocked. Section 10 builds on that property.

The **fast_send** paper [36] built and measured a production trading pipeline; that pipeline is the design this paper re-examines. It has three sections joined by exactly two cross-thread hops. Two socket-reader actors, one per redundant multicast channel, do nothing but drain their UDP sockets and hand datagrams to a buffer actor by an asynchronous **send**; that hop is asynchronous because a reader that blocked on its callee would drop packets under a burst. From the buffer, arbitration between the A and B feeds, SBE decode, order-book construction, and signal generation run as a single chain of **fast_sends** on one thread, with no queue and no context switch between stages. When the signal decides to act it hands the order to an order-management actor by a second asynchronous **send**, because the iLink write takes microseconds and blocking the decode thread on it would stall the market-data queue. The measured cost of each delivery path is given in Section 10. Against a servicing chain of $\sim 7\mu\text{s}$ measured tick-to-book on live ES, NQ, and ZN feeds, the framework’s own per-message contribution is under 1% of the decode floor, and the same measurement attributed the latency tail to the arrival stream rather than to the framework; the stream was strongly clustered, with a Fano-implied branching-ratio ceiling of 0.85–0.97. That observation motivated the present paper.

Read against Part II, this pipeline is the single-thread servicing chain of Section 1.2 realised with actors: the whole T sits on one thread between two hops that were placed where blocking was unacceptable. The present paper adds that the same logic applies inside the chain once the chain is long enough for the feed to queue behind it, and the recommendations that follow are stated as changes to this pipeline: where to cut it, in what order to place the stages, which boundaries to leave on **fast_send**, and how to decide each by measurement.

The design rule applied to this pipeline. The rule of Section 4.6, cut only where the cut shortens the slowest stage and never below the publisher period, can be run on the reference chain directly. It needs one input: the tail excess that a single stage of a given length s would carry, $\Delta_{\text{single}}(s)$. Table 11 supplies it. Section 4.6 showed that the excess depends only on the per-stage service T/N , so every entry of that table is the excess of one stage of length T/N , and read that way the table is a lookup: zero for a stage shorter than the publisher period, and growing fast above it.

Whatever the split of the $\sim 7\mu\text{s}$ chain across its natural boundaries (arbitration, SBE decode, book application, signal), every partition has its slowest stage at $7\mu\text{s}$ or less, just under the publisher period, where Part II finds no queueing tail in almost every window (Section 4.8). The chain has

no queueing excess to remove, every cut returns exactly its hop tax, and $N^* = 1$. The tail it does carry comes from the few packets that carry many messages: their decode one message after another, which a stage cut does not shorten, and the queue their long service opens behind them (Section 7.7).

The design rule applied to a longer chain. The rule applies once the chain exceeds the publisher period, for instance when a strategy stage is added, or on a feed with a shorter publisher period. Take a $16\mu\text{s}$ chain with the same four boundaries, decode and book now at $6\mu\text{s}$ each and the two small stages at $2\mu\text{s}$:

partition of the $16\mu\text{s}$ chain	s_{max} (μs)	hops	$\Delta_{\text{single}}(s_{\text{max}})$ (μs)	p_{99} excess + hop tax
single thread (16)	16	0	18.59	18.59
8 + 8 (arb+decode book+signal)	8	1	0.61	2.31
8 + 6 + 2 (three stages)	8	2	0.61	4.01
2 + 6 + 6 + 2 (all four split)	6	3	0.00	5.10

Every entry is read off Table 11 and the refined sweep of Section 4.8. Three points follow. The balanced two-way cut is the best option: it removes almost all of the tail excess for one hop, about an eightfold improvement in the objective on these illustrative stage times. Three stages do not lower s_{max} , since the $8\mu\text{s}$ stage remains the bottleneck, so the second hop costs $1.7\mu\text{s}$ and removes nothing (Remark 2.3, third consequence). Splitting all four lowers s_{max} to 6 and removes the small remaining excess at the cost of two more hops, and is worse. The recommendation for a chain of this length is one cut, placed so that the two halves are as equal as the natural boundaries allow, with the larger half at ingress (Remark 2.4). The numbers are a prior for the end-of-build measurement of Section 10.3, not a substitute for it. The per-stage service times are illustrative and are replaced by the operator’s profile of the chain.

10 Implementation on KASPAR-HFT: setting the send mode of each boundary

The practical claim of this paper is that on Hawkes-clustered feeds the choice of **fast_send** versus asynchronous **send** at each actor boundary is a late-stage optimisation decision, made per boundary from measured end-to-end median and tail latency of the assembled pipeline on the target feed, rather than an early architectural commitment. KASPAR-HFT supports this workflow. This section states the property that makes it possible and the procedure it enables.

The design equation maps onto framework choices in a specific way. The paper’s model of an N -stage tandem assumes each stage’s actor runs on its own thread and sends its output to the next stage through an asynchronous mailbox; the hop cost h is the wall-clock latency of that handoff.

Two hop paths in the shipping KASPAR-HFT framework. Of the delivery mechanisms described in Section 9, two matter for the design equation. The default asynchronous **send** uses **BQueue**, a blocking mailbox built on **std::mutex** + **std::condition_variable** + **boost::circular_buffer**; a producer thread enqueues under the mutex and a consumer thread waits on the condition variable. This is the hop path the paper’s Section 3.3 model corresponds to, and halving its measured $\sim 3.4\mu\text{s}$ round trip gives the one-way hop cost $h \approx 1.7\mu\text{s}$ used throughout Part II.

The **fast_send** path is different: it is a *synchronous* dispatch that acquires a per-actor mutex, calls the receiver’s handler inline on the sender’s thread, and returns the reply. There is no queue

and no cross-thread wake-up. Its per-call cost is much smaller than the BQueue path’s, but it is not a pipelined hop: the sender blocks until the receiver finishes, so a chain of `fast_sends` serialises through all stages on one thread and offers none of the throughput speedup of Proposition 1. The paper’s design equation does not apply to `fast_send` chains.

10.1 Receiver transparency of `fast_send` and `send`

Every actor in KASPAR-HFT registers its handlers through the `MESSAGE_HANDLER(MsgType, handler_method)` macro. A handler is a function of one message argument; nothing in its body can access, observe, or condition on the delivery mode by which it was invoked. This property is called *receiver transparency* in Maciejewski [36] and is what makes the send-mode choice a purely sender-side decision.

Concretely:

- **Synchronous delivery:** `target->fast_send(m, this)`. The sender’s thread acquires `target`’s per-actor mutex, calls the receiver’s handler inline on the sender’s thread, and takes the reply as a return value. No queue, no cross-thread wake-up. Round-trip cost ~ 30 ns per Table 4 of Maciejewski [36].
- **Asynchronous delivery:** `target->send(m, this)`. The message is enqueued on `target`’s BQueue mailbox (`mutex + condition_variable + boost::circular_buffer`); `target`’s own thread wakes up on the condition variable and runs the handler. Round-trip cost $\sim 3.4 \mu\text{s}$ across threads, or ~ 90 ns same-thread within a `Group`, per the same reference.

Both paths deliver the same handler with the same message. Neither the handler’s body nor any actor downstream of it can tell which path fired. Switching a boundary between the two is a one-line change at the sender’s call site — `target->fast_send(m, this)  $\leftrightarrow$  target->send(m, this)` — with no code change on the receiver side, no header change, no interface change, no test change.

10.2 End-to-end measurement of the boundary choice

The end-to-end latency profile of an HFT pipeline is a joint property of (i) which stages exist and where they are cut, (ii) which pairs of adjacent stages are joined by `fast_send` versus `send`, and (iii) the arrival law of the input feed. Proposition 1 and Theorem 1 both fold across multiple boundaries: the throughput speedup is N -fold and the tail contraction is $N^{-\gamma}$ with $\gamma \geq 1$, so the joint optimum is not deducible from local reasoning at a single boundary. A designer who reasons one boundary at a time underestimates the aggregate tail benefit of adding hops; this is the reasoning behind the single-thread convention of Section 1.2.

Getting the joint optimum therefore requires end-to-end measurement of the assembled pipeline against a representative feed, and iteration over per-boundary flips. The observable objective is a two-tuple (p_{50}, p_{99}) of end-to-end latency under the target feed, with an operator-chosen trade-off between the two (κ of Section 4.6). The design equation of Section 4.6, $N^* = \arg \min_N [(N - 1)h + \kappa \Delta(N)]$, or in its partition form the cut with the smallest reachable $s_{\max}$ and then the fewest hops, supplies a prior on where the optimum lies under a Hawkes feed; the measurement decides.

10.3 The measure-and-flip procedure for setting each boundary

The procedure is: draft the pipeline as a chain of actors at the natural cut points; wire each boundary with whichever of `fast_send` or `send` is convenient during development; once the pipeline runs end-to-end, measure the corpus-median p_{50} and p_{99} on a representative tape as a baseline; flip

one boundary at a time, re-measure, and accept the flip if (p_{50}, p_{99}) improves under the operator’s trade-off; iterate until no single flip improves the objective. The full design flow, including the steps that precede this, is given in Section 12.

Under the arrival law of a CME MDP3 feed the design equation of Section 4.6 predicts what the iteration converges to: `async send` at every boundary whose removal would raise the largest stage, and `fast_send` everywhere else, in particular between short sub-tasks below the publisher period of Section 4.6 and between sub-tasks downstream of the bottleneck whose combined service stays at or below $s_{\max}$ (Remark 2.3). The measurement decides; the prediction indicates where to expect it to converge.

10.4 Thread mapping and send mode as deployment decisions

A consequence of Section 10.3: in KASPAR-HFT two distinct topology decisions are not knowable at design time. The first is the *actor-to-thread mapping* — which actors share a thread by being placed in a common `Group`, and which run on their own thread. The second is the *per-boundary send-mode choice* — whether each pair of adjacent actors is joined by synchronous `fast_send` or asynchronous `send`. Neither decision is deducible from the source code alone. Both are functions of the target feed’s arrival statistics and the operator’s median-versus-tail trade-off, both of which are properties of the deployment environment. The optimum can only be measured from the assembled system running against a representative feed.

Under receiver transparency this is not a limitation. Actor code, message-type headers, and the full suite of unit and integration tests can be written and run before either the actor-to-thread mapping or the send-mode choice has been decided. Code construction and testing therefore proceed independently of the latency-optimisation consideration — both decisions are deferred to the end of the build, informed by measurements that could not have been taken any earlier because the system did not yet exist to measure. Correctness is established first; latency is optimised against the specific feed the system will run on, second. Two developers building two actors on the two sides of a boundary do not need to agree on whether their actors will ultimately live on the same thread or on different threads, nor on whether their handoff will be synchronous or asynchronous; they need to agree only on the message type. The message-type agreement is checked at compile time; the two topology decisions are parameters set at deployment.

This decoupling of correctness from latency optimisation is the practical consequence of receiver transparency, and distinguishes the design cycle from that of frameworks in which either topology decision enters the receiver’s interface (Section 10.5). Design proceeds without a commitment to a topology; the commitment is made once, near deployment, on the basis of measurements the code did not need to know about while it was being written.

10.5 Frameworks in which the send mode is part of the receiver’s type

Most low-latency actor frameworks bake the sender’s choice of delivery mode into the receiver’s type. An Akka actor’s `tell` versus `ask`, a CAF actor’s `send` versus `request`, and an Erlang process’s `cast` versus `call` each expose different receiver-side surfaces, so flipping a boundary from sync to async is a small refactor rather than a one-line change. Under those frameworks the design flow of Section 10.3 is not tractable, and the operator’s incentive is therefore to lock in the send-mode choice early — reasoning locally — rather than to defer it to an end-of-build measurement pass. This incentive is a plausible contributing cause of the single-thread convention: frameworks that penalise deferral push designers toward the topology that is least sensitive to a wrong early guess, which is the collapsed single-thread servicing chain.

KASPAR-HFT’s joint mailbox API of `fast_send` and `send`, with receiver transparency, is what makes measurement-driven send-mode optimisation practical. The design rule depends on this property operationally.

11 Pre-spinning receivers and pre-warming on self-exciting feeds

The paper’s central claim is that the single-thread hot-path recommendation does not hold above a service-time threshold on Hawkes-clustered feeds (Section 1.2, Section 4). Two adjacent conventions, pre-spinning receivers and pre-warming pipelines, are also affected by the arrival law, and this section examines them. Neither intervention is measured here; both are future work. The argument is that the arrival law provides much of the benefit these interventions are intended to deliver, on the tail quantile.

Pre-spinning receivers. The busy-poll recommendation of Rigtorp [41], Belay et al. [63], and the Aeron/Chronicle lineage (Section 1.2) pins a spinning thread per receiver so the wake path from a sleeping condition variable is never taken. On a Poisson feed, any arrival can find the receiver asleep, and the wake latency is paid on average; pre-spinning eliminates it. On a Hawkes feed with branching ratio $n \approx 0.8$, sleep/wake transitions happen only at cluster boundaries. Every p_{99} message is intra-cluster (Section 4.1: the p_{99} message waits behind a cluster, since at these service times the Poisson null produces no wait at all), so by the time a tail-setting message arrives the receiver is busy: at $8\text{--}16\mu\text{s}$ it is still serving the predecessor the publisher sent one spacing earlier, and from $64\mu\text{s}$ it has been processing a run for many multiples of the per-message service time (Section 4.2). In either case it is effectively spinning as a natural consequence of the load. Under the model used here, pre-spinning therefore saves little or no wake latency at the tail; this is a prediction of the queueing model, not a measurement of a production receiver’s state, and should be evaluated directly. It saves wake latency for the median and for the minimum latency (inter-cluster arrivals which do find the receiver asleep after seconds-scale gaps), but the saving is bounded by $\sim 1.4\mu\text{s}$ per hop (the $1.7\mu\text{s}$ wake-path cost less the few-hundred-nanosecond mutex and cache-line cost of Section 4.10) and is much smaller than the tail win from splitting. There is also an operational cost: in the present author’s production experience, pinning spinning threads across the servicing chain has caused overall system degradation — steady core burn at 100%, cache thrash between the spin loop and the actor’s own working set, and pressure on shared resources like the LLC and memory bandwidth — that offset the median-latency gains; this is anecdotal and not measured here.

Pre-warming. Pre-warming is a subset of the wake-up cost discussed above. The $h = 1.7\mu\text{s}$ one-way hop cost that this paper calibrates already captures the entire process-wake-up path on the receiver side: the kernel context switch, the reload of the receiver’s instruction cache, the L1/L2 misses on its actor state and message pool, the TLB refill, and the scheduler putting the thread back on a core. To the extent the thread hop is dominated by cluster-wait on Hawkes feeds — because every p_{99} arrival is intra-cluster and the receiver is, under the model, already awake — the cache-warmth components inside that h should follow the same pattern, since they are the same h decomposed. This is a consequence of the model rather than a measured result. Pre-warming interventions that fire synthetic traffic to keep instruction caches, data caches, TLBs, and branch predictors hot are therefore attacking the same h that Section 4 finds is dominated by cluster-wait. What such interventions can meaningfully deliver is confined to the median and the minimum — the inter-cluster first-arrival case where the receiver truly did cool down — and even there the delivery

is bounded by the wake-up cost itself at the cost of continuous L1/L2 pollution from the warmup traffic. Under Hawkes the tail is unaffected.

Measurement of both interventions. Pre-spinning and pre-warming are first-order tail-latency techniques when arrivals are uncorrelated. On self-exciting exchange feeds the arrival law itself provides the receiver-awake and cache-warm state on the tail quantile, so both interventions are second-order there and can be counterproductive once their steady costs are included. Both claims are corollaries of the paper’s main result rather than separate contributions, but each deserves a dedicated empirical evaluation on Hawkes-clustered feeds, analogous to the single-thread evaluation this paper carries out. The two open questions are:

- What is the actual ROI of pinned busy-poll receivers on the Hawkes-load $p_{50}, p_{99}, \text{max}$ triple, net of the steady core-burn cost and the cache pressure the spin loop imposes on the actor’s own working set?
- What is the actual ROI of periodic pre-warming traffic on the same triple, net of the L1/L2 pollution it creates and the residual advantage it delivers only on inter-cluster first-arrivals?

Both are direct extensions of the present paper’s methodology (measure under real CME MDP3 tapes; use a matched Poisson null to isolate the arrival-law dependence) and both are candidate follow-ons.

12 Recommendations for HFT system designers

Section 8 states, finding by finding, what Part II means for the design of a receiver. This section puts those consequences in the form of recommendations. The paper’s results qualify one architectural default (the single-thread hot path as an unconditional choice) and question two adjacent ones (busy-poll receivers, pre-warming). They do not contradict per-message code optimisation. This section separates what the paper leaves unchanged from what it argues against, and states the design flow the design equation implies.

Design principle (measure the split threshold on the target feed). The split threshold is the service time above which cutting a stage into two lowers its p_{99} . On NQ it lies between 9 and 10 μs (splitting wins on 67% of windows at 9 μs and 99.7% at 10, Section 4.8): just above the tail threshold of 7–8 μs , where a tail first appears, by the amount the tail needs to exceed the hop cost. Both are decided by how many packets arrive closer together than the stage’s service time, not by the packet rate or the branching ratio as such. That fraction is a property of the transaction stream’s gap distribution and of the publisher period. On CME MDP3 it is about 1% at the publisher period and does not vary with rate or with the fitted branching ratio at HFT service times, so a 7–8 μs stage is not split. On a feed that places tens of percent of its gaps at the publisher period at the same fitted branching ratio, the same stage is split (Appendix D). The split threshold cannot be read off fitted parameters. Measure the end-to-end latency of the assembled pipeline under the load the system will see, and split where the measured tail excess exceeds the hop cost.

12.1 Recommendations left unchanged

Per-message hot-path optimisation — branch-free decoders, cache-line-aligned actor state, atomic-free single-writer queues, SBE decode in place, memory pools, `fast_send` at ~ 30 ns, kernel-bypass

NICs (Solarflare Onload, DPDK, VMA), `isolcpus`, `SCHED_FIFO`, huge pages, RTC counters instead of `clock_gettime`, non-blocking IO on the socket, avoidance of allocation on the hot path, careful use of the branch predictor — remains necessary. All of it is contained in the service time T , which the paper takes as given. Proposition 1 gives an N -factor throughput speedup at any T , and the $1/N$ bound of Theorem 1 applies to a single-stage wait $\Delta(1)$ that itself scales with T , so the absolute gain from splitting grows with T while the hop cost does not. The splitting rule is therefore complementary to per-message optimisation: a two-stage tandem on a $20\mu\text{s}$ servicing chain has lower latency at every quantile than the same tandem on a $40\mu\text{s}$ chain, and the difference is entirely in the per-message layer; and per-message optimisation that brings a chain below the publisher period removes the case for splitting it at all (Section 4.8).

12.2 Recommendations the paper argues against

Two conventions in the layer above per-message optimisation. First, the single-thread servicing chain as the unconditional topology default (Section 1.2): on Hawkes-clustered feeds with T above the publisher period of Section 4.6, the tail is dominated by cluster wait and adding threads along the chain reduces it at a median cost of $(N - 1)h$. Second, pre-spinning receivers and pre-warming pipelines as always-on interventions (Section 11): on Hawkes-clustered feeds the receiver is already busy and warm at the arrival of every p_{99} message, so these interventions affect the median and minimum, not the tail, at a steady CPU and cache cost. The claim that a faster inner loop implies a single thread is preferable conflates two claims; the paper contradicts only the second.

12.3 Recommended design flow

The design flow that follows from the paper’s results, and that we recommend as the canonical HFT infrastructure workflow on self-exciting exchange feeds:

1. **Retain per-message optimisation.** This is orthogonal to the paper’s recommendation and determines absolute latency.
2. **Measure the target publisher period, and use it as the split/don’t-split test.** Take the corpus of packet arrivals, form the per-window interarrival-gap distribution, and read off a low quantile of it — we use p_1 . This single number is the service time at which a deterministic stage begins to accumulate a tail, and on NQ it is s_{pub} (Section 4.9). A stage whose service time sits comfortably below it will not develop a queue and must not be split: the split gains nothing and costs h at every quantile. A stage above it should be split, subject to the tail excess exceeding h , which on this corpus happens about $1.5\mu\text{s}$ further along (Section 4.8). Two cautions. First, use the tight end of the gap distribution, never the mean: on this corpus the mean gap is $2215\mu\text{s}$, some $300\times$ larger, and a rule stated on it would say never to split anything. Second, measure it under representative and peak-load conditions. On this corpus the publisher period is the service interval of the exchange’s market-data publisher, which spaces successive packets by at least that much while it drains a backlog (Section 5) — the packets on either side of it are distinct transactions (Section 4.3) — and it did not move between quiet and volatile sessions (Section 4.9), so it was measured once per feed; that invariance is a finding about this feed’s publisher, to be verified on another feed rather than assumed. At longer service times the tail does depend on intensity and branching ratio, and a chain in that regime needs its threshold measured under the load it will actually see.
3. **Draft the pipeline as a chain of actors at natural cut points, then balance it.** On a CME MDP3 handler: `SocketReader`, SBE decoder, order book, strategy actor, order

manager — four or five stages, no artificial decomposition (Section 10). The tail factor is $s_{\max}/T$ (Remark 2.3), so the goal of the decomposition is the smallest largest stage, and the largest stage belongs at ingress so that all queueing sits in one mailbox (Remark 2.4).

4. **Divide the stages above the publisher period by state.** A stage that carries no state from one packet to the next, such as SBE decode, can be served by a dispatch pool of whole-packet servers; with four or more cores for it, a pool is better than cutting it into the same number of stages, at two cores the two are level (Table 10). A stage that carries state, the order book above all, is served by cutting the chain.
5. **Wire boundaries with whichever of `fast_send` or `send` is convenient during development.** Receiver transparency in KASPAR-HFT makes this a one-line-per-boundary decision that can be deferred (Section 10).
6. **Do not commit to an actor-to-thread mapping at design time.** Grouping decisions and per-boundary send-mode decisions are both scalar dials at deployment, not code-structure decisions (Section 10.4).
7. **Measure the assembled pipeline against a representative feed.** Corpus-median p_{50} and p_{99} of end-to-end latency, per operator-chosen trade-off (Section 10.3).
8. **Flip one boundary at a time and re-measure.** Accept the flip if (p_{50}, p_{99}) improves. Iterate until no single-boundary flip helps.
9. **Prior on where step 7 converges.** On Hawkes-clustered feeds, async `send` at every boundary whose removal would raise the largest stage; `fast_send` at every other boundary, in particular between sub-tasks downstream of the bottleneck whose combined service stays at or below $s_{\max}$, and between short sub-tasks below the publisher period. A hop that does not lower $s_{\max}$ costs h at every quantile and compresses nothing.
10. **Do not pre-spin or pre-warm by default.** If pre-spinning or pre-warming is used, measure the ROI on a Hawkes-representative tape, net of the steady core-burn and cache-pollution costs. If the measurement improves the tail materially, keep the intervention. The paper’s prior is that on Hawkes feeds it will not (Section 11).

The workflow prioritises measurement over local reasoning because the arrival law of the target feed is a first-order determinant of the optimum and is not a property of the code.

Part IV

Conclusions

13 Conclusion

This paper is about queueing in the software that receives market data: what makes a single-threaded receiver’s latency tail, and how to reduce it. It is not about why the market behaves as it does. Section 13.1 gives the answer in one place; the remaining subsections state what the paper establishes about the cause of the tail, what it does not, how to reduce the tail, what it revises in a prior design context, and the caveats. All empirical results are for one instrument, NQ front-month on CME.

13.1 Summary: what causes the receiver’s tail and how to reduce it

What causes the tail. The chain starts at the exchange. The matching engine often processes one transaction less than a microsecond after the previous one (Table 13). The market-data publisher cannot send that fast. It is a queue with service time s_{pub} per packet, so a burst at the engine leaves it as a train of packets at that spacing, mostly one message each, with an occasional larger packet after a longer gap (Tables 14 and 17). The receiver is the next queue in line. If it handles a packet faster than the publisher sends them, it keeps up with any train and builds no queue from the trains alone. If it is slower, it falls behind by the difference on every packet of the train, and the messages at the end of the train wait. At service times a little above the publisher period the wait is mostly behind the packet or two just ahead; at much longer service times it is behind runs of transactions spread over a few hundred microseconds, which is the self-excitation a Hawkes fit measures (Table 4, Section 5.7).

A production receiver runs just under the publisher period, at about $7\mu\text{s}$, and there two further mechanisms carry most of its tail. The first is long packets. Decode cost grows with the number of messages, so a packet with two or more messages takes longer than the publisher period; its later messages wait for the earlier ones, and the packets arriving behind it queue while it holds the thread. In the simulation at that service time the decode inside packets supplies about two thirds of the tail, and on the live receiver the decode cost of a message’s position in its packet reproduces most of the observed p_{99} (Section 7.6); on ZN, where each extra message costs more, a few long packets set the far tail (Section 7.7). The second is service time that varies from packet to packet: on the live ZN receiver, messages with nothing queued ahead of them and first in their packet still take about twice the decode floor at p_{99} , and any such service in a train can start a queue. The simulation models the first mechanism only through NQ’s cost per message and the second not at all, so its tails at the production service time are lower than a real receiver’s. The packet rate does not cause the tail at HFT service times: utilisation there is tiny (Section 4.9). Why orders reach the engine within a microsecond of each other is a question about the market and is left open (Section 13.3).

How to reduce it: split serially. Cut the servicing chain into stages on separate threads, each passing its output to the next through a mailbox hop. Every message pays one hop per boundary at the median, and the queueing excess shrinks by at least the number of stages, because only the slowest stage queues: the tail is set by the service time of the largest stage (Theorem 1). A stage below the publisher spacing does not queue at all. On NQ, one cut into two halves removes most of the tail in every window at every service time well above the spacing; the best number of stages grows with the service time, limited by the natural cut points in the chain (Section 13.4). Serial

splitting is the only option for a stage that carries state from one message to the next, such as the order book.

How to reduce it: split in parallel. Give whole packets to a pool of cores, each running the full stage, and put them back in order on the way out. With the hop in, the hop out and the resequencing all charged, the median costs two hops whatever the pool size, and almost no packet waits to be put back in order (Table 10). A pool of two cores is level with a two-stage serial cut; a pool of four or more is better than a serial cut into as many stages (three cores were not simulated). It works only for a stage without state, such as SBE decode. A design that decodes in a pool and feeds one order-book thread combines the two and has not been costed here (Section 14).

Design rule (NQ front-month on CME). Measure the service time of the servicing chain per packet under real market load, including multi-message packets and bursts, and compare its upper quantiles, not its median, with the publisher period (about $7.5\ \mu\text{s}$).

- Upper quantiles below the spacing: keep one thread. Splitting only adds hop cost.
- Median below the spacing but upper quantiles above it, the regime of a production receiver: the tail comes from long packets and slow services. Reduce the per-message decode cost and the spread of service times; a stage cut does not shorten the decode of a long packet.
- Typical service above the spacing, stage carries state: split serially, at the natural boundary that minimises the largest stage, with more stages the longer the service time.
- Typical service above the spacing, stage carries no state, four or more cores available: dispatch whole packets to a pool and resequence.

On another feed, measure its publisher period first; the threshold is that spacing.

13.2 What causes the tail: what the paper establishes

The questions were what produces the queueing tail behind a single-threaded receiver — the packet rate, the clustering of arrivals, or the number of messages per packet — and are two queues better than one. The first is answered by controlled simulation on the real arrival times: each counterfactual stream of Section 3.6 removes one property of the stream and keeps the rest, so the change in the tail measures that property's contribution. The following hold on NQ.

1. *The tail is produced by the timing of matching-engine transactions and, at production service times, by the number of messages per packet.* Putting transactions at random moments removes most of the tail (Table 8); the message count is claim 5.
2. *Not by the rate.* Utilisation is tiny at every service time an HFT hot path occupies, and the busiest windows have the same tail, relative to the service time, as the quietest (Section 4.9); the rate matters only at service times far above the publisher period, where ordinary utilisation queueing begins. A stream that keeps every second's packet count and places packets at random within the second has no tail at HFT service times (Table 4).

3. *Not by the exchange's packetisation.* Almost every transaction fits in one packet, the pieces of a split one arrive well apart, and the short gaps lie between different transactions; merging each transaction into one packet or spreading its packets leaves the tail about as it was (Section 4.3).
4. *Not by how sizes are ordered in time.* Decoupling transaction sizes from their timing, or shuffling packet sizes across packets, leaves the tail about as it was at service times above the spacing (Table 8, Section 7.6). This says the order in which long packets arrive does not matter; it does not say their length does not matter, which is claim 5.
5. *At the service time production receivers run at, long packets carry most of the tail.* Decode time grows with the number of messages, so a long packet delays its own later messages and holds the thread while the packets behind it queue. At that service time this is about two thirds of the simulated tail, and on the live receiver the decode cost of a message's position in its packet reproduces most of the observed p_{99} ; it sets the live ZN far tail (Sections 7.6 and 7.7). At service times well above the spacing it adds a few percent. Most multi-message packets hold several transactions packed together by the publisher (Section 7.5).
6. *The timing acts through two channels whose weight depends on the service time.* At short service times it acts through consecutive transactions sent back to back at the publisher period, most of which the engine processed almost together (Table 14): a stream with the real gaps in random order keeps most of the tail there. At long service times it acts through runs of transactions, which the same shuffle breaks up, keeping little of the tail (Table 4). The runs are the self-exciting component of the transaction stream, which a Hawkes fit measures (Table 7).
7. *The shortest gap the receiver sees, and the pile-up of gaps just above it, are the exchange publisher's output.* On the matching engine's clock one consecutive transaction in six follows the previous one within the publisher period, against about one packet in a hundred on the publisher's clock (Table 13), and the delay from engine to packet grows about tenfold during engine bursts (Table 15). The publisher is a queue sending about one packet per s_{pub} , and the receiver is the second stage of a tandem that begins at the exchange (Section 5.6).
8. *The results do not depend on the transaction grouping.* Regrouping by exact `transactTime`, checked against the end-of-event flag, leaves them unchanged (Section 5.2), and session-bootstrap intervals on the key shares are a few points wide (Tables 4 and 8).
9. *The arrival statistics are not an artefact of replay.* A live production receiver, on other instruments and a different clock, gives the same clustering statistics, and its decode floor coincides with the service time at which the simulated tail appears (Section 7).

13.3 What causes the tail: what the paper does not establish

Why the matching engine bursts. The paper traces the receiver's short gaps to the exchange's publisher draining a backlog, and the backlog to the matching engine processing events a fraction of a microsecond apart (Section 5). It does not show why orders reach the engine so close together. They do not reach the receiver together: when the engine processes two transactions almost at once, the publisher nearly always sends them in separate packets at its own spacing (Table 14). The open question is about the engine's input. Three mechanisms remain open. (a) Reaction: a transaction causes other participants to cancel, replace or trade within microseconds, each as a separate transaction. (b) Common reaction: many participants respond to the same public signal at once, their orders reach the matching engine together, and the engine processes them one after

another. (c) Pacing or batching inside the exchange’s order entry, with no market information in the spacing. Separating them needs the time each order *entered* the exchange, not only when it was processed and published: (a) predicts that the second order entered after the first transaction was published, (b) that both entered before either was published. The public MDP3 feed carries processing and publication times only, and the paper’s own order-entry sessions record only its own orders.

The nearest published evidence is Aquilina et al. [33], who had message-level data from the London Stock Exchange with participants’ order-entry times and found that *races* — several participants sending messages at the same price level in response to the same public signal, within microseconds of each other — are frequent and account for a material share of trading volume. That is mechanism (b). The sub-microsecond engine-clock gaps are what (b) would produce, and it is likely that the same mechanism is at play on CME; the paper has no data that test it, and states it as a hypothesis.

Whether the publisher’s capacity is designed. The publisher behaves as a queue with a capacity of one packet per publisher period; whether that capacity is a design choice or an engineering limit, and whether it acts as a speed limit on the information side of the market, is stated as a conjecture in Section 5.8, with its qualifications.

Self-excitation at HFT service times. It is not the cause there, and this is now shown rather than argued. The fitted self-excitation acts over about $160\mu\text{s}$ and places only a few percent of its triggering within the publisher spacing (Section 5.7); it is the cause of the runs, which carry the tail at long service times. At short service times the carrier is the publisher draining, at its own spacing, backlogs built by orders reaching the engine within a microsecond of each other, whose origin is the open question above.

Events below the publisher period. Two transactions closer together than the publisher’s service interval reach the receiver in one packet, so every gap statistic on the publisher’s clock is censored there. On the engine’s clock they are visible (Table 13), and on the publisher’s clock neither the message count per packet nor the share of packets carrying several transactions rises with the fitted branching ratio (Section 4.9), so no faster self-excitation scaling with the branching ratio is hidden in the packets. Faster self-excitation of constant strength would not be seen by that test.

Why a receiver’s service time varies. On the live ZN receiver, messages that met an empty queue and were first in their packet, so that neither queueing nor in-packet decode contributes, still take about twice the decode floor at p_{99} and several times it at $p_{99.9}$ (Table 28). The records do not show why. Because packets arrive at the publisher period, each such service can start a queue, so this spread is a source of tail in its own right, and the simulation, whose service is constant or grows with message count, does not contain it.

How the live far tail divides. The ZN far tail is argued to be long packets and the queues behind them, from the measured decode floor and cost per message (Section 7.7). How it divides between the decode inside long packets, the queue behind them, and slow services was not measured, because the live per-message records were not retained.

Instruments other than NQ. One instrument was measured. On ZN the live data show the message count per packet rising with queue depth, where on NQ it falls, and whether the two channels compound there has not been tested; no ZN corpus exists. The paper also treats NQ

front-month as a stream on its own, while the publisher serves the whole channel. On NQ the other instruments are thin; on ZN the Treasury futures along the yield curve are likely to excite one another, and where they share a channel the publisher would carry those reactions in the same packets. Cross-excitation between instruments is a candidate cause of ZN’s large packets that the paper does not test (Section 14.1).

13.4 How to reduce the tail

The second question was whether a servicing chain cut into two or more stages on separate threads is better than one thread. Above a service-time threshold, yes; below it, no; and the threshold is a property of the feed, measured here on NQ. Above the publisher period the single-thread tail is many times the hop cost, so the hop is small against what it lets the pipeline remove; below it the hop is cost only. Cutting a task into two halves removes most of the tail for one hop on the median, on every window, at every service time well above the spacing, and it loses at service times at or below the spacing (Table 3). A tandem of N stages divides the tail by N or better on every sample path (Theorem 1), in practice by far more where a cut takes each stage below the publisher period, while paying one hop per boundary at every quantile; only its largest stage matters. The best stage count rises with the service time, up to the number of natural cut points in the decode chain. These results do not depend on what places transactions close together: the design rule needs the gaps the receiver sees, not their cause.

The single-thread rule is correct whenever the service time of the chain is below the feed’s threshold, and on this corpus that includes the reference system’s socket-to-book path, at about $7\ \mu\text{s}$, which should not be split (Section 9). The threshold is the service time of the exchange’s publisher upstream of the receiver, which is the tight end of the gap distribution the receiver sees, and it does not move with packet rate or fitted branching ratio at HFT service times; at longer service times both raise the tail (Section 4.9). Appendix D constructs a feed on which both dependences appear at a production service time and on which splitting pays where on CME it does not. On another feed the threshold, and its dependence on load and on clustering, must be measured.

Spending the same cores as a dispatch pool that sends whole packets to identical servers, with the ingress hop, the egress hop and in-order resequencing all charged, costs two hops on the median whatever the pool size and resequences almost nothing (Table 10). For a stage that carries no state, such as SBE decode, a pool of two cores is level with a two-stage cut, one hop dearer on the median and about the same in the tail; a pool of four or more cores is better than a cut into as many stages on both (three cores were not simulated). The order-book stage carries state from one update to the next and cannot be dispatched; for it, cutting the chain is the way to spend cores. The design rule is therefore: cut stateful chains above the threshold; dispatch stateless stages above it when four or more cores are available.

The recommendation is that HFT designers measure end-to-end latency of the assembled pipeline under actual market conditions, including peak load, and split the servicing chain wherever the measurement shows a tail excess larger than the hop cost. Splitting the hot path is a design option of equal standing with optimising it. For a chain above the threshold, one cut at a natural boundary reduces the tail by more than any per-message optimisation is likely to: on this corpus a balanced cut of a chain twice the publisher spacing removes almost all of its tail excess for one hop (Section 9), whereas per-message work moves the decode floor, and the cost per message, by tenths of a microsecond. Below the threshold, for the typical packet, per-message optimisation is the lever; the far tail that a few many-message packets open on a live channel (Section 7.7) is a separate regime, whose remedy the end-of-build measurement decides. Framework designers should measure busy-poll dequeue on pinned isolated cores against the wait/notify mailbox on a representative tape rather

than assume it is faster (Section 11), drive the mailbox mutex uncontended, keep memory-pool discipline, and cut the decode pipeline at whichever of its natural boundaries minimises the largest stage, with that stage at ingress and everything downstream merged up to its service time; the tail is set by the largest stage, and a hop that does not lower it is pure cost. Strategy designers of both percentage-of-volume (POV) and race-sensitive algorithms benefit for the same reason.

13.5 Revision of a prior design context

An earlier paper by the present author [36] did not itself argue for the single-thread servicing chain; it took the collapsed single-thread servicing chain as the widely accepted design context of the time and offered **fast_send** as the mechanism that makes actor composition inside it free of context-switch overhead. What the present paper retracts is the design context, not the mechanism: on Hawkes-clustered feeds the single-thread servicing chain should no longer be the unconditional default. The corrected position is that **fast_send** (synchronous, inline, sub-microsecond) and asynchronous **BQueue** handoffs (a real pipelined hop at a few microseconds) are *complementary* tools, and the choice between them is a per-boundary design decision that depends on the service time being chained and the arrival law of the feed the pipeline sits on.

In shipping KASPAR-HFT the **fast_send** versus asynchronous-send choice at each actor boundary is normally left open until the end of the build, and is resolved by measuring the end-to-end median and tail latency of the assembled pipeline under the target feed. A convenient property of the KASPAR-HFT actor API is that the receiving actor’s **MESSAGE_HANDLER** is agnostic to how it was invoked: swapping a synchronous **fast_send** at a call site for an asynchronous **send** (or vice versa) is a one-line change on the sender side and requires no code change on the receiver side. The choice therefore lives entirely at each individual call site and can be flipped per boundary at end-of-build with minimal engineering cost. The paper’s design equation now supplies the arrival-law-aware default value of that end-of-build measurement: on Hawkes-clustered feeds, tasks above the threshold should be split into shorter actors joined by asynchronous sends, on the measured result that splitting reduces the tail by more than the hop cost raises the median. The rule presupposes that the task being split has a service time comfortably above the publisher period of Section 4.6; a task no longer than a hop cannot be usefully split. Short synchronous chains inside one actor’s servicing of one message therefore continue to use **fast_send** unchanged. The end-of-build measurement decides any specific boundary.

13.6 Caveats

The corpus is a single instrument, NQ front-month; a ZN recording is the first replication needed, for the reason given in Section 13.3, ahead of ES and BTC. Transactions are grouped by exact **transactTime**, checked against the end-of-event flag (Section 5.2). The live measurements of Sections 7 and 7.7 come from one host on one afternoon, without replication, and the corpus windows are nested in sessions, so uncertainty on the aggregates is session-clustered; the headline survival shares carry session-bootstrap intervals. Constant per-packet service is a simplification, and at the production service time it is the one that matters most. Section 7.6 removes it for message count, at NQ’s cost per message, which is lower than ZN’s, and finds the stage count unchanged; service that varies for other reasons, which the live receiver shows at twice the decode floor at p_{99} with no queue, is not modelled, so the simulated tails at the production service time understate a real receiver’s. The cache-pollution cost of spinning receivers on many cores is bounded but not measured. And Theorem 1 is a bound whose slack depends on the arrival law; the paper measures that slack rather than deriving it, and a closed form for the slack on Hawkes input is open.

None of these caveats reverses the two answers on this corpus.

14 Future work

The section first states the open questions the paper raises, then lists the remaining work in two groups: questions about the market, which are outside the paper’s scope, and questions about the receiver, which are within it but were not answered with the data at hand.

14.1 Open questions this paper raises

The paper answers its two questions on NQ front-month. Measuring it raised others that need data or scope beyond this paper, and each is better answered in a shorter, focused study.

1. **Does filtering the channel to NQ front-month change any result?** Channel 318 carries other instruments, and the publisher packs their messages into the same packets as NQ. Every packet statistic in the paper counts NQ front-month messages only: span, transactions per packet, and the packet stream fed to the simulated receiver. Two things could change on the full channel. First, packets are larger in bytes and messages than the NQ span shows, which bears on the span-dependent service of Section 7.6 and the tables of Section 5.5. Second, a receiver subscribed to the channel sees every packet, including those without NQ, so its arrival stream is denser than the one simulated unless those packets are discarded at negligible cost. The raw captures hold the full channel; the measurement is packet bytes and message counts on every packet, joined to the NQ tapes on the publisher’s send time, and the sweep rerun with every channel packet as an arrival. The publisher period is not in question: Table 17 shows it on the channel’s order and trade packets.
2. **Do instruments excite each other, above all on ZN?** The Treasury futures along the yield curve (two-, five-, ten- and thirty-year and the ultra bond) are priced off one another, so a move in one is likely to trigger quotes and trades in the others within microseconds. Where they share ZN’s channel, those reactions reach the publisher together and leave in the same packets, which would make ZN’s large packets a product of the curve and not of ZN alone. The test is a multivariate Hawkes fit across the channel’s instruments on the engine clock and packet size conditioned on cross-triggering. It needs a full-channel ZN recording.
3. **Why do orders reach the matching engine within a microsecond of each other?** The receiver sees them one publisher period apart; the question is about the engine’s input. Reaction, common reaction to one signal (races), or pacing inside order entry (Section 13.3). It needs order-entry times for all participants.
4. **How much of the engine’s activity is racing?** Table 19 shows the race signature in tight gaps that follow a trade, but rejected race orders do not reach the public feed.
5. **Is the publisher’s capacity a design choice, and does it act as a speed limit on information?** The conjecture of Section 5.8.
6. **Where does the spread in a receiver’s service time come from?** On the live ZN receiver the hot path reaches twice the decode floor at p_{99} with nothing queued and nothing ahead in the packet. The measurement is the full distribution of per-packet service for single-message packets, with the candidate causes (cache misses, interrupts, rare message templates, book state) recorded alongside.

7. **What does a random service time do to the tail and the design rule?** The sweep rerun with service drawn from the measured live distribution instead of a constant, and the span-aware sweep rerun at ZN's per-message cost, three times NQ's.
8. **How does the live far tail divide?** Retaining the per-message live records would allow the ZN $p_{99.9}$ to be split into in-packet decode, the queue behind long packets, and slow services.
9. **Can decoding one packet's messages in parallel shorten the long-packet tail?** A stage cut does not shorten it. An unmerged experiment in the KASPAR-HFT repository that dispatched every message to parallel decoders was slower than serial at every book percentile; a later one that sent only multi-message packets to parallel decoders lost at the median but improved $p_{99.9}$ on five of six streams, on a build with recovery disabled and without validation of its output. Whether it pays in a production build is open.
10. **Does the result replicate on ZN, ES and BTC?** The design rule is measured on NQ; the threshold equals the upstream publisher's service time only if each channel is paced the same way.

The lists below give the remaining work in two groups.

Why orders reach the matching engine within microseconds of each other (questions about the market).

- What places two distinct transactions within a few microseconds of each other: reaction to the previous transaction, common reaction of many participants to the same signal, or pacing by the gateway or matching engine (Section 13.3). Separating them needs order-entry times for all participants, the data Aquilina et al. [33] had for the London Stock Exchange. Races of the kind they found are the leading hypothesis. Their data were exchange message data, which, unlike the order-book data of a public feed, include the attempts to trade or cancel that failed, so both the winner and the losers of a race can be seen. On FTSE 100 stocks they found that latency-arbitrage races, in which several firms react to the same public signal and try to take or cancel the same stale quote, are frequent (about one per minute per symbol), extremely fast (the modal race lasts five to ten microseconds) and account for about a fifth of trading volume; that the top six firms account for over 80% of race wins and losses; that races make up roughly a third of the cost of liquidity as measured by price impact and the effective spread; and that a market design removing them would lower that cost by about 17%. The timescale of their races is the timescale of the engine-clock gaps of Table 13, and the failed attempts that reveal a race are what the CME public feed does not carry (Section 5.7).
- Which instruments' transactions sit on either side of a tight gap on the engine clock, across the whole channel. Table 19 gives message type, side and price level for NQ front-month; which other instruments' transactions sit within microseconds of an NQ transaction is not measured.
- Cross-excitation between instruments, above all on ZN (Section 14.1, question 2).
- A power-law Hawkes kernel against the exponential, fitted on the engine clock.
- The publisher queue measured channel-wide rather than on NQ front-month only, and on ES and ZN, so that its service interval and backlog are measured under the full load it serves.

The queue in the receiver.

- A ZN recording (pcap and per-message records) and the full sweep on it, with span-dependent service. On ZN the live data suggest that large packets arrive during queues, so the queueing and serialisation channels may compound, which would add a span term to the design rule. This is the first replication; ES and BTC follow.
- Dispatch under state: a design that dispatches the stateless decode stage and feeds a single order-book stage, costed end to end.
- Closed form for the slack in Theorem 1 under Hawkes input, i.e. the exponent $\gamma - 1$ as a function of $(\rho, n, \beta T)$. By the reduction this is a single-server question about $M^{[K]}/D/1$ with Borel batches at two service levels, which is tractable; the paper measures γ instead.
- Load-conditioned hop cost $h(\rho)$. This paper calibrates $h = 1.7 \mu\text{s}$ from the low-load wake-latency figure in Maciejewski [36]; under sustained-load conditions inside a cluster the receiver never sleeps and the effective h collapses toward the mutex + cache-line-transfer cost of a few hundred nanoseconds (Section 4.10). Measuring $h(\rho)$ directly and rerunning the sweep against it would sharpen the reported tail reduction, which the single- h simulation understates.
- Cache-pollution cost per spinning-receiver stage count, measured.
- Overnight-session regime.
- Own-capture pcap validation of the network sub-stage.

15 Reproducibility

The code for this paper is in the KASPAR-HFT repository [37], <https://github.com/vincent212/kaspar-hft>.

References

- [1] Hawkes, A. G. (1971). Spectra of some self-exciting and mutually exciting point processes. *Biometrika* 58, 83–90.
- [2] Hawkes, A. G., Oakes, D. (1974). A cluster process representation of a self-exciting process. *Journal of Applied Probability* 11(3), 493–503.
- [3] Tanner, J. C. (1961). A derivation of the Borel distribution. *Biometrika* 48(1–2), 222–224.
- [4] Asmussen, S. (2003). *Applied Probability and Queues*, 2nd ed. Springer. Chapter X (heavy-tailed and subexponential asymptotics for single-server queues).
- [5] Gao, X., Zhu, L. (2018). Functional central limit theorems for stationary Hawkes processes and application to infinite-server queues. *Queueing Systems* 90, 161–206.
- [6] Jackson, J. R. (1957). Networks of waiting lines. *Operations Research* 5, 518–521.
- [7] Burke, P. J. (1956). The output of a queuing system. *Operations Research* 4, 699–704.
- [8] Kelly, F. P. (1979). *Reversibility and Stochastic Networks*. Wiley.

- [9] Kleinrock, L. (1976). *Queueing Systems, Vol. II: Computer Applications*. Wiley.
- [10] Whitt, W. (1983). The queueing network analyzer. *Bell System Technical Journal* 62, 2779–2815.
- [11] Friedman, H. D. (1965). Reduction methods for tandem queueing systems. *Operations Research* 13(1), 121–131.
- [12] Lindley, D. V. (1952). The theory of queues with a single server. *Mathematical Proceedings of the Cambridge Philosophical Society* 48(2), 277–289.
- [13] Foss, S., Korshunov, D. (2000). Steady-state asymptotics for tandem, split-match and other feedforward queues with heavy-tailed service. *Queueing Systems* 35, 65–80.
- [14] Baccelli, F., Foss, S., Lelarge, M. (2005). Tail asymptotics for monotone-separable networks. *J. Appl. Probability* 42(3), 793–812.
- [15] Heindl, A. (2001). Decomposition of general tandem queueing networks with MMPP input. *Performance Evaluation* 44(1–4), 5–23.
- [16] Kim, Y. H., Chydzinski, A. (2002). Connection-wise end-to-end performance analysis of queueing networks with MMPP inputs. *Performance Evaluation* 47(2–3), 137–156.
- [17] Dębicki, K., Mandjes, M. (2015). *Queues and Lévy Fluctuation Theory*. Springer.
- [18] Daw, A., Pender, J. (2018). Queues driven by Hawkes processes. *Stochastic Systems* 8(3), 192–229.
- [19] Koops, D. T., Boxma, O. J., Mandjes, M. R. H. (2018). Infinite-server queues with Hawkes arrivals. *Queueing Systems* 92, 27–52.
- [20] Chen, X., Blanchet, J. (2021). Perfect sampling of Hawkes processes and queues with Hawkes arrivals. *Stochastic Systems* 11(3), 264–283.
- [21] Sheldon, D., et al. (2023). Steady-state analysis and online learning for queues with Hawkes arrivals. arXiv 2311.02577.
- [22] Gao, X., Zhu, L. (2024). Single-server queues with state-dependent Hawkes arrivals. *Mathematics of Operations Research*.
- [23] Ozaki, T. (1979). Maximum likelihood estimation of Hawkes’ self-exciting point processes. *Ann. Inst. Statist. Math.* 31(1), 145–155.
- [24] Filimonov, V., Sornette, D. (2012). Quantifying reflexivity in financial markets: toward a prediction of flash crashes. *Physical Review E* 85, 056108.
- [25] Filimonov, V., Sornette, D. (2015). Apparent criticality and calibration issues in the Hawkes self-excited point process model: application to high-frequency financial data. *Quantitative Finance* 15(8), 1293–1314.
- [26] Achab, M., Bacry, E., Gaïffas, S., Mastromatteo, I., Muzy, J.-F. (2018). Uncovering causality from multivariate Hawkes integrated cumulants; analysis of order book flows using a nonparametric estimation of the branching ratio matrix. *Journal of Machine Learning Research* 18(192), 1–28; arXiv 1706.03411.

- [27] Hardiman, S. J., Bercot, N., Bouchaud, J.-P. (2013). Critical reflexivity in financial markets: a Hawkes process analysis. arXiv 1302.1405.
- [28] Bacry, E., Mastromatteo, I., Muzy, J.-F. (2015). Hawkes processes in finance. *Market Microstructure and Liquidity* 1(1).
- [29] Budish, E., Cramton, P., Shim, J. (2015). The high-frequency trading arms race: frequent batch auctions as a market design response. *Quarterly Journal of Economics* 130(4), 1547–1621.
- [30] Yoon, J. (2026). The world’s fastest matching engine algorithm. arXiv 2606.01183.
- [31] Byrd, D., Hybinette, M., Balch, T. H. (2020). ABIDES: Towards high-fidelity market simulation for AI research.
- [32] Frey, S., Li, K., Nagy, P., Sapora, S., Lu, C., Zohren, S., Foerster, J., Calinescu, A. (2023). JAX-LOB. arXiv 2308.13289.
- [33] Aquilina, M., Budish, E., O’Neill, P. (2022). Quantifying the high-frequency trading “arms race”. *Quarterly Journal of Economics* 137(1), 493–564.
- [34] Dean, J., Barroso, L. A. (2013). The tail at scale. *Communications of the ACM* 56(2), 74–80.
- [35] Zhao, K., Alizadeh, M., Pan, R., Ports, D. R. K., Perry, J., Yang, X., et al. (2023). Parsimon: scalable tail latency estimation for data center networks. NSDI 2023.
- [36] Maciejewski, V. (2026). Adapting the actor model of concurrency for high-frequency trading: synchronous message delivery (`fast_send`) and a tick-to-book latency study. arXiv:2609.21173. <https://arxiv.org/abs/2609.21173>
- [37] Maciejewski, V. (2026). KASPAR-HFT: an open-source C++20 actor framework and trading system for CME futures. Source repository. <https://github.com/vincent212/kaspar-hft>
- [38] Thompson, M. (2011). The Single Writer Principle. *Mechanical Sympathy*, 29 September 2011. <https://mechanical-sympathy.blogspot.com/2011/09/single-writer-principle.html>
- [39] Fowler, M. (2011). The LMAX Architecture. 12 July 2011. <https://martinfowler.com/articles/lmax.html>
- [40] LMAX Exchange (2011). Disruptor: high-performance alternative to bounded queues for exchanging data between concurrent threads. <https://lmax-exchange.github.io/disruptor/>
- [41] Rigtorp, E. (2020). Low latency tuning guide. <https://rigtorp.se/low-latency-guide/>
- [42] Chronicle Software (2023). Building fast trading engines: Chronicle’s approach to low-latency trading. <https://chronicle.software/insights/blogs/building-fast-trading-engines-chronicles-approach-to-low-latency-trading>
- [43] Real Logic (2016). Aeron: efficient reliable UDP unicast, UDP multicast, and IPC message transport. <https://aeron.io/>
- [44] Kwan, A., Wang, H., Luk, W. (2023). C++ design patterns for low-latency applications including high-frequency trading. arXiv 2309.04259.

- [45] Hu, W. (2024). The deterministic event-driven sequencer architecture: a competitive edge for high-frequency trading. Medium, October 2024. <https://medium.com/@hu.wenzhe124124/the-deterministic-event-driven-sequencer-architecture-a-competitive-edge-for-high-frequency-371cbfbe9c2f>
- [46] Schmidt, D. C. (1995). Reactor: an object behavioral pattern for concurrent event demultiplexing and event handler dispatching. In *Pattern Languages of Program Design*, J. O. Coplien and D. C. Schmidt (eds.), Addison-Wesley.
- [47] Kegel, D. (1999). The C10K problem. <https://www.kegel.com/c10k.html>
- [48] Sysoev, I. (2004). nginx: a high-performance HTTP server and reverse proxy. <https://nginx.org/>
- [49] Dahl, R. (2009). Node.js: JavaScript runtime built on the V8 engine. Presented at JSConf EU, Berlin, November 2009. <https://nodejs.org/>
- [50] Welsh, M., Culler, D., Brewer, E. (2001). SEDA: an architecture for well-conditioned, scalable internet services. *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, 230–243.
- [51] Welsh, M. (2010). A retrospective on SEDA. <https://matt-welsh.blogspot.com/2010/07/retrospective-on-seda.html>
- [52] Kohler, E., Morris, R., Chen, B., Jannotti, J., Kaashoek, M. F. (2000). The Click modular router. *ACM Transactions on Computer Systems* 18(3), 263–297.
- [53] Murray, D. G., McSherry, F., Isaacs, R., Isard, M., Barham, P., Abadi, M. (2013). Naiad: a timely dataflow system. *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, 439–455.
- [54] Marty, M., de Kruijf, M., Adriaens, J., Alfeld, C., et al. (2019). Snap: a microkernel approach to host networking. *Proc. 27th ACM Symposium on Operating Systems Principles (SOSP)*, 399–413.
- [55] Kaffes, K., Chong, T., Humphries, J. T., Belay, A., Mazières, D., Kozyrakis, C. (2019). Shinjuku: preemptive scheduling for μ second-scale tail latency. *Proc. 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 345–359.
- [56] Ousterhout, A., Fried, J., Behrens, J., Belay, A., Balakrishnan, H. (2019). Shenango: achieving high CPU efficiency for latency-sensitive datacenter workloads. *Proc. 16th USENIX NSDI*, 361–378.
- [57] Fried, J., Ruan, Z., Ousterhout, A., Belay, A. (2020). Caladan: mitigating interference at microsecond timescales. *Proc. 14th USENIX OSDI*, 281–297.
- [58] Chen, H., Delimitrou, C., Kozyrakis, C. (2021). Perséphone: a microsecond-scale time-sharing scheduler for wide service-time distributions. *Proc. 28th ACM SOSP*, 653–668.
- [59] DPDK Project (2024). DPDK Programmer’s Guide: run-to-completion and pipeline modes. https://doc.dpdk.org/guides/prog_guide/
- [60] Zhang, X. (2016). Pipelined FAST protocol market-data decoder for high-frequency trading. Chinese patent CN105654383A.

- [61] Vinnikov, A. (2014). Batch latency analysis and phase transitions for a tandem of queues. arXiv 1403.2400.
- [62] David, T., Guerraoui, R., Trigonakis, V. (2013). Everything you always wanted to know about synchronization but were afraid to ask. *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, 33–48.
- [63] Belay, A., Prekas, G., Klimovic, A., Grossman, S., Kozyrakis, C., Bugnion, E. (2014). IX: a protected dataplane operating system for high throughput and low latency. *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 49–65.

Appendices

A The Borel batch-size tail at fixed branching ratio

This appendix records the calculation behind Equation (8) and the numerical check that the critical $k^{-1/2}$ survival tail of the Borel distribution is confined to a finite window at every $n < 1$.

Stirling's formula $k! = \sqrt{2\pi k} k^k e^{-k} (1 + O(1/k))$ in Equation (7) gives

$$\Pr(K = k) = \frac{(nk)^{k-1} e^{-nk}}{k!} = \frac{n^{k-1} k^{k-1} e^{-nk}}{\sqrt{2\pi k} k^k e^{-k}} (1 + O(1/k)) = \frac{1}{n\sqrt{2\pi}} k^{-3/2} (ne^{1-n})^k (1 + O(1/k)).$$

Writing $n = 1 - \epsilon$, $\log(ne^{1-n}) = \log(1 - \epsilon) + \epsilon = -\epsilon^2/2 - \epsilon^3/3 - \dots$, so the geometric factor is $\exp(-k\epsilon^2/2(1 + O(\epsilon)))$. The pmf is therefore $\propto k^{-3/2}$ for $k \ll k^* = \epsilon^{-2}$ and geometric beyond; summing, $\Pr(K > k) \asymp k^{-1/2}$ on the same window and $\asymp k^{-3/2} e^{-k\epsilon^2/2}$ beyond it. At $n = 1$ exactly, $k^* = \infty$, $\Pr(K > k) \sim \sqrt{2/\pi} k^{-1/2}$ (Borel–Tanner), and $E[K] = \infty$; that is the unstable case $\rho = \infty$, which no stationary queue attains.

Because the tail is geometric, K has finite exponential moments $E[e^{\theta K}] < \infty$ for $\theta < \epsilon^2/2(1 + O(\epsilon))$ and is therefore outside the subexponential class $\mathcal{S}$. Heavy-tail queueing results that assume subexponential batch work, including the single-big-batch principle for $M^{[K]}/G/1$ and the monotone-separable tail theorem of Baccelli et al. [14], do not apply, and the stationary waiting time of the batch-arrival queue has a Cramér–Lundberg (exponential) tail at every fixed $n < 1$.

Table 30 reports the local log-log slope of the exact pmf, $\Delta \log \Pr(K = k) / \Delta \log k$ between successive k , which a pure $k^{-3/2}$ law would hold at -1.5 .

n	k^*	$10 \rightarrow 30$	$30 \rightarrow 100$	$100 \rightarrow 300$	$300 \rightarrow 1000$	$1000 \rightarrow 3000$
0.80	25	−1.92	−2.84	−5.71	−14.96	−43.63
0.90	100	−1.59	−1.81	−2.48	−4.62	−11.26
0.95	400	−1.52	−1.57	−1.73	−2.25	−3.85
0.99	10000	−1.50	−1.50	−1.51	−1.53	−1.59

Table 30. Local log-log slope of the Borel pmf, Equation (7), between successive cluster sizes k . The critical slope -1.5 holds while $k \ll k^* = (1 - n)^{-2}$ and steepens geometrically beyond.

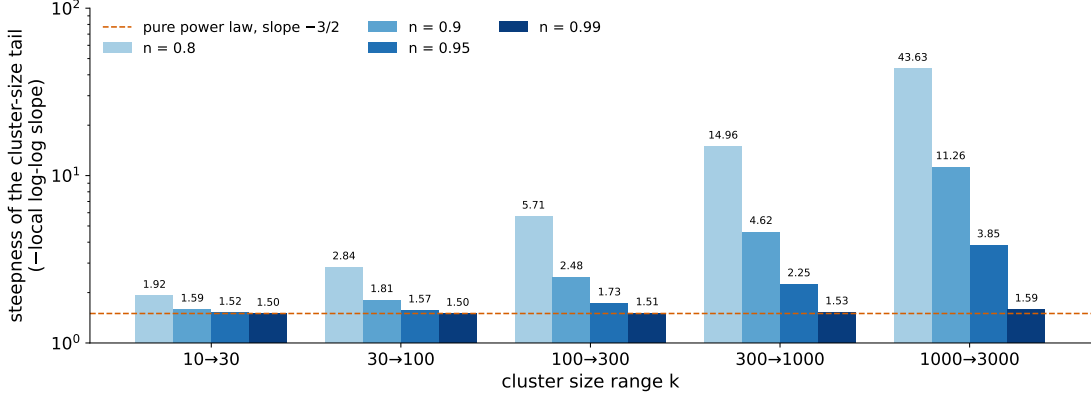


Figure 30. Table 30 as bars, log scale: how steeply the cluster-size distribution falls over successive size ranges, for four branching ratios. Near criticality ($n = 0.99$) it keeps the power-law slope of $3/2$ (dashed) over the whole range; at $n = 0.8$ it falls off exponentially beyond about 25, so very large clusters are rare.

The practical reading is that at the branching ratios fitted on the corpus ($n_{\text{pkt}} \approx 0.8$, Section 3.2), $k^* \approx 25$: clusters of order a hundred packets are the largest that occur with non-negligible frequency, and the single-stage wait behind the largest clusters has scale $Tk^* \approx 25T$, against the corpus-median p_{99} tail excess of Section 4, which is zero below the publisher period and rises from $1.2T$ at $T = 16\mu\text{s}$ to $39T$ at $T = 128\mu\text{s}$. This is the object that Theorem 1 divides by N , and the reason the $n = 0.80$ row of Table 30 is the relevant one.

B Numerical check of Theorem 1

A direct simulation of the model of Theorem 1 checks both the identity Equation (4) and the bound Equation (6). Immigrants arrive as a Poisson process at rate $\mu = 0.004\mu\text{s}^{-1}$, each spawning a Galton–Watson cluster with Poisson(0.9) offspring, so $K \sim \text{Borel}(0.9)$ and $E[K] \approx 10$; clusters are placed as instantaneous batches, giving $\rho = \lambda T \approx 0.40$ at $T = 10\mu\text{s}$. The tandem has $N = 4$ equal stages of $2.5\mu\text{s}$ and $h = 1.7\mu\text{s}$; 2×10^5 messages are simulated with an explicit per-stage FIFO recursion (independent of the paper’s sweep code). Over all messages, $\max_j |W_j^{(4)} - (w_j(T/4) + 3h)| < 10^{-9}\mu\text{s}$ and Equation (6) is violated by no message. The quantiles are

q	$W_q^{(1)}$	$W_q^{(4)}$	$W_q^{(1)}/4 + 3h$	$(W_q^{(4)} - 3h)/W_q^{(1)}$
0.5	350.0	55.1	92.6	0.14
0.9	2066.6	355.1	521.8	0.17
0.99	5342.1	1127.6	1340.6	0.21
0.999	7586.2	1585.1	1901.7	0.21

in microseconds. The tandem lands below the bound at every quantile, with the ratio in the last column between 0.14 and 0.21 against the bound’s 0.25; the slack is the ρ/N utilisation effect described in the remark after Theorem 1. Under the Poisson null at the same message rate ($\rho = 0.40$, which is far above the corpus utilisation and is used here only to make the Poisson wait visible) the single-stage wait quantiles at $q = 0.9, 0.99, 0.999$ are $10.3, 24.8, 38.5\mu\text{s}$ and the tandem’s are $5.1, 7.5, 9.2\mu\text{s}$, again inside the bound. At the corpus utilisation ($\rho = 0.0045$ at this appendix’s $T = 10\mu\text{s}$) the same two collapse to 0 and $3h$ at $q = 0.9$ and $q = 0.99$, which is Corollary 2; at $q = 0.999$ they do not, since ρ there exceeds $1 - q = 0.001$ and the corollary does not apply.

C Numerical check of the unequal-stage claims

The same generator as Appendix B (Borel(0.9) clusters, $\rho = 0.40$, $T = 10 \mu\text{s}$, $h = 1.7 \mu\text{s}$), with each cluster’s members spread uniformly over $3 \mu\text{s}$ instead of placed as an instantaneous batch and 4×10^5 messages, run through the partitions of Remark 2.3 with an explicit per-stage FIFO recursion. The last two columns are the tail-excess ratio $(\Delta_{99}(P) - \text{hops} \cdot h) / \Delta_{99}(\text{single})$ and the bound $s_{\max}/T$ from Theorem 1.

partition	$s_{\max}$	hops	p_{50}	p_{99}	$p_{99.9}$	ratio	bound
10	10.0	0	388.2	5719.0	8249.3	1.00	1.00
7 + 3	7.0	1	213.6	3429.5	5297.1	0.60	0.70
3 + 7	7.0	1	213.6	3429.5	5297.1	0.60	0.70
5 + 5	5.0	1	134.9	2265.5	3574.2	0.39	0.50
7 + 2 + 1	7.0	2	215.3	3431.2	5298.8	0.60	0.70
4 + 3 + 3	4.0	2	104.6	1746.5	2714.6	0.30	0.40
4 + 3 + 2 + 1	4.0	3	106.3	1748.2	2716.3	0.30	0.40
2.5×4	2.5	3	66.4	1049.8	1562.2	0.18	0.25

Latencies are end-to-end in microseconds. The three claims of Remark 2.3 read off directly: the tail excess depends on the partition only through $s_{\max}$ and sits below the bound at every row; 7 + 3 and 3 + 7 are identical to the reported precision; and 7 + 2 + 1 differs from 7 + 3, and 4 + 3 + 2 + 1 from 4 + 3 + 3, by exactly one hop cost $h = 1.7 \mu\text{s}$ at every quantile. The $3 \mu\text{s}$ intra-cluster spread, which is above $s_{\max}$ for the last row, is why the equal-stage ratio here (0.18) sits slightly below the batch-input value of Appendix B (0.21): a cluster that arrives spread out lets the faster tandem drain part of it before the next member lands.

D A synthetic feed on which the tail at $8 \mu\text{s}$ depends on intensity and branching ratio

Section 4.9 finds the normalised tail on CME MDP3 invariant to packet rate and branching ratio at HFT service times. This appendix constructs, with the same simulator, a feed on which both dependences appear at $T = 8 \mu\text{s}$, and identifies what distinguishes it from CME.

Arrivals are generated as a Hawkes process through its cluster representation: immigrants arrive as a Poisson process of rate $\mu = \bar{\lambda}(1 - n)$, and each spawns a Galton–Watson tree with Poisson(n) offspring at exponential delays of mean $1/\beta$. Three conditions are run. In the first the kernel is fast, $1/\beta = 1 \mu\text{s}$, and there is no minimum period. In the second the same fast kernel is followed by a minimum period equal to the publisher period, $s_{\text{pub}} = 7.5 \mu\text{s}$: each arrival is delayed until at least s_{pub} after its predecessor, modelling the exchange’s publisher (Section 5.4). In the third the kernel is slow, $1/\beta = 80 \mu\text{s}$, with the same spacing; this is the condition closest to the CME gap structure, though the kernel fitted on the corpus is slower still, about $160 \mu\text{s}$ (Section 3.2). The grid is $n \in \{0.5, 0.7, 0.8, 0.9, 0.95\}$ and $\bar{\lambda} \in \{500, 2000, 8000, 32000, 64000\}$ packets per second, so that $\rho = \bar{\lambda}T$ at $T = 8 \mu\text{s}$ runs from 0.004 to 0.51; 2×10^5 events per cell, three seeds, median reported. The last column of Table 31 is the fraction of interarrival gaps at the minimum period, measured at $\bar{\lambda} = 8000$; on the CME corpus 2.15% of packet gaps are shorter than $7.9 \mu\text{s}$, just above the publisher period (Table 6).

p_{99}/T at $T = 8 \mu s, N = 1$						gaps at s_{pub}
n	$\bar{\lambda}=500$	2000	8000	32000	64000	at $\bar{\lambda}=8000$
<i>fast kernel, no minimum period</i>						
0.50	12.2	12.5	13.2	15.5	22.6	52%
0.70	38.9	40.7	41.2	47.3	66.6	71%
0.80	91.4	100.0	97.1	105.0	140.7	80%
0.90	421.3	429.4	411.3	413.7	620.4	90%
0.95	1544.2	1482.6	1721.3	1802.4	2487.2	95%
<i>fast kernel, publisher period</i>						
0.50	1.75	1.75	1.88	2.44	4.38	53%
0.70	3.50	3.62	3.88	5.44	11.87	72%
0.80	6.81	7.38	7.56	10.88	23.12	81%
0.90	28.06	28.50	30.44	41.25	100.19	91%
0.95	98.06	94.75	116.36	193.96	382.18	95%
<i>slow kernel, publisher period</i>						
0.50	1.12	1.12	1.19	1.51	3.56	13%
0.70	1.25	1.31	1.50	3.18	10.44	20%
0.80	2.38	2.71	3.12	8.12	19.56	28%
0.90	23.50	22.57	26.25	37.25	95.75	47%
0.95	90.12	86.19	112.19	188.78	364.30	67%
<i>Poisson, matched rate</i>						
—	1.00	1.39	1.87	2.69	4.45	—
ρ at $T = 8$						
	0.016	0.064	0.256	0.512		

Table 31. Synthetic Hawkes feeds at $T = 8 \mu s$. The ρ row gives $\bar{\lambda}T$ for the last four columns; the first column is $\rho = 0.004$.

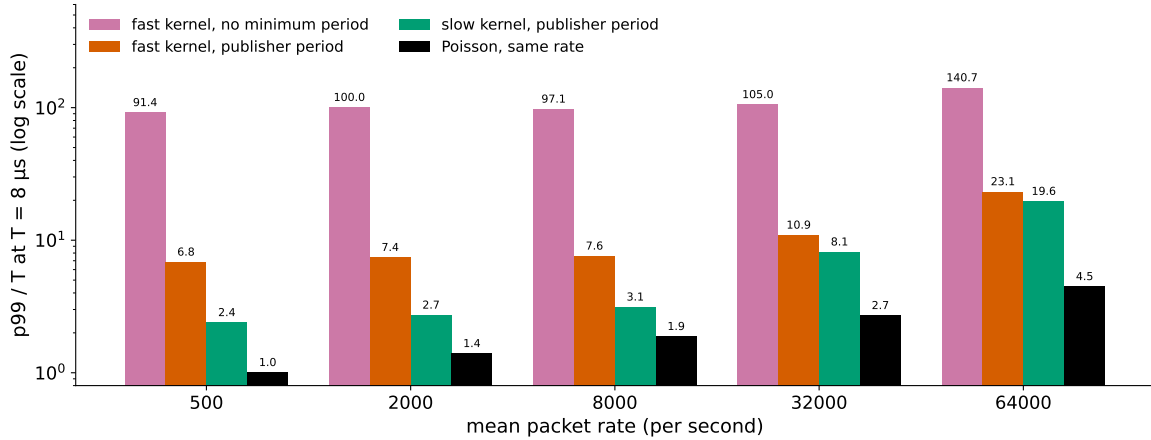


Figure 31. Table 31 as bars, log scale, for branching ratio 0.8: the normalised tail p_{99}/T at $T = 8 \mu s$ on synthetic feeds, by packet rate. A fast kernel with no minimum period (pink) has a tail of about a hundred times T ; adding a minimum period equal to the publisher period (vermillion) cuts it more than tenfold; a slow kernel with the spacing (green) is lower still, closer to the Poisson stream (black). What sets the tail at $8 \mu s$ is how many gaps sit at the minimum period, which the kernel and the spacing control and the branching ratio alone does not.

Three observations.

Intensity. In every Hawkes condition the tail is flat in $\bar{\lambda}$ at $\rho \leq 0.02$ (the Poisson row rises from 1.00 to 1.39 over the same range) and rises from $\rho \approx 0.06$, steeply by $\rho = 0.5$. The CME corpus at $T = 8 \mu\text{s}$ spans $\rho = 0.002$ to 0.008 between its quietest and busiest quintiles, which is inside the flat region of every synthetic condition. The intensity invariance of Section 4.9 is therefore what a Hawkes model predicts at that utilisation, and it ends where utilisation queueing begins, as the CME sweep also finds at $T \geq 64 \mu\text{s}$.

Criticality. The synthetic feeds depend strongly on n in every condition, by a factor of 55 to 130 between $n = 0.5$ and $n = 0.95$, and by a factor of 1.9 to 2.6 between $n = 0.7$ and $n = 0.8$ alone. The CME corpus shows no dependence over its observed range $n = 0.77$ –0.83. The last column explains the difference. A Hawkes process at $n = 0.8$ places 28% to 81% of its interarrival gaps at the minimum period, depending on the kernel; on the CME corpus about 2% of gaps are that close. On the synthetic feeds the branching ratio controls how many arrivals are packed at the minimum period, and that packing is what queues on an $8 \mu\text{s}$ server. On CME the packed fraction is set by the publisher, is about 1%, and does not vary with the fitted n over the observed range; the fitted n describes the ordering of arrivals at longer timescales, consistent with Section 3.2, where the corpus is found to be self-exciting with heavy-tailed gaps rather than a pure exponential-kernel Hawkes process.

Feed dependence of the design rule. At $T = 8 \mu\text{s}$, $n = 0.8$, $\bar{\lambda} = 8000$, a two-stage tandem has the lower p_{99} on every synthetic condition: $97.1 \rightarrow 47.4$ without a minimum period, $7.56 \rightarrow 1.21$ with the fast kernel and spacing, $3.12 \rightarrow 1.21$ with the slow kernel and spacing. On CME at the same T , $N^* = 1$ (Section 4.8). The service time at which splitting begins to pay is not a constant; it is a property of the feed.

Summary. Whether splitting a stage reduces its tail depends on how many packets arrive closer together than the stage's service time, not on the packet rate or the branching ratio as such. On CME about 2% of packet gaps are shorter than $7.9 \mu\text{s}$, just above the publisher period. That fraction does not change when the market becomes busier, since a busier market produces more clusters with the same spacing inside them, nor when the fitted branching ratio moves within its observed range. At the 7 – $8 \mu\text{s}$ service time of a production decode path there is therefore little to queue behind across packets, and one thread is the right design as far as that queue goes; the tail from long packets and variable service at that service time is separate (Sections 7.6 and 7.7). On a feed that places a large share of its gaps at the minimum period, the result inverts: a Hawkes process at the same $n = 0.8$ places 28–81% of its gaps at the minimum period instead of about 2%, the tail depends strongly on the branching ratio, splitting reduces the tail at $T = 8 \mu\text{s}$ where on CME it does not, and at high enough load the packet rate matters as well. The Poisson null shows only the load effect, and only at high utilisation, as a null should. The rule is therefore neither one thread nor two. The split threshold, the service time above which cutting a stage lowers its p_{99} , is a property of the feed's packet spacing; it cannot be read off the fitted parameters, and it must be measured under the load the system will see. On the CME feed a $7 \mu\text{s}$ stage sits below the split threshold of 9 – $10 \mu\text{s}$; another exchange, or the same exchange after a change to its publisher, may not. Section 12 states this as a design principle.